Herald

Herald — Specification V4

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active — supersedes V3 (2026-05-31)
Supersedes	`docs/specs/mvp/archive/specification.V3.md` (V3 r13, 2026-05-28). V4 is a strict **superset**: it preserves every V3 section and weaves in two new mandatory features — **participant identity + workable-item attribution + notification @-tagging** (§7.6, §8.4, §32.6.A, §34.7) and **natural-language intent recognition + clarify-and-tag** (§32.6.B, §33.8, §34.8). Normative detail lives in the two design contracts `docs/design/PARTICIPANT_ATTRIBUTION.md` + `docs/design/INTENT_RECOGNITION.md` and the inherited constitution anchors §11.4.104 + §11.4.105.
What changed from V3	**(1) Participant identity (Subscribers/Users):** §7.6 formalizes the logical `subscribers` + per-channel `subscriber_aliases.username` model — one person carries a DIFFERENT `@username` per messenger; the HERALD_<CHANNEL>_OPERATOR_USERNAME operator designation (env-var, not a DB flag); and the `Claude` system-agent sentinel handle. **(2) Workable-item attribution:** §8.4 adds `created_by` + `assigned_to` (canonical-handle strings) to every workable item with the attribution rules (CLI-prompt→Operator / system-detected→`Claude` / received-message→@sender; `assigned_to` default = Operator). **(3) Notification @-tagging matrix:** §34.7 adds the outbound tagging rules (tag `assigned_to` if human≠Operator; tag `created_by` if human≠Operator≠`Claude`; Operator+`Claude` never tagged; skip if the participant has no alias on the target channel). **(4) Natural-language intent recognition:** §32.6.B + §33.8 + §34.8 add the three-tier resolution (Tier-1 deterministic `CommandRecognizer`, Tier-2 LLM intent inference via the dispatch envelope, Tier-3 clarify-and-tag), the recognized command set, and the new `clarify` <<<HERALD-REPLY>>> action — "no command syntax required / never guess / never ignore". As-built status per feature is annotated inline (LIVE vs contract-being-implemented). Full V3→V4 changelog: §29.3.
V3 status summary (carried)	**V3 r13 — Wave 7 generic messenger framework: Slack lands.** §11.0 gains the Wave 7 inbound-runtime extension note (the richer `commons_messaging/channels.Channel` interface embedding `commons.Channel` +

Field	Value
	SendReplyGeneric + BotSelfIdentity + DownloadAttachment + the init()-registered name→constructor registry + generalized IsSelfEcho across IdentityUsername/IdentityUserID/IdentityAddress + per-channel ~/.herald/inbox/<channel>/<sha256>.<ext>). §32.2 marks Telegram **LIVE (Wave 6 / Wave 7)** + Slack **LIVE (Wave 7 T6 — Socket Mode)**. §43 catalogue gains the multi-channel pherald listen row (HERALD_CHANNELS=tgram,slack, per-channel namespaced env, one Subscribe goroutine per enabled channel into one dispatcher). Slack ships as a Socket Mode adapter (commons_messaging/channels/slack/, 5 source files + hermetic httptest tests, 22 PASS -race), vendored via submodules/slack-go @ v0.16.0 (commit 209ec09). qaherald gains a parallel raw-HTTP Slack MessengerClient + channel-keyed Build() dispatcher (qaherald/internal/messenger/{slack.go,builder.go}, 30 tests PASS -race — 17 new + 13 pre-existing tgram, zero regression; xoxb token NEVER in errors — sanitizeError() + TestSlack_Send_ErrorDoesNotLeakToken pin it). Per docs/superpowers/plans/2026-05-27-wave7-generic-messenger.md T6-T8. Workspace module count unchanged at 16. **Prior r12 — Wave 6 lands the pherald INBOUND runtime (closed-loop subscriber → CC → reply).** New pherald listen Cobra subcommand runs Telegram getUpdates long-poll (OnText/OnPhoto/OnDocument/OnVoice handlers) + bot self-filter (bot.Me.Username anti-echo guard captured at Subscribe construction; refuses to start if getMe returned no username) + sha256 content-addressed attachment download into ~/.herald/inbox/<sha256>.<ext> (idempotent — duplicate downloads do NOT re-write) + Claude Code dispatch with Opus pinned in the spawned argv (claude --model claude-opus-4-7 --resume <UUID> --print <envelope>) + envelope pre-text — verbatim operator wording "We have received new message from our communication channel <channel-name>. <classification sentence>. <attachment list>" preceding the existing <<<HERALD-DISPATCH-v1>>> structured block + <<<HERALD-REPLY>>> parser with action routing (reply default / issue.open / event.emit) + tgram.Adapter.SendReply(ctx, chatID, text, replyToID, attachments) wiring reply_to_message_id from the original message_id. --qa-out-dir <path> flag journals every inbound + outbound + CC dispatch event as JSONL per §107.x. Three new spec subsections capture the substrate: §32.9 (bot self-filter), §33.6 (envelope pre-text — verbatim operator mandate), §33.7 (Opus model pin). §43 command catalogue gains pherald listen row. Workspace at 16 Go modules (Wave 5 added qaherald; Wave 6 added no new module — inbound is pherald/internal/inbound/). 12 Wave-6 commits T1=ad87d7f..T12=96c7c6b. 8 new e2e invariants E63-E70 land in this revision (currently SKIP-with-documented-reason — auto-convert to PASS once T10b lands operator-supplied live evidence under docs/qa/HRD-100-<run-id>/). Wave 6 mutation gate tests/test_wave6_mutation_meta.sh (3 paired hermetic mutations, 4/4 PASS). Tag **v0.4.0 deferred to T13b** (operator-supplied live evidence per §107.x). Prior r11 — Wave 4b lands TOON (Token-Oriented Object Notation, application/toon) content negotiation as Herald's preferred wire format with JSON as honest fallback. Every /v1/* business route now honors Accept: application/toon (responses are real TOON bytes via the digital.vasic.toon submodule + toon-format/toon-go upstream); Accept: application/json keeps JSON; q-value preference honored; */* and missing Accept go to the server default (TOON; HERALD_DEFAULT_RESPONSE_CODEC=json flips). POST /v1/events

Field	Value
	accepts both TOON and JSON request bodies (TOON transcoded to JSON before the Runner's EventParser; Runner stays codec-agnostic). TOONMiddleware auto-wired into `cli.RunServe` so every flavor binary (`pherald/cherald/sherald/bherald/rherald/iherald/scherald`) inherits for free. Real compression evidence — for sherald /v1/safety_state, TOON body is 215B vs 226B JSON (~5% saved on a tiny payload; larger tabular payloads see 30-60% per upstream measurements). 7 new e2e invariants (E56-E62; E62 SKIP-with-reason — encoder-failure fallback is unit-tested in `commons/cli/toon_test.go` because no Herald handler exposes a TOON-unencodable Go type). New `tests/test_wave4b_mutation_meta.sh` (5/5 PASS — M1 strip TOONMiddleware → E57 FAIL; M2 swap `toon.Marshal` → `json.Marshal` (2026-05-17 PASS-bluff recurrence) → E60 FAIL; M3 no-op transcodeRequestBody → pherald unit test FAIL; working-tree quiescence check; post-flight green). Tag **v0.3.0**. Workspace module count unchanged at 15. Prior r10 — Wave 4a HTTP/3 + Brotli + Alt-Svc + TLS 1.3 transport substrate; tag v0.2.0. Prior r9 — Wave 3b pherald `/v1/events` LIVE. r8 — Wave 3a substrate.
Issues	none
Issues summary	—
Fixed	V3-R9-01..V3-R9-04 (r9 Wave 3b — Runner live + e2e E37-E42 + mutation gate M2/M3/M4 + §32 implementation column + §44.N milestone); V3-R8-01..V3-R8-05 (r8 Wave 2 scaffolds + Branding extension + REST surface expansion + §44.M milestone + §43 catalogue HRD-092 entry); V3-R3-01..V3-R3-03 (r3 parent-doc spec-path sync); V3-R2-01..V3-R2-09 (r2); V3-R1-01..V3-R1-14 (r1); inherits closed V2 + V1 lineage.
Fixed summary	r9 captures Wave 3b as-built — pherald `/v1/events` is live (7-stage Runner orchestrator wired against real Postgres + Redis + commons_auth JWT verifier; 23 runner package tests green + 6 new e2e invariants E37-E42; HRD-016 closes atomic Issues → Fixed; cross-binary E45 still SKIP-with-reason pending Wave 3c wiring). r8 captures Wave 2 as-built — shared `commons/cli/` scaffold + 7 flavor binaries (1 refactored, 6 net new), Branding struct extended with 5 per-flavor identity fields, §41 REST surface gains `/v1/compliance` (cherald), `/v1/safety_state` (sherald), `/v1/webhooks/page` (iherald), §44.M Wave 2 milestone subsection records 33-invariant e2e + 4/4 mutation meta + HRD-092 catalogue-check (no-match → vendor). r7 lands §44 Foundation contract + records M1 as-built evidence (commons_constitution 14-file scaffold, in-process EventBus/Registry/Captureer/Ladder/Store/Runner all green under -race; full workspace inheritance gate 12 PASS / 0 FAIL).
Continuation	Wave 3c+: cross-binary E45 wiring (pherald deny POST → cherald GET /v1/compliance round-trip), HRD-024 iherald `/v1/webhooks/page` live, HRD-098 sherald `/v1/safety_state` daemon live, plus the 28 §43 command implementations (HRD-029..056), parallel-fan-out optimization, OpenAPI surface, INTEGRATION.md, v0.1.0 release tag. Each milestone followed by multi-mirror push. Then codegraph integration per https://github.com/colbymchenry/codegraph.

The **bi-directional event fan-out** system: Herald ingests events from heterogeneous sources and reliably fans them out to multiple notification channels so every alert reaches the right destination without confusion, and processes inbound replies/commands back from subscribers in a structured, security-validated way.

Table of contents

---

§1. Upstreams

All existing project upstreams:

- **GitHub** (main repository): `git@github.com:vasic-digital/Herald.git`
- **GitLab**: `git@gitlab.com:vasic-digital/herald.git`
- **GitFlic**: `git@gitflic.ru:vasic-digital/herald.git`
- **GitVerse**: `git@gitverse.ru:vasic-digital/Herald.git`

The local `origin` remote is a fan-out (one fetch URL + four push URLs). A single `git push origin <branch>` propagates to all four hosts (Helix Constitution §2.1 / Herald Constitution §103).

---

§2. Mission and scope

2.1 What Herald is

Herald is the **bi-directional event fan-out mechanism**. It receives input from one or more **sources** and dispatches the resulting content to one or more **destinations** (channels). Depending on the implementation (Flavor) of Herald, sources and destinations are heterogeneous: a single input type or many; a single output channel or many.

For example, the input may be the result of a CI pipeline execution (a build report, a test summary, a security-scan finding). Herald enriches, normalizes, routes, and dispatches that input to messaging channels (Telegram, Slack, Max, Email, Markdown diary, etc.) so that human and machine subscribers can be informed and can interact back.

The possibilities are not limited. The structure of the system MUST be **hierarchical**: shared abstractions live in the **commons** layers (closest to the root); flavor-specific implementations live in `flavors/<flavor>/`.

> **Note:** We MUST NOT be obligated to follow this hierarchical structure rigidly when a parent project's specific custom flavor must exist privately or in a different location. Flexibility is mandatory and fully supported.

2.2 What Herald is NOT

To bound scope (§102 Herald Constitution — mission boundary):

- Herald is **not** a general-purpose chat application; it is an event-driven fan-out + inbound-reply processor.
- Herald is **not** a general-purpose monitoring/observability platform — it integrates *with* them via webhook ingestion (Prometheus Alertmanager, Grafana, Datadog, PagerDuty, OpsGenie) but does not replace them.
- Herald is **not** a transactional/marketing email service provider in itself — it integrates *with* ESPs (SendGrid, Resend, Postmark) as one of its delivery transports.
- Herald is **not** a workflow orchestration platform — Temporal/Argo/Step Functions remain the right tool for that; Herald can be triggered by them and can dispatch notifications about them.

2.3 Architecture diagram

```
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│  CI / build  │ │  monitoring  │ │   cron job   │ │  AI agent /  │
│   pipeline   │ │ alertmanager │ │   scheduler  │ │  CLI invoker │
└──────────────┘ └──────────────┘ └──────────────┘ └──────────────┘
       │ HTTP webhook    │ HTTP webhook    │ `<flavor>herald   │ `<flavor>
       │ + CloudEvent    │ + CloudEvent    │   send …`         │   send …`
       v                 v                 v                   v
┌──────────────────────────────────────────────────────────────────┐
│                  Herald flavor binary (one of                    │
│                  pherald, sherald, bherald, …)                   │
│                                                                  │
│   ┌─────────────┐     ┌──────────────┐      ┌──────────────┐     │       │
│   │   ingress   │ ──▶ │  Watermill   │ ──▶  │   channel    │     │       │
│   │ (HTTP/CLI)  │     │   router +   │      │   adapters   │     │
│   │  + HMAC     │     │  middleware  │      │              │     │
│   └─────────────┘     │  (retry +    │      │   tgram://   │     │
│                       │   throttle + │      │   slack://   │     │
│   ┌─────────────┐     │   dedup +    │      │   max://     │     │
│   │ subscriber  │     │   trace +    │      │   mailto://  │     │
│   │   replies   │ ◀── ┤   meter)     │      │   discord:// │     │       │
│   │ (long-poll  │     └──────────────┘      │   teams://   │     │
│   │  / webhook) │            │              │   ntfy://    │     │
│   └─────────────┘            v              │   diary://   │     │
│         │            ┌──────────────┐       │   webhook:// │     │
│         │            │  Postgres +  │       └──────────────┘     │
│         └────────────│  River queue │              │             │
│                      │  + RLS       │              v             │
│                      └──────────────┘       ┌──────────────┐     │
│                                             │  subscribers │     │
│                                             │  + channels  │     │
│                                             │ (per tenant) │     │
│                                             └──────────────┘     │
└──────────────────────────────────────────────────────────────────┘
                             │
                             v
```

```
docs/herald/diary/main.{md,pdf,html}
(every in/out message appended)
```

---

§3. Execution model

3.1 Single binary, two modes

Every Herald flavor is a **single statically-linked Go binary** with **Cobra-style subcommands** (per R-02 — cleanest fit, lowest deployment footprint, no duplicate auth/config plumbing). The same binary serves both runtime modes:

- **One-shot mode** — `<flavor>herald send …`: connects, transmits a single event, awaits delivery acknowledgement with a configurable deadline, exits non-zero on failure. Designed for CI integration, cron jobs, AI-agent invocation, pipeline steps.
- **Daemon mode** — `<flavor>herald serve`: long-running listener; blocks on (a) the HTTP ingress for webhook events, (b) per-channel subscriber-reply loops (Telegram long-poll, Slack Socket Mode, Email IMAP, Discord gateway, etc.), (c) scheduled jobs from the River queue.

Both modes share the same configuration loader, channel adapters, storage layer, and observability stack — only the entry point and process lifecycle differ.

Graceful shutdown semantics. Both modes MUST:

1. Trap `SIGTERM` and `SIGINT`.
2. Stop accepting new ingress requests immediately (`/readyz` returns 503).
3. Drain in-flight work with a configurable grace period (default 30 s, env `HERALD_SHUTDOWN_GRACE`).
4. Refuse `Send()` calls that arrive after grace begins (returning `context.Canceled`).
5. Flush the OpenTelemetry exporter (`tracerProvider.Shutdown(ctx)` with its own 5 s sub-deadline).
6. `Close()` the Postgres pool and Redis client.
7. Exit with code `0` if drain completed cleanly, `1` if grace expired with work still in flight.

A second `SIGTERM` within the grace window forces immediate exit. `SIGKILL` is uncatchable by definition; operators MUST size the grace period for their workload's tail latency.

Additional subcommands:

- `<flavor>herald doctor` — verifies environment health (Postgres ping, Redis ping, channel-credential validity, DKIM/SPF/DMARC records, port availability).
- `<flavor>herald migrate` — applies database migrations (idempotent).
- `<flavor>herald deadletter [list | replay <id> | purge]` — operate on the dead-letter table.
- `<flavor>herald subscriber [list | add | verify <token>]` — manage subscribers.
- `<flavor>herald digest …` — generate scheduled summaries (digests, weekly reports).

3.2 Distribution + invocation surfaces

Herald applications are CLI binaries primarily designed for:

- **CI integration** — invoked from GitHub Actions / GitLab CI / Jenkins / CircleCI / Drone steps.
- **Pipelines** — invoked from build/deploy scripts.
- **AI CLI agents** — invoked by Claude Code, OpenCode, Cursor, Aider, etc., as a structured way to surface progress and accept human feedback.
- `cron` — scheduled checks and digests.
- **Webhook recipients** — running in daemon mode behind a reverse proxy.

Some Herald application names (illustrative — flavors are fully enumerated in §18):

- `pherald` for **Project Herald** (development-lifecycle events).
- `sherald` for **System Herald** (OS/host/service events).
- `bherald` for **Build Herald** (CI/CD build events).
- `dherald` for **Deploy Herald** (release/deploy events).
- `aherald` for **Alert Herald** (monitoring alert routing).
- `scherald` for **Schedule Herald** (cron/scheduled-task notifications).
- `iherald` for **Incident Herald** (incident management).
- `rherald` for **Release Herald** (release lifecycle).
- `cherald` for **Compliance Herald** (audit/compliance events).

3.3 Configuration & credentials

Configuration precedence (12-factor aligned, per R-04):

1. Explicit CLI flag (highest precedence).
2. Shell-exported environment variable (from `.bashrc` / `.zshrc` / k8s env / Docker `-e`).
3. Value from `.env` (fallback only — does NOT override exported shell vars).
4. Compiled default (lowest).

`.env` MUST BE git-ignored. `.env.example` is the documented template, committed. An opt-in `--env-file-override` flag maps to `godotenv.Overload()` for the rare case operators want the file to win (one-off rescues, debugging).

Configuration file layout (`config.toml`, optional, loaded after `.env`):

```
[herald]
flavor        = "pherald"
tenant_id     = "00000000-0000-0000-0000-000000000001"  # default
        if multi-tenancy disabled
locale        = "en-US"
diary_root    = ""
        # blank → discover via parent-walk
admin_port    = 24090                                   # /
        livez, /readyz, /metrics, pprof
http_port     = 24091
        # webhook ingress + reply webhooks

[postgres]
```

```
dsn          = "${HERALD_POSTGRES_DSN}"                # env
        interpolation
max_conns    = 20
rls_enforced = true

[redis]
addr         = "${HERALD_REDIS_ADDR}"
acl_user     = "${HERALD_REDIS_USER}"
acl_password = "${HERALD_REDIS_PASSWORD}"

[channels]
# Apprise-style URLs (see §6.1)
default = [
  "tgram://${TELEGRAM_BOT_TOKEN}/${TELEGRAM_CHAT_ID}?
        tags=oncall,prod",
  "slack://${SLACK_TOKEN}/${SLACK_CHANNEL}?tags=oncall",
  "mailto://herald@example.com?tags=audit",
  "diary://?tags=*",  # always log to diary
]

[observability]
otlp_endpoint = "http://otel-collector:4317"
log_level     = "info"
```

3.4 Worker pools, concurrency, and hot-reload

Worker pool sizing. `herald serve` runs three independently-sized worker pools:

Pool	Default size	Configurable	Workload
HTTP ingress	`2 × NumCPU`	`[server].http_workers`	accepts inbound CloudEvents + webhook deliveries; CPU-bound on signature verification + JSON parsing
Router/ dispatch	`4 × NumCPU`	`[router].workers`	template render + preference filter + tag matching; mostly CPU + small Postgres lookups
River channel-delivery	`8 × NumCPU` (capped at `[river].max_workers`, default 64)	`[river].workers`	I/O-bound HTTP calls to channel APIs (Telegram, Slack, …); bottlenecked by upstream rate limits

Operators tune via env vars (`HERALD_HTTP_WORKERS`, `HERALD_ROUTER_WORKERS`, `HERALD_RIVER_WORKERS`) or `config.toml`. Sizing guidance:

- **Single-tenant small** (≤ 1 alert/sec): defaults (~16 workers total on a 2-vCPU host).

- **Multi-tenant production** (~50 alerts/sec sustained): bump `[river].max_workers` to `min(256, 8 × tenants)`; tune `[postgres].max_conns` to `[router].workers + [river].workers + 4` (admin connections reserved).
- **High-burst CI fanout** (~500 alerts/min in 10-sec spikes): rely on River's backpressure rather than over-provisioning workers; River queues durably in Postgres and drains as channels free up.

Hot-reload via SIGHUP. `herald serve` traps `SIGHUP` and re-reads `config.toml` + `.env` + `channel_addresses` (database-backed) without dropping in-flight work. Reload is constrained to **safe-to-change** sections:

Reloadable on SIGHUP	NOT reloadable (require process restart)
`[channels].*` (channel addresses + tags)	`[server].http_port` / admin port
`[router].workers` (resized live)	`[postgres].dsn` (connection identity)
`[river].workers` / `max_workers`	`[redis].addr`
`[observability].log_level`	`[observability].otlp_endpoint` (exporter identity)
`[security].allowlists.*`	binary version / migrations
`[rate_limits].*` (token bucket caps)	RLS policy changes (DB-side)

Reload semantics:

1. Compute the diff between in-memory config and freshly-loaded config.
2. Reject if any "NOT reloadable" key changed — log `ERROR config reload rejected: <key> requires restart`; old config remains active.
3. For reloadable keys, apply atomically (worker pools resize via a lockless swap; channel adapters re-initialize against new addresses; rate-limit buckets refresh with new caps; in-flight work continues against the live config snapshot it captured).
4. Emit `digital.vasic.herald.system.config.reloaded` event with the diff for audit.

No hot-reload in one-shot mode: `herald send` reads config once at startup and exits; SIGHUP is ignored.

3.5 Time / clock abstraction

Herald never calls `time.Now()` directly outside of `commons/clock`. The clock abstraction lets tests fast-forward time (essential for quiet-hours, batching windows, retry backoff, idempotency TTLs, escalation chains).

```
package commons // L0

// Clock is the time-source abstraction. Production uses RealClock;
// tests use FakeClock with controllable Now()/After()/NewTimer().
type Clock interface {
    Now() time.Time
    Since(t time.Time) time.Duration
    Sleep(d time.Duration)
```

```go
    After(d time.Duration) <-chan time.Time
    NewTimer(d time.Duration) Timer
    NewTicker(d time.Duration) Ticker
}

// RealClock wraps the stdlib time package. Used in production.
type RealClock struct{}

// FakeClock is the test implementation. Advance(d) moves the
        clock
// forward by d and fires any pending timers/tickers whose
        deadlines
// the advance crosses. Available in commons/clock/clocktest.
type FakeClock struct {
    // unexported state
}

// Default exports a process-global Clock — Herald's CLI
        bootstrap
// sets it to RealClock; tests swap it in TestMain.
var Default Clock = RealClock{}
```

Rationale: this is the same pattern that `github.com/benbjohnson/clock` popularised; `commons/clock` ships a minimal in-tree implementation rather than depending on a third-party module that has no maintainer activity since 2022. The interface surface is intentionally small — only what Herald needs.

Discipline: anyone calling `time.Now()` outside `commons/clock` is a bug — the lint rule `herald-no-direct-time-now` (custom `golangci-lint` analyzer, planned) flags violations.

---

§4. Event model & wire format

4.1 CloudEvents v1.0 as canonical envelope

Herald speaks **CloudEvents v1.0** (CNCF graduated, Jan 2024) natively on every external boundary — ingress, internal transport, audit/diary record. Inside the binary, events are passed as `cloudevents.Event` values from `github.com/cloudevents/sdk-go/v2`.

Required CloudEvents attributes for every Herald event:

Attribute	Source	Example
`specversion`	constant `"1.0"`	`"1.0"`
`id`	UUIDv7 (natural ordering by time)	`"01931a7c-3f4e-7000-9abc-def012345678"`
`source`	URI of the emitter	`"https://ci.example.com/pipelines/4231"`
`type`	reverse-DNS event type	`"digital.vasic.herald.ci.failed"`

Attribute	Source	Example
`time`	RFC 3339 timestamp	`"2026-05-19T17:30:00Z"`
`datacontenttype`	media type of `data`	`"application/json"`
`subject`	target hint (channel address, tag, route key)	`"tag:oncall,prod"` or `"channel:slack-incidents"`

Two wire modes accepted on HTTP ingress:

- **Structured mode** — single JSON object: `{"specversion":"1.0","id":"…","type":"…","data":{…}}`.
- **Binary mode** — HTTP headers carry attributes (`ce-id`, `ce-type`, `ce-source`, …), body is the raw `data` payload.

Herald-specific extension attributes:

- `heraldtenant` — tenant UUID (overrides default).
- `heraldidempotencykey` — explicit dedup key (overrides `id` if present).
- `heraldpriority` — `low` / `normal` / `high` / `urgent` (ntfy-style 1–5 mapping).

4.2 Herald event-type taxonomy

CloudEvents `type` is a reverse-DNS string. Herald's namespace is `digital.vasic.herald.*`.

Subtree	Used by	Examples
`digital.vasic.herald.ci.*`	`bherald`	`.build.queued`, `.build.started`,`.bui`
`digital.vasic.herald.project.*`	`pherald`	`.task.opened`, `.task.assigned`, `.task`
`digital.vasic.herald.system.*`	`sherald`	`.host.cpu_high`, `.host.disk_full`,`.s`
`digital.vasic.herald.deploy.*`	`dherald`	`.deploy.started`, `.deploy.succeede`
`digital.vasic.herald.alert.*`	`aherald`	`.alert.firing`, `.alert.resolved`,`.al`
`digital.vasic.herald.schedule.*`	`scherald`	`.job.started`, `.job.succeeded`, `.job`
`digital.vasic.herald.incident.*`	`iherald`	`.incident.opened`, `.incident.escal`
`digital.vasic.herald.release.*`	`rherald`	`.release.tagged`, `.release.publish`
`digital.vasic.herald.compliance.*`	`cherald`	`.audit.access`, `.policy.violation`,`.`
`digital.vasic.herald.reply.*`	all	`.reply.received`, `.command.parsed`,`.`
`digital.vasic.herald.delivery.*`	internal	`.delivery.accepted`, `.delivery.rou`

4.3 Idempotency keys

Per R-05 + research Topic 4: **at-least-once delivery with deterministic dedup**.

- Every ingress accepts an `Herald-Idempotency-Key` HTTP header (or `heraldidempotencykey` CE extension, or CLI flag `--idempotency-key`).
- If absent, Herald falls back to the CloudEvents `id`.
- Dedup is enforced via a Postgres `idempotency_keys` table:

```sql
CREATE TABLE idempotency_keys (
    tenant_id        UUID  NOT NULL,
    idempotency_key  TEXT  NOT NULL,
    request_hash     BYTEA NOT NULL,                -- SHA-256 of
        canonical request body
    response_id      UUID,                          -- pointer to
        first response
    locked_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at       TIMESTAMPTZ NOT NULL DEFAULT (now() +
        interval '24 hours'),
    PRIMARY KEY (tenant_id, idempotency_key)
) WITH (FILLFACTOR = 80);

ALTER TABLE idempotency_keys ENABLE ROW LEVEL SECURITY;
ALTER TABLE idempotency_keys FORCE ROW LEVEL SECURITY;
CREATE POLICY idem_isolation ON idempotency_keys
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Dedup `INSERT … ON CONFLICT DO NOTHING` is in the **same transaction** as the channel-fanout enqueue. Stripe-pattern (cite: Brandur's writeup).
- Redis is used as a **fast-path negative cache** (`SET NX EX <ttl>`) in front of Postgres, never as sole authority.
- TTL: 24h for synchronous API replies (matches Stripe); 7d for delivery-side dedup (matches reply-queue retention).
- Replay (same key, same body) returns the cached response; replay (same key, *different* body) returns HTTP 409 Conflict.

---

§5. Architecture overview

5.1 Components

A Herald flavor binary is composed of these in-process components:

1. **Ingress layer** — HTTP server (CloudEvents binding) + CLI command parser. Verifies signatures (§15.5), enforces idempotency (§4.3), and emits a Watermill message.
2. **Watermill router** — central message bus. Channel adapters, evaluators, and middleware (retry, dedup, throttle, trace, meter) are registered as `HandlerFunc`s on a single router.
3. **Channel adapters** — one per supported channel; implement the `Channel` interface (§10).
4. **Subscriber-reply consumers** — per-channel inbound loops (Telegram `getUpdates` long-poll or webhook, Slack Socket Mode / Events API webhook, Email IMAP, Discord gateway, etc.).
5. **Storage layer** — Postgres (durable state, RLS-isolated per tenant) + Redis (rate-limit token buckets, fast-path dedup, transient state).
6. **Queue backend** — Postgres + River as default; NATS JetStream as opt-in for multi-replica fan-out.
7. **Observability layer** — OpenTelemetry SDK (traces + metrics + logs), OTLP exporter, `/metrics` for Prometheus scrape, `/livez` / `/readyz` / `/startupz` probes.

5.2 Internal routing (Watermill)

Per research Topic 2: **Watermill** (`github.com/ThreeDotsLabs/watermill`, ~8.5k stars, v1.4+) is the routing kernel inside `commons_messaging`.

- Each channel adapter (Telegram, Slack, …) is registered on a single `message.Router` as a `HandlerFunc`.
- Cross-cutting concerns (correlation ID, retry, poison-queue, metrics, throttling, tracing) are implemented as **Watermill middlewares** rather than per-channel code — eliminating ~60% of the plumbing the team would otherwise write.
- Pubsub backend is **pluggable**: Postgres adapter (default), NATS adapter (opt-in), in-memory adapter (tests).

5.3 Queue backend (Postgres + River default, NATS opt-in)

Per research Topic 3:

- **Default: Postgres** + [`riverqueue/river`] — transactional enqueue (jobs only run if the originating tx commits), built-in retries with backoff, uniqueness constraints. Zero new infrastructure since Postgres is already required.
- `LISTEN/NOTIFY` wakes consumers sub-millisecond between polls.
- **Opt-in alternative: NATS JetStream** — for deployments needing fan-out across multiple Herald instances, edge sites, or sub-millisecond cross-host latency (~200k–400k msg/s with persistence, built-in KV/Object stores).
- **Kafka is intentionally NOT the default** — overkill for Herald's alert-volume profile (dozens to low-thousands of events/sec per tenant). Document as Watermill-pluggable for operators who already run Kafka.

5.4 Retries & dead-lettering

Per research Topic 5 and AWS arch blog "Exponential Backoff and Jitter":

- **Backoff curve**: **decorrelated jitter** — `sleep = min(cap, random_between(base, prev_sleep * 3))` with `base = 1s`, `cap = 5min`.
- **Max attempts**: 8 per message for transient errors. Empirically: ~30min worst-case retry window, matching what humans expect of an alert system.
- **No retries on 4xx** from upstream channels — `400` / `401` / `403` / `404` from Telegram/ Slack/etc. are permanent.
- Implementation: Watermill `Retry` middleware in front of channel handlers.
- **Dead letter sink: Postgres `dead_letters` table** (NOT a Redis list — operators need to query, triage, replay):

```
CREATE TABLE dead_letters (
    id                UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id         UUID NOT NULL,
    original_event_id UUID NOT NULL,
    channel           TEXT NOT NULL,
    attempt_count     INTEGER NOT NULL,
    last_error        TEXT,
    payload_jsonb     JSONB NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    triaged_at        TIMESTAMPTZ,
```

```sql
    triage_status     TEXT  -- new | acknowledged | replayed |
        discarded
);
ALTER TABLE dead_letters ENABLE ROW LEVEL SECURITY;
ALTER TABLE dead_letters FORCE ROW LEVEL SECURITY;
CREATE POLICY dl_isolation ON dead_letters
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Every entry into `dead_letters` also emits a `digital.vasic.herald.delivery.dead_lettered` CloudEvent — Herald observes its own failures.
- CLI: `<flavor>herald deadletter list | replay <id> | purge`.

5.4.1 Outbound idempotency (channel-side composition)

The §4.3 idempotency table guards **ingress**: same event id arriving twice produces one logical delivery. It does NOT, however, guard against the case where Herald has already called `Channel.Send` once and the call's *response* was lost (network blip, container OOM, channel-API timeout). Under at-least-once semantics, the retry layer (§5.4) WILL re-call `Channel.Send` — and a naive channel adapter would re-post the message, producing visible duplicates.

Resolution: every `Channel.Send` invocation MUST pass an **outbound-idempotency key** derived deterministically from `(tenant_id, EventID, ChannelID, ChannelAddressID)`. Adapters that support upstream idempotency (Slack `idempotency_key` extension, Stripe-style `Idempotency-Key` header on REST channels, Email `Message-ID` header) MUST forward this key. Adapters whose upstream does NOT support idempotency (raw SMTP, Telegram Bot API, Discord) MUST consult a Postgres `outbound_dedup` table inside the same River job transaction:

```sql
CREATE TABLE outbound_dedup (
    tenant_id           UUID NOT NULL,
    outbound_key        TEXT NOT NULL,                    --
        "<event_id>:<channel>:<channel_address_id>"
    sent_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
    channel_msg_id      TEXT,                             --
        adapter's Receipt.ChannelMsgID
    expires_at          TIMESTAMPTZ NOT NULL DEFAULT (now() +
        interval '24 hours'),
    PRIMARY KEY (tenant_id, outbound_key)
);
ALTER TABLE outbound_dedup ENABLE ROW LEVEL SECURITY;
ALTER TABLE outbound_dedup FORCE ROW LEVEL SECURITY;
CREATE POLICY od_isolation ON outbound_dedup
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

The adapter wraps each send in: `INSERT ... ON CONFLICT DO NOTHING; if row already existed, return cached Receipt without re-calling` `upstream`. The window is short (24h) because we only need to cover the retry-storm window — far shorter than ingress idempotency.

5.5 Webhook ingestion

Per R-16 + research Topic 8. Every webhook source registers a per-source signing secret (rotatable) in the `webhook_sources` table.

The ingress handler enforces, in order:

1. **HMAC-SHA256 signature verification** with `crypto/hmac.Equal` (constant-time) against the source-configured header:
   - GitHub: `X-Hub-Signature-256`
   - Stripe: `Stripe-Signature` (sign `timestamp.payload`)
   - Generic CloudEvents source: `Webhook-Signature`
2. **Timestamp window** ≤ 5 min vs. server clock (replay protection).
3. **Delivery-ID dedup** via the idempotency table (GitHub's `X-GitHub-Delivery`, Stripe's event `id`).
4. **Optional IP allowlist** as L4 defense-in-depth.

Verified payloads are normalized into a CloudEvent and handed to the Watermill router.

`webhook_sources` **schema:**

```
CREATE TABLE webhook_sources (
    id                UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id         UUID NOT NULL,
    name              TEXT NOT NULL,                    -- "github-push-prod", "stripe-webhook", ...
    signature_kind    TEXT NOT NULL,                    -- 'hmac-sha256' | 'stripe' | 'telegram-secret-token' | 'dkim' | 'none'
    signature_header TEXT NOT NULL,                     -- 'X-Hub-Signature-256', 'Stripe-Signature', ...
    secret_encrypted BYTEA NOT NULL,                    -- pgcrypto pgp_sym_encrypt
    secret_rotated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    ip_allowlist      INET[],                           -- optional L4 defense-in-depth
    replay_window_s  INTEGER NOT NULL DEFAULT 300,      -- 5 min
    enabled           BOOLEAN NOT NULL DEFAULT true,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, name)
);
CREATE INDEX webhook_sources_tenant_idx ON webhook_sources
        (tenant_id) WHERE enabled = true;
ALTER TABLE webhook_sources ENABLE ROW LEVEL SECURITY;
ALTER TABLE webhook_sources FORCE ROW LEVEL SECURITY;
CREATE POLICY ws_isolation ON webhook_sources
```

```
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

Secrets are rotatable: <flavor>herald webhook rotate <name> generates a new secret, returns it once, and starts accepting the new signature while continuing to accept the old one for replay_window_s × 4 (default 20 min) to drain in-flight deliveries.

Dev / behind-NAT support: smee.io and gh webhook forward are documented as supported proxy paths.

5.6 Orchestration of long-running operations (Temporal, opt-in)

Per research Topic 7:

- **Default routing** — Watermill handlers + River jobs cover the vast majority of fan-out cases.
- **Temporal sidecar (opt-in)** — required for operations that genuinely need durable state across hours/days, deterministic replay, and human-friendly timeline UIs:
    - **Incident escalation chains** (page A → wait 5min → escalate to B → wait 10min → page C). Used by iherald.
    - **Scheduled digest builders** with human-in-the-loop acknowledgment.
    - **Multi-channel orchestrated rollouts** (used by dherald / rherald).
- Temporal is explicitly **not** the default — its operational footprint (separate cluster, gRPC, server upgrades) violates Herald's "lightweight CLI" ethos.

5.7 Ingress API URLs (HTTP surface)

Herald's HTTP ingress (port 24091 default per §9.4) exposes a small, stable URL surface. All paths under /v1/ are public; everything under /admin/ is admin-port only (port 24090).

Method	Path	Purpose
POST	/v1/events	Canonical CloudEvents ingest (binary + structured modes). Body MUST be a CloudEvent per §4.1.
POST	/v1/send	Convenience REST ingest for ad-hoc senders that don't speak CloudEvents — JSON body {type, source, data, tags?, idempotency_key?, priority?} is wrapped into a CloudEvent server-side.
POST	/webhooks/{source_id}	Verified webhook receiver — source_id is a UUID matching webhook_sources.id; the handler enforces §5.5 signature + replay + dedup before forwarding.
GET	/v1/events/{id}	Idempotent replay of a previously-accepted event (returns 200 with cached response if id matches, 409 if id matches but body differs).
GET	/v1/deadletters	List dead-lettered messages (RLS-isolated to caller's tenant).
POST	/v1/deadletters/{id}/replay	Replay a dead-lettered message.
GET	/v1/subscribers / POST /v1/subscribers	Subscriber CRUD (operator role required).

Method	Path	Purpose
`POST`	`/v1/subscribers/{id}/forget`	GDPR right-to-erasure (§16.1).
`GET`	`/v1/subscribers/{id}/export`	GDPR right-to-portability — emits JSON bundle.
`GET`	`/v1/channels`	List configured channel addresses for the caller's tenant.
`GET`	`/livez`	Liveness probe (admin port — §17.5).
`GET`	`/readyz`	Readiness probe (admin port).
`GET`	`/startupz`	Startup probe (admin port).
`GET`	`/metrics`	Prometheus scrape (admin port).
`GET`	`/admin/version`	Build info (commit SHA, build time, Go version).
`GET`	`/admin/pprof/*`	Go runtime profiling (admin port; loopback-only by default; opt-in `[admin].pprof_external=true`).

Auth model:

- `/v1/events` and `/v1/send` accept either (a) HTTP Basic auth with a per-tenant `ingest_token` from the `tenants` table, or (b) signed bearer token in `Authorization: Bearer <jwt>` (Herald validates against JWKS configured in `[auth.oidc]`). Self-hosted operators MAY enable mTLS via a reverse proxy.
- `/webhooks/{source_id}` does its own signature verification (§5.5) — no token required at the HTTP layer.
- `/v1/subscribers*` requires a JWT with operator-role claim.
- Admin endpoints listen only on the admin port (default loopback or trusted-network only).

Versioning: the `/v1/` prefix is the stable contract; breaking changes ship as `/v2/` with at least 6 months of `/v1/` co-existence. The OpenAPI schema (`docs/api/openapi.v1.yaml`) is the source of truth — `<flavor>herald openapi` prints the embedded spec.

Rate limits: per-tenant token bucket (§16, default 1000 req/min per tenant on `/v1/events`); per-IP secondary bucket for unauthenticated webhook receivers (default 60 req/min/IP).

---

§6. Channel addressing & routing

6.1 URL-scheme channel addresses (Apprise-style)

Per research Topic 1 (notification-platforms): adopt Apprise's URL-scheme channel model. Each destination is a single string that fully captures credentials + endpoint + targeting:

```
tgram://<bot_token>/<chat_id>?tags=oncall,prod
slack://<token_a>/<token_b>/<token_c>/<channel>?tags=oncall
max://<bot_token>/<chat_id>?tags=ru,prod
mailto://<user>:<pass>@<smtp_host>:<port>?tags=audit&from=herald@example.c
discord://<webhook_id>/<webhook_token>?tags=community
teams://<incoming_webhook_path>?tags=corp
lark://<app_id>:<app_secret>@<chat_id>?tags=apac
ntfy://<server>/<topic>?priority=high&tags=mobile
gotify://<token>@<server>?priority=5
```

```
webhook://<endpoint_url>?secret=<hmac_secret>&format=cloudevent
diary://?tags=*&format=markdown
```

This URL form maps 1:1 to a row in Postgres `channel_addresses` and lets credentials be `.env`-interpolated at config-load time so the URLs themselves stay git-safe in committed `config.toml`.

6.2 Tag-based fan-out

Per Apprise's tag model: every channel address is annotated with one or more **tags**. Senders address tags, not specific channels:

- **OR-union by default**: `--tags=oncall,prod` matches any channel tagged `oncall` OR `prod`.
- **AND-intersection** when explicitly comma-joined inside a single argument: `--tags=oncall+prod` matches channels tagged `oncall` AND `prod`.
- Tag `*` is the wildcard (matches every channel).
- Tag `!oncall` means "exclude oncall" (intersect with negation).

Tag→channel mapping lives in the `channel_addresses` table; senders never reference channel addresses directly. This decouples sender code from topology — the same event can fan out to a different mix of channels per environment without code change.

`channel_addresses` schema:

```sql
CREATE TABLE channel_addresses (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id       UUID NOT NULL,
    channel         TEXT NOT NULL,                  -- "tgram", "slack", "mailto", ...
    address_url     TEXT NOT NULL,                  -- "tgram://${BOT_TOKEN}/${CHAT_ID}?…" (env-interpolated at load time)
    tags            TEXT[] NOT NULL DEFAULT '{}',   -- ['oncall','prod']
    priority_floor  INTEGER NOT NULL DEFAULT 1,     -- ntfy 1..5; messages below floor are dropped for this address
    enabled         BOOLEAN NOT NULL DEFAULT true,
    last_health_at  TIMESTAMPTZ,
    last_health_ok  BOOLEAN,
    last_health_err TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, address_url)
);
CREATE INDEX ch_addr_tags_idx ON channel_addresses USING gin
        (tags) WHERE enabled = true;
CREATE INDEX ch_addr_tenant_idx ON channel_addresses (tenant_id,
        channel) WHERE enabled = true;
ALTER TABLE channel_addresses ENABLE ROW LEVEL SECURITY;
ALTER TABLE channel_addresses FORCE ROW LEVEL SECURITY;
CREATE POLICY ca_isolation ON channel_addresses
```

```
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

The `address_url` MAY contain `${ENV_VAR}` placeholders that are interpolated at config load time (so secrets stay out of the row body — only the placeholder is persisted; secrets remain in `.env` / shell env).

6.3 HeraldBranding (per-flavor visual identity)

Per Apprise's `AppriseAsset`: a `HeraldBranding` struct injected per-flavor:

```
type Branding struct {
    AppName        string // "Project Herald", "System
        Herald", ...
    BinaryName     string // "pherald", "sherald", ...
    IconURL        string // for rich embeds
    AccentColorHex string // "#2C7BE5"
    DefaultFooter  string // "Sent by pherald 1.0 · github.com/
        vasic-digital/Herald"

    // Wave 2 (r8) — per-flavor identity propagated to the shared
    // commons/cli/ scaffold. Drives Cobra Use/Short/Long output,
    // default HTTP serve-port binding, and version --json shape.
    Flavor      string // "p", "s", "b", ... (single-letter key;
        "sc" for scherald)
    Prefix      string // 3-letter §8.2 anchor (e.g. "PHR",
        "SHR", "CHR")
    DisplayName string // operator-facing display name (often ==
        AppName)
    DefaultPort int    // default HTTP listen port (24791..24799
        block; 0 for CLI-only flavors)
    Mission
        string // one-line mission statement for --help / about
}
```

Channel adapters consult `Branding` when rendering rich messages (Slack Block Kit headers, Discord embed `author`, Adaptive Card `Container` accent color, Email From-name). Per-flavor `Branding` is constructed via `commons.DefaultBranding(<flavor>, version)` at startup of each `<prefix>herald` binary and threaded through every `OutboundMessage`; the Wave 2 fields (`Flavor`, `Prefix`, `DisplayName`, `DefaultPort`, `Mission`) drive both the shared `commons/cli/` scaffold (Cobra Use/Short/Long, default HTTP serve-port binding, `version --json` shape) and adapter-side branding when rendering rich messages.

---

§7. Subscriber model

7.1 Identity & reconciliation (per-channel-id + operator alias)

Per R-07 + research Topic 3 (notification-platforms): the **matterbridge model** is the default — per-channel-id is the source of truth. A subscriber on Telegram is identified by `(channel:"tgram", chat_id:"42"); the same human on Slack is` (channel:"slack", user_id:"U0123")`; these are **separate entries** unless explicitly linked.

Schema:

```
CREATE TABLE subscribers (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id       UUID NOT NULL,
    handle          TEXT,                               -- operator-
        assigned, e.g. "alice"
    display_name TEXT,
    locale          TEXT DEFAULT 'en-US',
    timezone        TEXT DEFAULT 'UTC',
    roles           TEXT[] NOT NULL DEFAULT '{}',       -- e.g.
        ['operator','reader','ic']
    metadata        JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, handle)                          -- handle is
        unique within a tenant
);
CREATE INDEX subscribers_tenant_idx ON subscribers (tenant_id);
ALTER TABLE subscribers ENABLE ROW LEVEL SECURITY;
ALTER TABLE subscribers FORCE ROW LEVEL SECURITY;
CREATE POLICY sub_isolation ON subscribers
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);

CREATE TABLE subscriber_aliases (
    subscriber_id   UUID NOT NULL REFERENCES subscribers(id) ON
        DELETE CASCADE,
    channel         TEXT NOT NULL,
    channel_user_id TEXT NOT NULL,
    verified_at     TIMESTAMPTZ,
        -- self-claim verification time
    last_seen_at    TIMESTAMPTZ,
    UNIQUE (channel, channel_user_id)
);
CREATE INDEX subscriber_aliases_sub_idx ON subscriber_aliases
        (subscriber_id);
```

Linking flows:

- **Operator-mapped** (default): operator runs `pherald subscriber link --handle alice --tgram 42 --slack U0123`.
- **Self-claim** (opt-in per tenant): subscriber DMs a one-time token to the bot on each channel; the bot calls `pherald subscriber verify <token>` to record the alias.
- **Never inferred** — Herald does not guess identity across channels.

7.2 Preferences (Knock-style PreferenceSet)

Per research Topic 2 (notification-platforms): a Knock-inspired `PreferenceSet` JSON per subscriber:

```
{
  "categories": {
    "incidents":    { "channels": ["slack","tgram","email"], "muted": fals
    "deploys":      { "channels": ["slack"], "muted": false },
    "daily_digest": { "channels": ["email"], "muted": false }
  },
  "workflows": {
    "digital.vasic.herald.alert.firing":   { "muted": false, "channels": [
    "digital.vasic.herald.alert.resolved": { "muted": true }
  },
  "quiet_hours": { "tz": "Europe/Belgrade", "start": "22:00", "end": "07:0
  "channel_data": {
    "tgram": { "chat_id": "42", "thread_id": null },
    "slack": { "user_id": "U0123", "im_channel": "D098" }
  }
}
```

- The router consults `PreferenceSet` before dispatching: if `muted` for the workflow/category, drop (and log to diary as "filtered"); otherwise restrict to listed channels and respect quiet hours.
- `channel_data` carries provider-specific routing keys that don't fit the generic alias model (Slack im_channel for DMs, Telegram thread_id for forum-topic posts).

7.3 Quiet hours & throttling

- **Quiet hours** — TZ-aware window per subscriber; exempt categories override.
- **Throttling** — per-`(tenant, subscriber, category)` token bucket in Redis (`herald:t:<id>:rl:<sub>:<cat>` with `INCR`+`EXPIRE`); default cap configurable per tenant.
- **Batching** — per category, optional `batch_window` (e.g. 5min); events within the window collapse into a single digest message. Implementation: River job scheduled `batch_window` after the first event arrives.

7.4 Locale (i18n)

Per research Topic 6 (notification-platforms): `nicksnyder/go-i18n` **v2** with CLDR plural rules.

- One `Bundle` loaded at process start from `commons_messaging/locales/*.toml`.

- One `Localizer` per outgoing message, constructed from the subscriber's stored `locale` field (BCP-47 tag).
- Per-channel templates can override (emoji-heavy Telegram template vs sober email template, both for the same locale).
- Initial locales for V2: `en-US`, `sr-Latn-RS`, `ru-RU`. Additional locales as community contributes them.

7.5 AI-agent subscribers (distinct from human subscribers)

Herald is explicitly a target for AI CLI agent invocation (per §3.2). Agents are a fundamentally different kind of subscriber from humans: they invoke far more frequently, they don't sleep, they don't have quiet hours, and their failure modes (runaway loop, infinite retry, prompt-injection-driven misuse) need stronger throttling. V2 models them as a first-class subscriber kind.

Schema extension to `subscribers`:

- `kind` column (enum: `human` | `agent` | `service`). Default `human`. Indexes added on `(tenant_id, kind)`.
- `agent` rows MUST carry a `metadata.agent_token_id` pointing at a row in `agent_tokens` (separate table, encrypted), used as the auth credential when the agent calls `/v1/send`.
- `agent` rows MUST carry `metadata.parent_subscriber_id` (the human operator who created the agent) so audit + GDPR right-to-erasure cascades correctly.

```
CREATE TABLE agent_tokens (
    id                UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id         UUID NOT NULL,
    subscriber_id     UUID NOT NULL REFERENCES subscribers(id) ON
        DELETE CASCADE,
    token_hash        BYTEA NOT NULL,
        -- argon2id of the bearer token
    name              TEXT NOT NULL,                    -- "claude-
        code-laptop", "build-bot-prod"
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_used_at      TIMESTAMPTZ,
    expires_at        TIMESTAMPTZ,
        -- nullable = no expiry
    revoked_at        TIMESTAMPTZ,
    rate_limit_per_min INTEGER NOT NULL DEFAULT 60,    -- default
        60 req/min per token
    UNIQUE (tenant_id, name)
);
ALTER TABLE agent_tokens ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_tokens FORCE ROW LEVEL SECURITY;
CREATE POLICY at_isolation ON agent_tokens
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

Default throttling (operator-overridable per token):

Limit	Default	Burst	Notes
`/v1/send` requests	60 req/min/token	10	per the rate_limit_per_min column
`/v1/events` requests	30 req/min/token	5	tighter because CloudEvents body is unbounded
Outbound deliveries per token per min	120	20	a single agent CAN'T flood a tenant's channels
Quarantine threshold	3 consecutive rejected sends → token disabled for 30 min	—	auto-cool-down to prevent prompt-injection loops

Audit + observability: every agent send emits a span with `herald.subscriber.kind="agent"` and `herald.agent.token_id`; dashboards split agent vs human traffic by default. The `cherald` flavor (per §18.10) flags suspicious agent patterns (sudden volume spike, off-hours bursts) as compliance events.

Quiet hours: ignored for `kind=agent` by default — agents are expected to send during off-hours. Operator MAY opt-in to per-token quiet hours via `agent_tokens.metadata.quiet_hours`.

Revocation: `<flavor>herald subscriber agent-token revoke <name>` zeroes the row's `revoked_at` and invalidates the token immediately (Redis cache TTL 30 s for token validity).

7.6 Participant identity (logical Subscriber/User + per-channel @username) — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/PARTICIPANT_ATTRIBUTION.md` §1. Constitution authority: inherited §11.4.104 (restated in HelixConstitution `Constitution.md/CLAUDE.md/AGENTS.md/QWEN.md` per §11.4.35). **As-built:** LIVE — `commons/participant.go` (`Participant`, `IdentityResolver`, `MemoryResolver`, `OperatorHandleFromEnv`) + `commons_storage/migrations/000015_subscriber_alias_username.{up,down}.sql`.

Every messenger relates inbound + outbound messages to a **Participant** — one logical Subscriber/User (§7.1) who may carry a **different** `@username` **on every messenger**. The same human is `@milos85vasic` on Telegram and potentially `@milos.vasic` on Slack; these per-channel handles all resolve to one canonical, messenger-neutral handle.

Backed by the existing §7.1 tables (no new table; one new column):

- `subscribers` — the logical party: `handle` (canonical, messenger-neutral), `display_name`, `kind ∈ {human, agent, service}` (§7.5).
- `subscriber_aliases` — the per-channel handle, with the **NEW** `username TEXT` **column** (migration `000015`): the per-channel `@handle` used for tagging, **distinct from** `channel_user_id` (which is the chat/user id). The `UNIQUE (channel, channel_user_id)` constraint from §7.1 is unchanged; a partial index covers the new `username`.

```sql
-- migration 000015 (V4) — additive to the §7.1 subscriber_aliases schema:
ALTER TABLE subscriber_aliases ADD COLUMN username TEXT;   -- per-channel @handle for tagging
CREATE INDEX subscriber_aliases_username_idx ON subscriber_aliases (channel, username)
    WHERE username IS NOT NULL;
```

Canonical handle — the messenger-neutral string stored in a workable item's `created_by` / `assigned_to` (§8.4). Closed set:

- `Claude` — the **system-agent sentinel** (`commons.SystemAgentHandle`; `kind=agent`). Reserved; **NEVER @-tagged** (it is the system, not a human participant).
- a human's **canonical handle** — defaults to their Telegram `@username` (Telegram is the primary messenger) but is messenger-neutral; per-channel `@usernames` are resolved through `subscriber_aliases.username`.

Operator — the one human who drives the system through the Claude Code CLI. Designated by **env var, NOT a DB flag**: `HERALD_<CHANNEL>_OPERATOR_USERNAME` (e.g. `HERALD_TGRAM_OPERATOR_USERNAME=@milos85vasic`, generalizing to `HERALD_SLACK_OPERATOR_USERNAME`, …). The operator's canonical handle is their Telegram operator username; the operator is a normal Participant whose handle equals that env value.

Go contract surface (`commons`, load-bearing — matches the §1 contract exactly):

```go
const SystemAgentHandle = "Claude" // reserved created_by/assigned_to sentinel; never tagged

type Participant struct {
    Handle      string            // canonical handle (e.g. "@milos85vasic" or "Claude")
    DisplayName string
    Kind        string            // "human" | "agent" | "service" (§7.5)
    Usernames   map[string]string // channel -> "@username" on that channel
}

// IdentityResolver bridges the per-channel runtime world to canonical handles.
type IdentityResolver interface {
    // Inbound: map a received message's sender to a canonical handle.
    ResolveSender(channel, channelUserID, username string) (handle string)
    // Outbound: the @username for a canonical handle on a target channel
    // (ok=false if the participant has no alias there — cannot tag someone
    // who is not on that messenger).
```

```go
    UsernameFor(handle, channel string) (username string, ok
        bool)
    // OperatorHandle returns the canonical operator handle
    // (from HERALD_<CHANNEL>_OPERATOR_USERNAME).
    OperatorHandle() string
}
```

`MemoryResolver` is the concrete in-memory resolver built from a participant roster + the operator handle; unknown senders resolve to their raw normalized `@username` so first-contact users stay attributable. `OperatorHandleFromEnv(channel)` reads HERALD_<CHANNEL>_OPERATOR_USERNAME.

The §7.1 linking rules still apply: aliases are operator-mapped or self-claim-verified, **never inferred** — Herald does not guess identity across channels.

---

§8. Workable-item naming prefix

8.1 Static prefix `HRD-`

For all opened workable items for the Herald project (Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, etc.) use the prefix `HRD`. Examples: `HRD-001`, `HRD-002`. Strict zero-padded sequence within each docset.

8.2 Derived 3-letter prefix algorithm

Per R-17 — when a consuming project does NOT define its own workable-item prefix, Herald derives a deterministic 3-letter prefix from the project name. Implementation lives in `commons/prefix`.

Algorithm (~80 LOC):

1. **Normalize**: Unicode NFKD → strip diacritics → retain `[A-Za-z0-9]` → split on CamelCase boundaries and `[-_ /]` into tokens.
2. **Rule A (≥3 tokens)**: first letter of each of the first three tokens. `HeraldRouterCore` → `HRC`.
3. **Rule B (2 tokens)**: first letter of token 1; first letter of token 2; first internal consonant of token 2. `HeraldRouter` → `HRT`; `HeraldRunner` → `HRN`.
4. **Rule C (1 token)**: first letter, first internal consonant, last consonant. `Herald` → `HRD`.
5. Uppercase.
6. **Collision resolution**: maintain committed `.herald/prefix.lock` (TOML `name → prefix`). On collision with a *different* project, compute `fnv1a32(name) mod 26` and replace the third letter with `'A' + (h mod 26)`; iterate up to 26 times, then fall back to numeric suffix `HR0…HR9`.
7. **Persistence**: the lock file is committed so the mapping is stable across machines and regenerations.

No mature Go library exists for 3-letter abbreviation generation (the only Go prior art `Defacto2/releaser/initialism` is a curated lookup table, not a generator); Herald ships its own.

8.3 Workable-item lifecycle (`HRD-NNN` flow)

A workable item moves through this lifecycle across `pherald`'s subscriber-command handlers and the `docs/` tracker files (per Universal §11.4.12 + §11.4.53):

```
+------------------+
| Subscriber DM    |   e.g. "Bug: telemetry pipe drops every hour"
|  to pherald      |
+------------------+
        |  (security validation §15; allowlist verified)
        v
+------------------+
|  Command         |   parser identifies "Bug:" → opens new item
|  parsed (§15.3)  |
+------------------+
        |
        v
+------------------------------------------------------+
|  Item allocation                                     |
|   • prefix = "HRD-" (or derived 3-letter via §8.2)   |
|   • next sequence number from `workable_items` table |
|   • full id = "HRD-042"                              |
+------------------------------------------------------+
        |
        v
+------------------------------------------------------+
|  Append row to `docs/Issues.md`                      |
|   | HRD-042 | Bug   | open    | <subscriber>         |
|   |          | <date>| <one-line summary>            |
|  (per §11.4.12 Issues format)                        |
+------------------------------------------------------+
        |
        v
+------------------------------------------------------+
|  Emit `digital.vasic.herald.project.task.opened`     |
|   event with the new HRD-NNN id                      |
+------------------------------------------------------+
        |
        v
+------------------------------------------------------+
|  Diary append (§19)                                  |
+------------------------------------------------------+
        |
        v
+------------------------------------------------------+
|  ACK back to the subscriber on the original channel  |
|   (reply in-thread) — "Tracked as HRD-042."          |
+------------------------------------------------------+
        |
        :
        :  ... time passes, subscriber sends commands ...
        :
        v
```

```
Resolution: "Done: HRD-042" or "Resolve: HRD-042"
 (operator role required)
→ mark `Issues.md` row as resolved
→ atomic migration to `Fixed.md` (per §11.4.19)
→ emit `…project.task.closed`
→ ACK in original thread
```

`workable_items` **schema** (lightweight pointer table — the canonical record lives in the Markdown files for human edit-ability per Universal §6):

```sql
CREATE TABLE workable_items (
    tenant_id    UUID NOT NULL,
    item_id      TEXT NOT NULL,                   -- "HRD-042"
    prefix       TEXT NOT NULL,                   -- "HRD"
    sequence     INTEGER NOT NULL,                -- 42
    item_type    TEXT NOT NULL,                   -- 'bug' |
        'issue' | 'query' | 'request' | 'question'
    status       TEXT NOT NULL,                   -- 'open' |
        'in_progress' | 'blocked' | 'resolved' | 'wont_fix'
    opened_by    UUID,                            -- subscriber
        id
    opened_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    resolved_at  TIMESTAMPTZ,
    source_thread JSONB,                          --
        ConversationRef serialised
    PRIMARY KEY (tenant_id, item_id),
    UNIQUE (tenant_id, prefix, sequence)
);
CREATE INDEX wi_status_idx ON workable_items (tenant_id, status)
        WHERE status <> 'resolved';
ALTER TABLE workable_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE workable_items FORCE ROW LEVEL SECURITY;
CREATE POLICY wi_isolation ON workable_items
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

Sequence allocation: a per-tenant Postgres advisory lock around `MAX(sequence) + 1` inside the transaction; safe under concurrent `Bug:` commands. For high-volume tenants, a per-prefix sequence is preallocated in batches (default batch size 100) cached in Redis to avoid contention.

Reopening: `Reopen: HRD-042` (operator role) reverses the migration: row moves back to `Issues.md`, `status='in_progress'`, `resolved_at` cleared, history preserved in `docs/Reopens/HRD-042.md` per Universal §11.4.55.

8.4 Workable-item attribution (`created_by` + `assigned_to`) — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/PARTICIPANT_ATTRIBUTION.md` §2 + §4. Constitution authority: inherited

§11.4.104. **As-built:** LIVE — `commons_workable` `Item.CreatedBy`/`Item.AssignedTo` (+ SQLite columns + change-feed); inbound wiring `pherald/internal/inbound/attribution.go`; the workable-items SSoT tool (`constitution/scripts/workable-items/`) adds the same two columns.

Every workable item gains two attribution fields, each storing a **canonical handle string** (§7.6) — self-contained in the SSoT and the Markdown trackers:

- `created_by` — who opened/originated the item.
- `assigned_to` — who is responsible for it.

Attribution rules — `created_by`**:**

Origin of the item	`created_by`
Opened via the **Claude Code CLI prompt** (operator-driven)	`OperatorHandle()` — the operator's canonical handle
Opened by **System/ Claude** detecting an issue/task/improvement/ missing-feature	`Claude` (`commons.SystemAgentHandle`)
Received **through Herald** (a subscriber message opens/updates an item)	the sender's canonical handle, resolved via `IdentityResolver.ResolveSender(channel, channelUserID, @username)`

Attribution rules — `assigned_to`**:**

- **Default** = `OperatorHandle()` (the operator's canonical handle).
- **Overridden** explicitly when a prompt or message assigns the item (e.g. `assign: @bob`, or the Tier-1 `assign ATM-5 to @bob` intent — §32.6.B).

The inbound wiring (§32.6.A) never overwrites an LLM-supplied value, and preserves Wave-6 behaviour when no `IdentityResolver` is configured (nil resolver = attribution disabled, not a crash).

Storage & format changes (additive; legacy empty values accepted):

- **Workable-items SQLite SSoT** (the constitution tool — the canonical schema): `items` gains `created_by TEXT NOT NULL DEFAULT ''` + `assigned_to TEXT NOT NULL DEFAULT ''`; the parser reads `**Created-By:** <handle>` / `**Assigned-To:** <handle>` from each item block, the renderer writes them, byte-identical round-trip preserved.
- `commons_workable` (Herald's mirror store): `Item` gains `CreatedBy string` + `AssignedTo string`; the SQLite `items` table + Go migration add the two columns; the change-feed `Diff` emits `item.field.changed` for `created_by`/`assigned_to` changes.
- **Markdown trackers**: the Herald pipe-table `Issues.md`/`Fixed.md` add **Created-By** + **Assigned-To** columns; heading-format trackers add `**Created-By:**` / `**Assigned-To:**` fields. HTML/PDF/DOCX siblings + summaries regenerate for every edited doc.

These two columns are the substrate the §34.7 notification @-tagging matrix reads.

---

§9. Technology stack

9.1 Go (single-binary, multi-flavor)

Herald and all flavors MUST BE written in **Go**.

Toolchain pin: Go ≥ **1.22** is mandatory (stdlib `log/slog`, OTel `slog` bridge, `slices` + `maps` generics). The repo's `go.mod` files declare `go 1.22` and set `toolchain go1.22.x` to the current patch release. Bumps to ≥ 1.23 are allowed when CI proves no regression; the `commons` module is the authoritative version (every flavor MUST follow).

License: Herald is published under the terms in [`LICENSE`] (parent project's chosen license). All `go.mod` files MUST declare the same license via a top-level comment, and every `LICENSE` file in vendored submodules MUST remain present. License compliance is checked by the planned `cherald` flavor (per §18.10) on every CI run when configured.

Layout (per R-18, research Topic 5):

```
Herald/
├── go.work                    # local dev only, .gitignored
├── commons/       go.mod      # module github.com/vasic-digital/herald/common
├── pherald/       go.mod      # module github.com/vasic-digital/herald/pheral
├── sherald/       go.mod      # module github.com/vasic-digital/herald/sheral
├── bherald/       go.mod      # ... etc
├── .goreleaser.yaml           # one config, builds all flavors
└── docs/, tests/, …
```

Each flavor's `go.mod` declares `require github.com/vasic-digital/herald/commons vX.Y.Z`. **Lockstep release**: one git tag triggers GoReleaser to build all flavors and tag `commons/vX.Y.Z`. Use [release-please] in `manifest` mode to bump from Conventional Commits. `go.work` is git-ignored so CI catches forgotten bumps.

9.2 Postgres + Row-Level Security

Postgres is the main database, deployed via the `containers` submodule. Schema highlights:

- Every business table carries `tenant_id UUID NOT NULL`; composite indexes `(tenant_id, …)`.
- `FORCE ROW LEVEL SECURITY` enabled on every multi-tenant table.
- Runtime role: `herald_app` (no `BYPASSRLS`). Migration role: `herald_migrator` (`BYPASSRLS`).
- SET LOCAL app.tenant_id = '<uuid>' at the start of every transaction.
- UUIDv7 (`uuidv7()` function) for time-ordered primary keys.

9.3 Redis (per-tenant ACL)

Redis is the in-memory store, deployed via `containers` submodule.

- **Per-tenant ACL user**: redis-cli ACL SETUSER tenant_<id> on >pwd ~t:<id>:* +@read +@write ….
- **Key prefixing**: t:<tenant_id>:… on every key.

- Usage: rate-limit token buckets, fast-path idempotency-cache, transient deduplication, hot subscriber lookups.

9.4 Container ports (`24XXX`)

Per R-01: all Container host ports start with `24XXX` prefix (1024–49151 IANA User Ports, below the Linux ephemeral floor at 32768, above all common service defaults — Postgres 5432, Redis 6379, web 3000/5000/8000/8080/9000).

Reserved sub-blocks:

Range	Purpose
`24000-24099`	flavor app data ports (one per flavor instance)
`24090-24099`	flavor admin ports (`/livez`, `/readyz`, `/metrics`, pprof)
`24100-24199`	Postgres instances (default 24100)
`24200-24299`	Redis instances (default 24200)
`24300-24399`	NATS JetStream (when enabled)
`24400-24499`	OpenTelemetry Collector (default OTLP gRPC 24417, HTTP 24418)
`24500-24599`	Prometheus / Grafana (when self-hosted alongside)
`24600-24699`	Temporal (when enabled)
`24700-24999`	reserved for future / per-tenant

9.5 `containers` submodule

The full Docker/Podman Compose stack is provided by [`vasic-digital/containers`]() as a Git submodule (per R-12, the owned-submodule set in `HERALD_CONSTITUTION.md` will be updated in the PR that introduces the submodule). All container names MUST start with prefix `herald`.

9.6 Database migration tooling

Herald uses [`golang-migrate/migrate`]() (the de-facto standard Go migration tool) embedded as a library inside `commons_storage`. Rationale: `golang-migrate` is binary-distributable AND embeddable, supports `up`/`down`, file checksums (drift detection), `schema_migrations` table version locking, and ships idempotent migrations.

File layout (`commons_storage/migrations/`):

```
commons_storage/migrations/
├── 000001_init_core.up.sql          # tenants, roles, encryption keys
├── 000001_init_core.down.sql
├── 000002_idempotency_keys.up.sql
├── 000002_idempotency_keys.down.sql
├── 000003_subscribers.up.sql        # subscribers + subscriber_aliases (§7.
├── 000003_subscribers.down.sql
├── 000004_channel_addresses.up.sql         # §6
├── 000004_channel_addresses.down.sql
├── 000005_webhook_sources.up.sql           # §5.5
├── 000005_webhook_sources.down.sql
├── 000006_thread_refs.up.sql               # §12
├── 000006_thread_refs.down.sql
```

```
├── 000007_quarantined_messages.up.sql    # §15.2
├── 000007_quarantined_messages.down.sql
├── 000008_dead_letters.up.sql            # §5.4
├── 000008_dead_letters.down.sql
├── 000009_email_suppressions.up.sql      # §11.9
├── 000009_email_suppressions.down.sql
├── 000010_river_jobs.up.sql              # River queue schema (managed by
├── 000010_river_jobs.down.sql
└── 000011_rls_policies.up.sql            # FORCE RLS on every multi-tenan
```

Numbering: zero-padded 6-digit sequence (`000001..000999` reserved for V2 baseline; `001000..` reserved for V3+). Down migrations MUST exist for every up migration so `<flavor>herald migrate down -n 1` is always safe.

Runtime contract:

- `<flavor>herald migrate up [--steps N]` — apply pending migrations forward.
- `<flavor>herald migrate down --steps N` — rollback N migrations (gated behind interactive confirmation unless `--yes`).
- `<flavor>herald migrate status` — show current version + pending count.
- `<flavor>herald migrate force <version>` — recovery only; sets the version without running migrations (operator MUST verify manually).
- `<flavor>herald migrate validate` — checksum every applied migration against the source files; fails if drift detected.

Roles (per §16):

- `herald_migrator` — `BYPASSRLS`, owns schema, runs DDL. Only used by `migrate` subcommand.
- `herald_app` — runtime role; cannot run DDL.

Forward compatibility (per §26.3): migrations MUST be backward-compatible for **two minor versions** so rolling restarts during a deploy don't break the running fleet. Practical patterns: add nullable columns first, backfill, then add NOT NULL; never drop a column the previous binary version still reads; never rename a column — add the new one, dual-write, then drop the old in a later release.

---

§10. Commons (architecture layers)

Per the layering principle in V1: `commons -> commons level 1 -> … -> commons level N -> Flavor`. V2 names the layers concretely:

Layer	Module	Owns
L0	`commons`	CloudEvents envelope, `Channel` interface, `Branding`, error types, time/uuid helpers
L1	`commons_messaging`	Watermill router, retry/throttle/dedup middlewares, channel adapters (Telegram, Slack, …), templating, i18n
L1	`commons_storage`	Postgres connection + RLS middleware, River queue setup, Redis client + tenant ACL, migrations
L1	`commons_security`	

Layer	Module	Owns
		HMAC verifiers (Slack/GitHub/Telegram/generic), DKIM/SPF/DMARC helpers, allowlist evaluator, command parser
L1	`commons_observability`	OTel setup, span helpers, metrics registration, log handler wiring (`slog` + `otelslog`)
L1	`commons_prefix`	3-letter prefix algorithm (§8.2)
L1	`commons_diary`	append + export + sync (§19)
L2	`commons_workflows`	Temporal workflow scaffolding (escalation chains, digest builders) — opt-in
Flavor	`pherald`, `sherald`, …	Per-flavor commands, event-type handlers, flavor-specific subscriber commands

Rule of thumb (carry from V1): put new shared code in the **lowest layer that still makes sense**; flavors inherit upward.

---

§11. Channels — per-channel capabilities matrix

11.0 Channel adapter contract (Go interface + value types)

Each channel adapter implements the `Channel` interface defined in `commons` (L0). The full contract:

```go
package commons // L0

import "context"

// Channel is the interface every channel adapter implements.
type Channel interface {
	Name()
		string                                                  //
		"tgram", "slack", ...
	Capabilities()
		Capabilities                                      //
		declarative feature flags
	Send(ctx context.Context, msg OutboundMessage) (Receipt,
		error)
	Subscribe(ctx context.Context, h InboundHandler)
		error            // long-running; called by `serve`
	HealthCheck(ctx context.Context) error
}

// Capabilities advertises what an adapter supports at runtime.
// Routers consult Capabilities before dispatching; mismatches
		are logged
// and the message is dropped or downgraded (per the adapter's
		choice).
type Capabilities struct {
```

```go
    Text             bool
    Markdown         bool   // adapter's native markdown flavor
        (Slack mrkdwn, Telegram MarkdownV2, etc.)
    HTML             bool
    Attachments      bool
    AttachmentMaxMiB int
    Threads          bool   // first-class reply threading
    InteractiveURL   bool   // URL buttons / link actions
    InteractiveCall  bool   // callback handlers (advanced tier)
    DeliveryCeiling  DeliveryEvidence // see §17 / R-05
}

// DeliveryEvidence enumerates the strongest reachable signal per
        channel.
type DeliveryEvidence int

const (
    DeliveryUnknown   DeliveryEvidence = iota
    DeliveryAccepted                   // hop-by-hop ack (SMTP
        250)
    DeliveryRouted                     // platform stored &
        broadcast (Telegram/Slack API ok)
    DeliveryDelivered                  // recipient transport
        confirmed (Email DSN, WA delivered receipt)
    DeliveryRead                       // recipient read (Email
        MDN, Telegram Business read marker)
)

// OutboundMessage is the value passed to Channel.Send.
type OutboundMessage struct {
    EventID         string             // CloudEvents id (§4)
    IdempotencyKey
        string            // explicit; falls back to EventID
    TenantID        string
    To              []Recipient        // resolved from
        subscriber preferences + tag fan-out
    Subject         string             // optional (Email, RSS-
        like channels)
    Body            Body               // rendered per-channel
        template output
    Attachments     []Attachment
    Thread          *ConversationRef   // optional; per §12
    Priority        Priority           // ntfy-compatible 1..5
    Actions         []Action           // optional interactive
        buttons
    Branding        Branding           // per-flavor (§6.3)
    Trace           TraceContext       // OTel propagation
}
```

```go
// Body carries one or more rendered representations; adapter picks the best
// match for its Capabilities (e.g., HTML for email, Markdown for Telegram,
// Block-Kit JSON for Slack).
type Body struct {
    Plain    string
    Markdown string
    HTML     string
    Native   map[string]any          // adapter-specific (Slack blocks, Adaptive Card JSON, ...)
}

// Attachment is a single attached file.
type Attachment struct {
    Filename  string
    MIMEType  string
    SizeBytes int64
    Reader    func() (io.ReadCloser, error) // lazy open so the adapter streams without buffering
    CID       string                        // optional inline-image Content-ID (Email)
}

// Recipient is a resolved (channel, channel_user_id) pair.
type Recipient struct {
    Channel       string  // "tgram", "slack", "mailto", ...
    ChannelUserID string  // chat_id, U0xxx, email address, ...
    DisplayName   string  // best-effort, for templating
}

// Action is an interactive UI hint (rendered as URL button, callback button,
// Adaptive Card action, ntfy X-Action, etc. depending on Capabilities).
type Action struct {
    Type   ActionType   // ActionView | ActionURL | ActionCallback | ActionHTTP | ActionCopy
    Label  string
    URL    string       // for ActionView / ActionURL / ActionHTTP
    Method string       // "GET" | "POST" (ActionHTTP)
    Body   []byte       // ActionHTTP
    Data   string       // ActionCallback payload (provider-defined, e.g. Telegram callback_data)
}
```

```go
type ActionType int

const (
    ActionView     ActionType = iota // open a URL
    ActionURL                        // synonym for ActionView;
        provider-specific styling
    ActionCallback                   // round-trip back into
        Herald via inbound handler
    ActionHTTP                       // fire-and-forget HTTP
        request from the recipient device (ntfy)
    ActionCopy                       // copy text to clipboard
        (ntfy)
)

// Priority maps to ntfy 1..5 and to per-channel native
        priorities.
type Priority int

const (
    PriorityLow     Priority = 1
    PriorityNormal  Priority = 3
    PriorityHigh    Priority = 4
    PriorityUrgent  Priority = 5
)

// Receipt is what Channel.Send returns on success — the
        adapter's evidence
// of acceptance/routing/delivery so the router can decide
        whether to retry.
type Receipt struct {
    Evidence       DeliveryEvidence
    ChannelMsgID   string              // Slack ts; Telegram
        message_id; SMTP queue-id; ...
    SentAt         time.Time
    LatencyMillis  int64
    Native         map[string]any      // adapter-specific raw
        response (for diary)
}

// InboundHandler receives messages emitted by the adapter's
        Subscribe loop.
// Implementations enqueue events back into the router (§5).
type InboundHandler interface {
    Handle(ctx context.Context, ev InboundEvent) error
}

// InboundEvent is a CloudEvent constructed from a subscriber
        message.
```

```go
type InboundEvent struct {
    EventID         string                  // UUIDv7
    CloudEvent      CloudEventEnvelope      // §4.1
    Sender          Recipient               // who sent it
    Subscriber      *Subscriber             // resolved via
        subscriber_aliases, nil if unknown
    Body            Body
    Attachments     []Attachment
    Thread          *ConversationRef
    Raw             map[string]any          // adapter-specific
        raw payload (for diary)
}

// Subscriber is the in-memory projection of one row from
        `subscribers`
// (§7.1) plus all linked `subscriber_aliases` for the resolved
        channel.
// Pointer fields are nullable; consumers MUST nil-check before
        use.
type Subscriber struct {
    ID           uuid.UUID
    TenantID     uuid.UUID
    Handle       string                 // empty if not operator-
        mapped
    DisplayName  string
    Locale       string                 // BCP-47, e.g. "en-US", "sr-
        Latn-RS"
    Timezone     string                 // IANA, e.g. "Europe/
        Belgrade"
    Roles        []string               // e.g.
        ["operator","ic","reader"]
    Metadata     map[string]any         // free-form per-subscriber
        data
    Aliases
        []SubscriberAlias  // all channels this human is
        reachable on
    Preferences  *PreferenceSet         // see §7.2; nil ⇒ tenant
        defaults apply
}

// SubscriberAlias is one row from `subscriber_aliases`.
type SubscriberAlias struct {
    Channel       string                // "tgram", "slack",
        "mailto", ...
    ChannelUserID string                // chat_id, U0xxx, email
        address, ...
    VerifiedAt    *time.Time            // nil ⇒ operator-mapped
        (not self-verified)
```

```go
    LastSeenAt    *time.Time
}

// CloudEventEnvelope is Herald's typed projection of a
        CloudEvents v1.0
// payload. SDK type used at the boundaries is cloudevents.Event
        from
// github.com/cloudevents/sdk-go/v2 — this struct is the in-
        process
// canonical form after parsing/validation.
type CloudEventEnvelope struct {
    SpecVersion      string                // "1.0"
    ID               string                // UUIDv7 (natural
        ordering)
    Source           string                // URI
    Type
        string                  // reverse-DNS, e.g.
        "digital.vasic.herald.ci.failed"
    Time             time.Time             // RFC 3339
    Subject          string                // tag:/channel:/
        empty
    DataContentType  string                // e.g.
        "application/json"
    Data             []byte                // opaque payload
    Extensions       map[string]string     // heraldtenant,
        heraldidempotencykey, heraldpriority, ...
}

// TraceContext carries OpenTelemetry trace propagation across
        Herald
// boundaries (HTTP ingress → router → River job → channel
        adapter).
// Stored alongside messages so spans link correctly even when a
        job
// runs asynchronously minutes after the originating request
        returned.
type TraceContext struct {
    TraceID    string  // 32-hex chars (W3C Trace Context)
    SpanID     string  // 16-hex chars (the parent span at
        handoff)
    TraceFlags byte    // sampling flags (W3C)
    TraceState string  // vendor-specific (W3C tracestate header)
    Baggage    string  // W3C baggage header value
}

// Branding is the per-flavor visual identity (§6.3 reference).
// One Branding is constructed per flavor binary at startup via
```

```go
// commons.DefaultBranding(<flavor>, version) and threaded through
// OutboundMessage so adapters render channel-specific bling. The Wave 2
// (r8) fields below also drive the shared commons/cli/ scaffold (Cobra
// metadata + default HTTP serve-port binding + version --json shape).
type Branding struct {
    AppName        string  // "Project Herald", "System Herald", ...
    BinaryName     string  // "pherald", "sherald", ...
    IconURL        string  // for rich embeds (Slack auth user, Discord author, ...)
    AccentColorHex string  // "#2C7BE5" — used as Slack attachment color, Discord embed color, Adaptive Card accent
    DefaultFooter  string  // "Sent by pherald 1.0 · github.com/vasic-digital/Herald"

    // Wave 2 per-flavor identity fields (r8).
    Flavor      string  // single-letter (or short) flavor key: "p", "s", "b", "sc", ...
    Prefix      string  // 3-letter §8.2 anchor (e.g. "PHR", "SHR", "CHR")
    DisplayName string  // operator-facing display name (typically == AppName)
    DefaultPort int     // default HTTP listen port (24791..24799 block; 0 for CLI-only flavors)
    Mission     string  // one-line mission statement for --help / about
}

// ChannelID is the canonical channel identifier. It MUST match the
// scheme used in `channel_addresses.address_url` and in the URL
// scheme registered by the adapter (e.g. "tgram", "slack", "mailto").
type ChannelID string

const (
    ChannelTelegram ChannelID = "tgram"
    ChannelMax      ChannelID = "max"
    ChannelSlack    ChannelID = "slack"
    ChannelDiscord  ChannelID = "discord"
    ChannelTeams    ChannelID = "teams"
    ChannelLark     ChannelID = "lark"
```

```go
    ChannelWhatsApp ChannelID = "whatsapp"
    ChannelViber    ChannelID = "viber"
    ChannelEmail    ChannelID = "mailto"
    ChannelNtfy     ChannelID = "ntfy"
    ChannelGotify   ChannelID = "gotify"
    ChannelWebhook  ChannelID = "webhook"
    ChannelDiary    ChannelID = "diary"
    ChannelNull     ChannelID = "null"  // §11.14 sandbox/no-op adapter for tests
)

// PreferenceSet is a typed view of the per-subscriber preferences JSON
// stored in `subscribers.metadata.preferences` (§7.2). See §7.2 for the
// JSON shape; this is the Go decoded form.
type PreferenceSet struct {
    Categories  map[string]CategoryPref   // category_id → pref
    Workflows   map[string]WorkflowPref   // CloudEvents type → pref
    QuietHours  *QuietHours               // nil ⇒ no quiet hours configured
    ChannelData map[ChannelID]any         // provider-specific routing data
}

type CategoryPref struct {
    Channels []ChannelID
    Muted    bool
}

type WorkflowPref struct {
    Channels []ChannelID  // may be empty (= use Category default)
    Muted    bool
}

type QuietHours struct {
    TZ                 string    // IANA TZ
    Start              string    // "HH:MM" 24h
    End                string    // "HH:MM" 24h
    ExemptCategories   []string  // categories that override quiet hours
}
```

These types live in `commons/types.go` and `commons/preferences.go`. Every adapter under `commons_messaging/channels/<name>` consumes them; no adapter is allowed to invent its own equivalent (the contract is the contract). Additional helpers in `commons/`

`cloudevents.go` (CloudEventEnvelope ⇄ cloudevents.Event), `commons/branding.go` (per-flavor Branding factory), `commons/trace.go` (TraceContext propagation), `commons/uuidv7.go` (UUIDv7 generator).

> **Wave 7 inbound-runtime extension (channels.Channel).** The `commons.Channel` above is the outbound contract every flavor's `serve` path consumes. The inbound runtime (`pherald listen`, §32) needs three additional per-adapter methods — quoted `SendReplyGeneric` (reply-anchored send with attachment fan-out), `BotSelfIdentity` (the anti-echo self-identity, §32.9), and content-addressed `DownloadAttachment`. These live on the richer `commons_messaging/channels.Channel` interface, which **embeds** `commons.Channel` and adds only the inbound methods. Adapters self-register with the `commons_messaging/channels` name → constructor registry via `init()`; the generalized self-filter `channels.IsSelfEcho` compares each adapter's native identity (Telegram username / Slack `bot_user_id` / Email From address) uniformly via `SelfIdentity{Kind: IdentityUsername | IdentityUserID | IdentityAddress, Value: …}`. Per-channel attachments land under ~/.herald/inbox/<channel>/<sha256>.<ext> so multi-channel runs cannot collide. The first two concrete adapters under this richer interface are Telegram (`commons_messaging/channels/tgram/`, Wave 6) and Slack (`commons_messaging/channels/slack/`, Wave 7 T6 — Socket Mode).

11.1 Telegram

- **SDK**: `gopkg.in/telebot.v3` (tucnak/telebot) primary; `github.com/mymmrac/telego` alt for 1:1 Bot API fidelity.
- **V1 (MVP)** features:
  - `sendMessage` with MarkdownV2 parse mode.
  - `sendPhoto` / `sendDocument` for attachments.
  - `InlineKeyboardMarkup` with URL buttons + `callback_data`.
- **V2 advanced**:
  - `callback_query` handler (interactive replies).
  - `editMessageText` (in-place updates for ongoing incidents).
  - `web_app` buttons (open Mini-Apps).
  - `sendMediaGroup` (galleries).
  - Forum-topic threads via `message_thread_id`.
- **Out-of-scope** (V2): full Mini-App SDK, payments, Telegram Passport.
- **Delivery evidence ceiling**: `Routed` via Bot API; `Read` only via Business Bot connection with `can_read_messages` right (separate setup).
- **Webhook secret**: setWebhook?secret_token=<token>; verify `X-Telegram-Bot-Api-Secret-Token` header on inbound.

11.2 Max (max.ru)

- **SDK**: `github.com/max-messenger/max-bot-api-client-go` (official-adjacent, Apache-2.0, English+Russian docs).
- **Gating**: behind build tag `herald_max` and documented sanctions advisory in the operator manual. VK Group parent `VK Company Limited` is not on the OFAC SDN list at time of writing, but several VK-affiliated individuals are designated and EU restrictive measures evolve frequently — operators must check at deploy time.
- **Bot registration**: via in-app `MasterBot`.
- **V1**: send text message, send media; basic inline keyboard.

- **V2 advanced**: per `dev.max.ru` docs as they expand.
- **Delivery evidence ceiling**: `Routed` (API ack = stored & queued).

11.3 Slack

- **SDK**: `github.com/slack-go/slack` (pin ≥ v0.23.1 — GHSA-gxhx-2686-5h9g fixed empty-secret bypass).
- **V1**:
  - `chat.postMessage` with Block Kit `header` + `section` + `divider` + `image` + `actions` (URL buttons only).
  - Threading via `thread_ts` (R-08 mapping).
- **V2 advanced**:
  - `block_actions` callback handler (interactivity).
  - `views.open` modals.
  - Message shortcuts.
  - Socket Mode for daemon ingress.
- **Out-of-scope**: Workflow Builder steps, Slack Connect, Enterprise Grid management.
- **Delivery evidence ceiling**: `Routed` (`ok:true` + `ts` proves posted to channel; no per-user read event).
- **Signing**: HMAC-SHA256 verification using `slack.NewSecretsVerifier` over `v0:{X-Slack-Request-Timestamp}:{rawBody}`, comparing with `X-Slack-Signature`, 5-min replay window.

11.4 Discord

- **SDK**: `github.com/bwmarrin/discordgo` (~5.7k stars, BSD-3-Clause, stable).
- **V1**:
  - Incoming Webhook with embeds (title, description, fields, footer, thumbnail, color).
  - Threads via webhook + `?thread_id=`.
- **V2 advanced**:
  - Bot token; components (buttons + select menus).
  - Slash commands.
  - Forum channels.
- **Out-of-scope**: modals, voice, stage channels.
- **Delivery evidence ceiling**: `Routed`.

11.5 Microsoft Teams

- **SDK**: `github.com/atc0005/go-teams-notify/v2` (incoming-webhook-only — **no official Microsoft Go SDK** for full Graph access).
- **V1**:
  - Incoming Webhook with Adaptive Card v1.5: `TextBlock`, `Image`, `Container`, `FactSet`, `ActionSet` with `Action.OpenUrl`.
- **V2 advanced**:
  - `Action.Execute` / Actionable Messages with HMAC-signed tokens.
  - Bot Framework via Azure Bot Service (separate setup, complex).
- **Out-of-scope**: full Bot Framework conversational state, meeting extensions.
- **Delivery evidence ceiling**: `Routed`.

11.6 Lark / Feishu

- **SDK**: `github.com/larksuite/oapi-sdk-go` (**official**, MIT, code-gen flavor — verbose but accurate).

- **V1**: send text + rich message + image; basic interactive cards.
- **V2 advanced**: Mini-program cards, chatbot @-mentions, group management.
- **Delivery evidence ceiling**: `Routed`.

11.7 WhatsApp

Two transports for two use cases:

- **WhatsApp Web multi-device** — `go.mau.fi/whatsmeow` (`tulir/whatsmeow`, ~5.5k stars, MPL-2.0). Reverse-engineered WA Web; **not** for official Business API.
- **WhatsApp Business Cloud API** — `github.com/twilio/twilio-go` (official, multi-product; WA via Twilio Senders).
- **V1**: text + media via either transport; template-message support on Business Cloud API.
- **V2 advanced**: button-template messages, list messages, conversation pricing windows.

11.8 Viber

- **No official Go SDK.** Community: `mileusna/viber`, `strongo/bots-api-viber`.
- **V1**: hand-rolled REST client against the documented Viber REST API. Send text + media + carousel.
- **V2 advanced**: rich-media keyboards.

11.9 Email (deep)

The most operationally complex channel — Email gets the deepest treatment.

Two interchangeable transports behind one `Channel` interface:

1. **Raw SMTP + DKIM** — `net/smtp` (stdlib) + `github.com/emersion/go-msgauth/dkim`. Suitable for self-hosted operators.
2. **ESP HTTP API** — Resend (preferred), Postmark (fallback), SendGrid (alternative). Better deliverability + built-in bounce/complaint webhooks + suppression lists.

Mandatory features for every outbound email (V1):

- **DKIM signing** (RFC 6376) via `go-msgauth/dkim`.
- **List-Unsubscribe** (RFC 8058) — `List-Unsubscribe: <https://…>, <mailto:…>` + `List-Unsubscribe-Post: List-Unsubscribe=One-Click`. DKIM-signed.
- **Plain-text alternative** generated from the HTML body (MJML-rendered).
- **Inline images** via `Content-ID:` MIME parts (`cid:` references in HTML).
- **Suppression list lookup** — consult `email_suppressions` table before send; skip suppressed addresses and log to diary.

Bounce pipeline:

- Inbound mailbox parsed for RFC 3464 DSN parts, OR ESP webhook consumed for bounce/complaint/unsub events.
- Hard bounces, complaints, and manual unsubscribes write to `email_suppressions`:

```sql
CREATE TABLE email_suppressions (
    tenant_id     UUID NOT NULL,
    address       TEXT NOT NULL,             -- normalized
        lowercase
```

```
    reason       TEXT NOT NULL,            -- 'hard_bounce' |
        'complaint' | 'unsubscribe'
    source_event UUID,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, address)
);
ALTER TABLE email_suppressions ENABLE ROW LEVEL SECURITY;
ALTER TABLE email_suppressions FORCE ROW LEVEL SECURITY;
```

Doctor command: `<flavor>herald doctor email` verifies SPF, DKIM, DMARC records for the configured sending domain at startup. Without correct DNS, Gmail/Yahoo's 2024 sender requirements quietly bin the mail.

Templating: MJML (`<mj-section>`, `<mj-column>`, `<mj-image>`, `<mj-button>`) compiled to inline-CSS HTML at **template-author time** (vendored MJML CLI; check in the compiled HTML alongside the `.mjml` source). The compiled HTML then runs through `html/template` for variable substitution at send time. No Node.js in the runtime container.

Delivery evidence ceiling: `Accepted` by default (relay 250 OK), upgradable to `Delivered` by parsing DSN, `Read` only when recipient voluntarily returns RFC 8098 MDN (rare).

11.10 ntfy / Gotify

Per research Topic 3 (notification-platforms): adopt ntfy's wire model as both an **inbound ingest format** (alongside CloudEvents) and as a first-class outbound channel.

- Map: `X-Priority` (1–5) → Herald priority enum; `X-Tags` → Herald tags; `X-Click` → action URL; `X-Actions` (view/http/broadcast/copy) → Herald `Action` schema; `X-Attach`/PUT body → attachment.
- **Gotify** application/client token split: application token authenticates publishers; client token authenticates subscribers; both revocable independently.
- **V1**: send to ntfy/gotify topic with priority + tags + attachment.
- **V2 advanced**: receive via WebSocket / SSE for live updates.

11.11 Generic outbound webhook

Adapter `webhook://<url>?secret=<hmac>&format=<cloudevent|json|form>`:

- Sends Herald events as CloudEvents (binary or structured mode), or as plain JSON, or as form-encoded — chosen by query param.
- HMAC-signs outbound requests (`Webhook-Signature` header) so receiving side can verify.
- Retries per §5.4.

11.12 Diary (Markdown + PDF + HTML)

See §19 for full schema and sync strategy.

11.13 Feature matrix summary

Channel	Text	Markdown	Attachments	Threads	Interactive buttons	Signed-source verify	Delivery ceilin
Telegram	✓	MarkdownV2 / HTML	✓	forum topic_id	✓ V1 url, V2 callback	secret_token	Routed (Read v
Max	✓	platform-specific	✓	V2	V2	tba	Routed
Slack	✓	Block Kit	✓	thread_ts	✓ V1 url, V2 callback	X-Slack-Signature	Routed
Discord	✓	embeds	✓	thread_id	webhook url, bot components	webhook URL secrecy	Routed
MS Teams	✓	Adaptive Card	inline	conv references	OpenUrl / Action.Execute	webhook secret	Routed
Lark	✓	rich card	✓	V2	interactive card	event subscription	Routed
WhatsApp	✓	text/template	✓ (Cloud API)	conversation window	Cloud API templates	Twilio signing / Meta token	Routed
Viber	✓	rich-media	✓	—	rich-media keyboard	sig param	Routed
Email	✓	MJML→HTML	✓ (MIME)	In-Reply-To/ References	url buttons only	DKIM/SPF/ DMARC	Accepted→Del
ntfy	✓	X-Markdown	✓	—	X-Actions	basic auth / token	Routed
Gotify	✓	extras.client::display	✓	—	extras actions	bearer token	Routed
Webhook	✓	CloudEvent JSON	embedded	n/a	n/a	HMAC	Accepted/Route
Diary	✓	source format	path-refs	logical via parent ref	n/a	local FS	n/a (always-on)
`null://`	✓	✓	✓ (counted only)	✓	✓ (recorded)	n/a	configured ceili

11.14 `null://` sandbox channel (test-only)

The `null://` adapter is the in-process equivalent of `/dev/null` with full instrumentation. It implements the entire `Channel` interface but performs no I/O — every `Send` call records the `OutboundMessage` to an in-memory ring buffer (configurable size; default 1000), increments per-tag counters, and returns the configured `DeliveryEvidence` ceiling.

Use cases:

- **Unit tests** for `commons_messaging` routing: assert that a given event with a given preference set produces the expected fan-out without touching any real channel.
- **Load tests**: route traffic through `null://` to measure router/queue throughput without hitting upstream rate limits.
- **Quickstart / training**: operators can send test events to confirm routing logic before adding real channel credentials.

- **Chaos testing**: configure null:// to return error for X% of sends to exercise retry / dead-letter paths.

URL grammar:

```
null://[?seed=<int>&fail_rate=<0..1>&latency_ms=<int>&ceiling=<Accepted|Ro
```

Examples:

- null://?tags=test — happy path, instant, returns Routed.
- null://?fail_rate=0.1&tags=chaos — 10% of sends return transient error (exercises retry).
- null://?latency_ms=500&ceiling=Delivered&tags=load — adds 500 ms artificial latency, claims Delivered ceiling.

Inspector API (test-only HTTP endpoint, mounted when [testing].null_inspector=true):

Method	Path	Purpose
GET	/admin/null/messages	Recent ring-buffer contents (last N OutboundMessages).
GET	/admin/null/stats	Per-tag counters + send/fail tally.
POST	/admin/null/clear	Empty the ring buffer + reset stats.
POST	/admin/null/inject	Inject a synthetic inbound message (test the inbound path without a real subscriber).

null:// MUST NOT be enabled in production deployments — the operator gate CM-NULL-CHANNEL-DISABLED-IN-PROD (planned) verifies that null:// is absent from channel_addresses when [herald].environment=production. Operators that need null behaviour in prod (e.g., feature-flagged channels) use the [channel_address].enabled=false toggle instead.

Channel adapter test fixtures pattern. Every commons_messaging/channels/<name>/ package SHOULD ship:

- testdata/ — recorded HTTP request/response pairs (go-vcr-compatible cassettes).
- <name>_test.go — unit tests against testdata/ (no network).
- <name>_integration_test.go (build tag integration) — runs against the channel's sandbox/test mode (Telegram test bot, Slack workspace T0000 test team, etc.).
- <name>_e2e_test.go (build tag e2e) — only runs in nightly CI against real credentials in a dedicated test tenant.

---

§12. Messaging flows

Three mandatory message flows per channel (where the channel API supports them):

- **Simple message** — textual content sent to the channel; displayed to all subscribers.
- **Message with attachment(s)** — content + 1..N attachments; subscribers can download attachments.

- **Quote / reply message** — content sent as a reply to an existing channel message (uses §11 thread mapping: Telegram `reply_to_message_id`, Slack `thread_ts`, Discord `thread_id`, Email `In-Reply-To`); may include 0..N attachments.

Subscriber inbound flows (Project Herald and any flavor that opts into reply processing):

- **Direct reply to a Herald message** — parsed for commands (`Bug:`, `Issue:`, `Query:`, `Request:`, `Question:`).
- **Standalone message tagged for Herald** — same parsing, no parent message.
- **Thread continuation** — full thread context (chained replies from start) is parsed and processed.

Conversation reference per R-08:

```
type ConversationRef struct {
    Channel         ChannelID  // "tgram", "slack", ...
    ThreadID        string     // Slack thread_ts; Telegram
        message_thread_id (forum); Email References[0]
    ParentMessageID string     // Slack ts; Telegram
        reply_to_message_id; Email In-Reply-To
    RootMessageID   string     // first message in the thread
}
```

Persisted in the `thread_refs` table so re-sends to the same logical thread find the right native handle on every channel:

```
CREATE TABLE thread_refs (
    tenant_id          UUID NOT NULL,
    logical_thread_id  UUID NOT NULL,                    --
        Herald's stable identifier across channels
    channel            TEXT NOT NULL,                    --
        "tgram", "slack", "email", ...
    thread_id          TEXT,                             -- Slack
        thread_ts, Telegram message_thread_id (forum), Email
        References[0]
    parent_message_id  TEXT,                             -- Slack
        ts (parent), Telegram reply_to_message_id, Email In-
        Reply-To
    root_message_id    TEXT,                             -- first
        message in the thread
    last_activity_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, logical_thread_id, channel)
);
CREATE INDEX thread_refs_recent_idx ON thread_refs (tenant_id,
        last_activity_at DESC);
ALTER TABLE thread_refs ENABLE ROW LEVEL SECURITY;
ALTER TABLE thread_refs FORCE ROW LEVEL SECURITY;
CREATE POLICY tr_isolation ON thread_refs
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

A *logical thread* is Herald's tenant-stable conversation identifier; the per-channel rows map it to native handles. `last_activity_at` enables time-window garbage collection (default: 90 days after last activity, configurable).

---

§13. Templating & message composition

Three-layer stack per research Topic 5 (notification-platforms):

- **Layer 0 (engine)** — Go stdlib `text/template` (plaintext / Markdown / JSON channels) and `html/template` (HTML email body, auto-escaping).
- **Layer 1 (helpers)** — Sprig (`date`, `upper`, `default`, `trunc`, etc.) — universally expected by template authors; used by Helm, kubectl templates.
- **Layer 2 (email-specific)** — MJML compiled to HTML at template-author time (vendored MJML CLI; check in the compiled `.html` alongside `.mjml`), then `html/template` for runtime substitution.

Rejected: Liquid (extra runtime, no Go stdlib affinity); runtime MJML compilation (Node dependency in Go hot path).

Template directory layout:

```
commons_messaging/templates/
├── ci.failed/
│   ├── tgram.md.tmpl
│   ├── slack.json.tmpl       # Block Kit JSON
│   ├── email.mjml            # source
│   ├── email.html            # compiled, committed
│   └── diary.md.tmpl
├── deploy.succeeded/
│   └── …
└── _shared/                  # template helpers
    └── footer.tmpl
```

---

§14. Localization (i18n)

Per research Topic 6 (notification-platforms): `nicksnyder/go-i18n` **v2**.

- One `Bundle` loaded at process start from `commons_messaging/locales/*.toml`:

```
# locales/active.en-US.toml
[ci.failed.title]
other = "Build failed: {{.JobName}}"

# locales/active.sr-Latn-RS.toml
[ci.failed.title]
other = "Build pao: {{.JobName}}"
```

- Plurals handled via CLDR rules (built-in to go-i18n): `1 alert / 2 alerta / 5 alerti / 21 alert` for Serbian works correctly without custom code.
- Per-subscriber `locale` (BCP-47 tag) stored on the `subscribers` row.

- Per-channel templates may override (Telegram emoji-heavy variant vs sober email variant).

V2 initial locales: `en-US`, `sr-Latn-RS`, `ru-RU`. RTL rendering tweaks and dynamic locale negotiation from incoming events are deferred.

---

§15. Security model

Per R-16: a **three-layer pipeline** runs *before* any routing logic.

15.1 Transport-level (channel signature verification)

Per-channel signature verifier (constant-time HMAC comparison via `crypto/hmac.Equal`):

Channel	Header	Algorithm	Replay window
Slack	`X-Slack-Signature`	HMAC-SHA256 over `v0:{ts}:{body}`	5 min
GitHub-style	`X-Hub-Signature-256`	HMAC-SHA256 over body	configurable
Telegram	`X-Telegram-Bot-Api-Secret-Token`	shared-secret compare	n/a (no timestamp)
Stripe	`Stripe-Signature`	HMAC-SHA256 over `{ts}.{body}`	5 min
Email	DKIM-Signature header + DMARC alignment	RSA-SHA256 / Ed25519	per DKIM TTL

Generic webhook sources register their own per-source secret in `webhook_sources`.

15.2 Sender-level (allowlist + verified subscribers)

After transport verification, sender authority is checked:

- Per-channel allowlist keyed by canonical user-id (Slack `team_id+user_id`, Telegram `chat_id`, email DKIM-aligned `From`) loaded from `.herald/subscribers.toml` and/or the `subscribers` + `subscriber_aliases` tables.
- Unknown senders enter a **quarantine queue** (`quarantined_messages` table), never the live fan-out.
- An operator (or auto-policy) reviews the quarantine and either promotes the sender or discards.

`quarantined_messages` **schema:**

```sql
CREATE TABLE quarantined_messages (
    id                UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id         UUID NOT NULL,
    received_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    channel           TEXT NOT NULL,
    sender_channel_user_id TEXT NOT NULL,
    sender_display    TEXT,
```

```sql
    reason          TEXT NOT NULL,                          --
        'unknown_sender' | 'unverified_signature' |
        'rate_limited' | 'content_flagged'
    payload_jsonb   JSONB NOT NULL,                         -- full
        inbound message
    triage_status   TEXT NOT NULL DEFAULT 'pending',
        -- 'pending' | 'promoted' | 'discarded' | 'expired'
    triaged_at      TIMESTAMPTZ,
    triaged_by      UUID                                    --
        subscriber id of operator
);
CREATE INDEX qm_pending_idx ON quarantined_messages (tenant_id,
        received_at DESC)
    WHERE triage_status = 'pending';
ALTER TABLE quarantined_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE quarantined_messages FORCE ROW LEVEL SECURITY;
CREATE POLICY qm_isolation ON quarantined_messages
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

Pending entries auto-expire after 30 days (`triage_status='expired'`) so the queue doesn't grow unbounded on a noisy channel. CLI: `<flavor>herald quarantine [list | promote <id> | discard <id> | purge]`.

15.3 Content-level (parsing & sanitization)

After sender verification, content is parsed:

- Commands (`Bug:`, `Issue:`, `Query:`, `Request:`, `Question:`, plus flavor-specific verbs) parsed with a strict regex-anchored tokenizer.
- **Never** shell out with user-provided content.
- **Never** `text/template` user input into shell or SQL.
- Command bodies are length-bounded (default 8 KiB).
- Attachments are scanned for MIME mismatch + size limits (default 25 MiB per attachment, 100 MiB per message).
- Optional malware scan via ClamAV when `commons_security` is configured with the integration.

15.4 Credential handling

Per Constitution §11.4.10:

- `.env` is git-ignored (the `.gitignore` MUST contain `.env`).
- `.env.example` is committed.
- Resolution order: CLI flag > shell env > `.env` > compiled default (§3.3).
- Credentials NEVER logged. The OTel logger has a redaction middleware that filters keys matching `(?i)(token|secret|password|key|credential|authorization)` to `***`.
- Credentials in Postgres are encrypted-at-rest with `pgcrypto` (`pgp_sym_encrypt`) using a key from `HERALD_DB_ENC_KEY` env var.

15.5 Webhook ingestion (defense-in-depth)

See §5.5. Four ordered checks: signature → timestamp → delivery-ID dedup → optional IP allowlist.

---

§16. Multi-tenancy & isolation

Per research Topic 6 (event-fanout) + Topic 4 (operations):

- **Primary boundary**: PostgreSQL Row-Level Security.
- **Every business table** carries tenant_id UUID NOT NULL with a composite index (tenant_id, …) leading every secondary index (RLS is 10–100× slower without it).
- **Policy template**:

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON <table>
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

- **Roles**:
  - herald_app — runtime role, no BYPASSRLS. The pgx connection wrapper in commons_storage runs SET LOCAL app.tenant_id = … at transaction start.
  - herald_migrator — schema-change role, BYPASSRLS.
- **Redis** — per-tenant ACL user (Redis 6+), key pattern ~t:<tenant_id>:*.
- **Rate limits** — token bucket per (tenant_id, channel) in Redis.
- **Schema-per-tenant** is reserved for high-isolation enterprise customers; documented as opt-in but not the default (overhead: connection-pool fragmentation, slow migrations N × M, harder backups).

16.1 Data retention & privacy

Herald processes content that may contain personally identifiable information (names, email addresses, IP addresses, organization data). Retention windows MUST be configurable per tenant and enforced by scheduled River jobs.

Default retention policy (overridable per tenant):

Class	Default retention	Purge mechanism
idempotency_keys	7 d after expires_at (already in §4.3)	nightly River job
dead_letters (triaged)	90 d	nightly River job
dead_letters (untriaged)	365 d (legal/audit floor)	manual triage prompt
quarantined_messages (pending)	30 d → auto expired	nightly River job
quarantined_messages (triaged)	90 d	nightly River job

Class	Default retention	Purge mechanism
`thread_refs`	90 d after `last_activity_at`	nightly River job
`email_suppressions`	indefinite (compliance — hard bounces must persist)	manual remove via CLI
`subscribers` (deleted)	tombstone 30 d, then hard delete	GDPR right-to-erasure flow
Diary entries (`docs/herald/diary/main.md`)	tenant-configurable; default unlimited (git history)	per-tenant rotation policy
OpenTelemetry data	controlled by Collector → backend (out of Herald scope)	n/a

GDPR / privacy mechanics:

- `<flavor>herald subscriber forget <id>` initiates right-to-erasure: tombstones the row, schedules hard-delete after 30 days, anonymises subscriber references in `dead_letters` / `quarantined_messages` / diary entries (replaces `display_name` with `<redacted>`, hashes `channel_user_id`).
- `<flavor>herald subscriber export <id>` (right-to-portability) emits a JSON bundle of every record referencing the subscriber.
- **Diary redaction** — when a subscriber is forgotten, the diary writer SHOULD overwrite their entries with a redaction note (in-place edit + re-export); operators that want immutable audit trails MUST opt-out per `[privacy].diary_redaction_on_forget = false`.

Data sovereignty: the `containers` submodule deploys Postgres + Redis to operator-chosen regions; Herald itself stores nothing outside that pair. Cross-region replication is the operator's choice (logical replication / streaming replication) and is documented in `docs/operations/replication.md`.

---

§17. Observability & SLOs

Per research Topic 1 (operations):

17.1 OpenTelemetry pipeline

- **All three signals** instrumented from day one:
  - **Traces** — `go.opentelemetry.io/otel/trace` spans across the full event flow (ingest → route → enqueue → deliver → ack).
  - **Metrics** — `go.opentelemetry.io/otel/metric` counters + histograms.
  - **Logs** — stdlib `slog` shipped as OTel log records via `go.opentelemetry.io/contrib/bridges/otelslog`.
- **Default export**: OTLP/gRPC to an OpenTelemetry Collector sidecar/DaemonSet on `localhost:4317` (no auth) — production deployments override via env vars.
- **Collector** fans out to Prometheus (scrape `/metrics`), Tempo/Jaeger (traces), Loki (logs).
- Instrumentation lives in `commons_observability` (L1) so every flavor inherits.

Standard OTel SDK env vars Herald honours (per OpenTelemetry SDK spec — values are NOT re-invented):

Variable	Default	Purpose
OTEL_SDK_DISABLED	false	Hard kill switch — w instrumentation is a n
OTEL_EXPORTER_OTLP_ENDPOINT	http://localhost:4317	Single endpoint for a signals (gRPC).
OTEL_EXPORTER_OTLP_TRACES_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_METRICS_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_LOGS_ENDPOINT	inherits	Per-signal override.
OTEL_EXPORTER_OTLP_PROTOCOL	grpc	grpc | http/prot http/json.
OTEL_EXPORTER_OTLP_HEADERS	unset	Authn (e.g. Authorization= …).
OTEL_EXPORTER_OTLP_INSECURE	false	Disable TLS for htt endpoints.
OTEL_RESOURCE_ATTRIBUTES	(Herald sets service.name, service.version, herald.flavor, herald.tenant_id)	Operator adds deployment.env service.namesp
OTEL_SERVICE_NAME	<flavor>herald	Convenience equival service.name.
OTEL_TRACES_SAMPLER	parentbased_traceidratio	Sampler choice.
OTEL_TRACES_SAMPLER_ARG	1.0	Ratio when sampler
OTEL_METRIC_EXPORT_INTERVAL	60000 (ms)	Push cadence.
OTEL_BSP_SCHEDULE_DELAY	5000 (ms)	Batch-span-processo

Herald's own resource defaults:

```
service.name=<flavor>herald
service.version=<git-tag-or-commit-sha>
service.instance.id=<hostname>-<pid>
herald.flavor=<flavor>
herald.tenant_id=<tenant_uuid>          # one-of label, set per request
telemetry.sdk.name=opentelemetry
telemetry.sdk.language=go
```

OTel semantic conventions tracked: **messaging v1.30+** (the spec moved out of experimental in 2025).

17.2 Metrics catalogue

OTel semantic conventions for messaging where they fit, Herald-specific names where they don't:

Metric	Type	Labels
messaging.client.sent.messages	counter	messaging.system="herald", messaging.destination.name= herald.tenant_id, herald.flav
messaging.client.operation.duration	histogram	+ messaging.operation.name=" messaging.operation.type="se
herald_delivery_attempts_total	counter	

Metric	Type	Labels
		`channel`, `outcome={ok,retry,drop,dead_` `tenant_id`
`herald_queue_depth`	gauge	`queue`, `tenant_id`
`herald_retry_backoff_seconds`	histogram	`channel`, `attempt_bucket`
`herald_template_render_duration_seconds`	histogram	`template_id`
`herald_subscriber_count`	gauge	`tenant_id`, `channel`
`herald_dead_letter_count`	gauge	`tenant_id`, `channel`

Cardinality budget: never `recipient_id` or message UUIDs as labels. Per-tenant labels are allowed; per-subscriber labels are NOT.

Canonical Grafana dashboard JSON shipped in `deploy/grafana/herald-overview.json`.

17.3 Span model

Canonical span names (and parent→child relationship):

```
herald.ingest              (root, per inbound CloudEvent)
├── herald.signature_verify
├── herald.idempotency_check
├── herald.route_match
├── herald.preference_filter
├── herald.template_render
├── herald.enqueue
└── herald.deliver
    ├── herald.channel.<name>
    │   └── herald.channel.<name>.api_call
    └── herald.diary_append
```

Attributes on every span: `herald.tenant_id`, `herald.flavor`, `cloudevents.type`, `cloudevents.id`.

17.4 SLOs

V2 published SLOs (per-tenant rolling 30-day):

SLO	Target
Ingress availability (HTTP 2xx + 4xx vs 5xx)	99.9%
End-to-end delivery (ingest → channel API `Accepted`) — p95	< 5 s
End-to-end delivery — p99	< 30 s
Delivery success rate (excluding 4xx from upstream channels)	99.5%
Dead-letter rate	< 0.1%
Doctor-CLI checks (SPF/DKIM/DMARC, DB ping, Redis ping)	run every 5 min, alert on 3 consecutive fail

17.4.1 Per-channel SLO budgets

The aggregate SLO table above can hide bad behaviour on one channel behind good behaviour on another. V2 also commits to **per-channel SLO budgets** — error budgets tracked independently so a single misbehaving adapter or upstream incident is visible immediately:

Channel	Delivery success target	p95 latency target (ingest → upstream `accepted`)	Notes
Telegram	99.5%	< 2 s	Telegram Bot API is famously reliable; alert if budget burn > 2× expected.
Slack	99.5%	< 3 s	`chat.postMessage` rate-limited per workspace; back off cleanly.
Discord	99.0%	< 3 s	Webhook rate limits are aggressive; expect retries.
MS Teams	98.0%	< 5 s	Incoming-webhook backend has visible jitter.
Lark	99.0%	< 4 s	
Max	99.0%	< 4 s	Subject to network reachability from non-RU operator regions.
Email (SMTP self-hosted)	95.0%	< 30 s	Includes DNS + greylist + tarpit latency.
Email (ESP — Resend/Postmark/SendGrid)	99.5%	< 5 s	ESP carries the deliverability risk; Herald sees `Accepted` quickly.
WhatsApp (Cloud API)	99.0%	< 3 s	Conversation-window restrictions can drop messages outside 24h windows.
Viber	98.0%	< 5 s	Hand-rolled REST client; expect rougher edges.
ntfy / Gotify	99.5%	< 2 s	Self-hosted = operator's own infra.
Webhook (generic outbound)	depends on operator's endpoint	< operator's deadline	Tracked but no Herald-side target.
Diary	99.99%	< 100 ms	Local file system + Pandoc batching; outliers indicate disk pressure.

Per-channel burn-rate alerts (multi-window, multi-burn-rate per Google SRE Workbook): page on **2-hour 14× burn** AND **6-hour 6× burn** simultaneously crossed; warn on either alone. Alerts route through the same Herald pipeline that any other event uses — Herald-monitoring-Herald MUST work, but operators are advised to also run a separate stand-by notifier for the case where Herald itself is the failure.

17.5 Health probes (livez / readyz / startupz)

Per research Topic 3 (operations): three endpoints on the **admin port** (`24090` default, separate from data port so probes don't compete for worker capacity):

- `GET /livez` — returns `200` unless an unrecoverable internal invariant trips (deadlock detector, fatal goroutine panic flag). Cheap, no downstream calls.

- `GET /readyz` — checks Postgres + Redis ping, queue consumer attached, ≥1 channel adapter healthy. Returns `503` during graceful shutdown.
- `GET /startupz` — used by Kubernetes startup probe while migrations / warm-up run; once `200`, liveness/readiness take over.

Liveness `failureThreshold` ~3× readiness so a transient downstream blip removes the pod from Service but does NOT restart it.

17.6 `doctor` CLI

`<flavor>herald doctor` runs a battery of environment checks:

- Postgres connectivity + RLS policy presence.
- Redis connectivity + ACL.
- All configured channel credentials valid (probes API per channel).
- DKIM/SPF/DMARC DNS records for the configured sending domain (cite Gmail/Yahoo 2024 sender requirements).
- OTLP collector reachable.
- Disk space, port availability.

Exits with non-zero status code on any failure + writes a structured report to stdout (machine-readable JSON option `--json`).

---

§18. Flavors (the implementations)

18.0 Wave 2 — Flavor scaffold catalogue (landed 2026-05-21)

Wave 2 (r8) lands the shared `commons/cli/` package and 7 flavor binaries (1 refactor + 6 net new). Every flavor's `cmd/<flavor>/main.go` is ~30 LOC: construct `commons.DefaultBranding(<flavor>, version)`, build the root command via `cli.NewRootCmd(branding)`, register flavor-specific stubs from `internal/stubs/stubs.go`, and (for serving flavors) call `cli.ServeCmd(opts)` with `internal/http.Routes()`.

Flavor	DisplayName	Default port	Serves?	§43 stubs (count)	Flavor-specific routes
`pherald`	Project Herald	24791	✓	6 (HRD-029, 030, 043, 044, 049, 053)	`POST /v1/events` (Wave 3b live — 202 Accepted + Receipt JSON; HRD-016 closed)
`sherald`	System Herald	24793	✓	5 (HRD-033, 034, 040, 046, 056)	`GET /v1/safety_state` (HRD-098)
`cherald`	Constitution Herald	24792	✓	11 (HRD-036..039, 042, 048, 050..052, 054, 055)	`GET /v1/compliance` (HRD-028)
`iherald`	Incident Herald	24794	✓	0	`POST /v1/webhooks/page` (HRD-024)
`bherald`	Build Herald	n/a	✗	3 (HRD-035, 041, 045)	—
`rherald`	Release Herald	n/a	✗		—

Flavor	DisplayName	Default port	Serves?	§43 stubs (count)	Flavor-specific routes
				3 (HRD-031, 032, 045)	
scherald	Scheduled-audit Herald	n/a	✗	1 (HRD-047)	—

Stub total: 28 unique §43 HRDs + 1 cross-flavor alias (HRD-045 shared bherald ? rherald) = **29 stub registrations**.

Port allocation: 24791..24799 (2479X block). Reserved: pherald 24791, cherald 24792, sherald 24793, iherald 24794. Free: 24795..24799 for future flavors.

The shared CLI scaffold lives in commons/cli/ (NewRootCmd, VersionCmd, ServeCmd with Middleware hook, StubCmd, Healthz/Readyz/Metrics handlers, Route + StubRouteHandler). See §44.M for the as-built evidence + HRD-092 catalogue-check verdict (no-match → vendor as Herald-internal package).

18.1 Common flavor contract

Every flavor binary MUST:

- Be named <prefix>herald and built from <prefix>herald/cmd/<prefix>herald/main.go.
- Implement the common Cobra-subcommand surface (version baseline; serving flavors also expose serve; other subcommands — send, doctor, migrate, deadletter, subscriber, digest — are stubbed via cli.StubCmd until their owning HRD lands).
- Consume the shared commons/cli/ package: root Cobra command + version subcommand + (for serving flavors) the serve subcommand are provided by cli.NewRootCmd(branding) / cli.VersionCmd(branding) / cli.ServeCmd(opts) respectively. The shared scaffold landed in Wave 2 (r8) — see §18.0 + §44.M.
- Embed commons + commons_messaging + commons_storage + commons_security + commons_observability + commons_diary at the same pinned versions.
- Publish a flavor-specific event-type subtree (digital.vasic.herald.<flavor>.*).
- Register flavor-specific subscriber commands (the Bug: / Query: / etc. tokens that map to flavor-specific handlers).
- Provide a flavor-specific HeraldBranding constructed via commons.DefaultBranding(<flavor>, version) (app name, icon, accent color, plus the Wave 2 identity fields Flavor, Prefix, DisplayName, DefaultPort, Mission).
- Ship its own user manual under docs/flavors/<flavor>/.

18.1.1 Common channel-interaction primitives (cross-flavor)

Every flavor inherits these interaction primitives from commons_messaging so subscribers learn one mental model and apply it across channels and flavors. Per-flavor extensions are documented in each flavor's "Channel interactions" sub-section.

Slash / prefix commands — available on every channel that supports them, with text-prefix fallback for channels that don't:

Token	Behaviour	Slack/ Discord native	Text-prefix fallback
/help	List the flavor's command palette + quick-action buttons	slash command	Help: prefix
/status	Snapshot of current state (open items, queue depth, channel health)	slash command	Status: prefix
/whoami	Echo the subscriber's resolved identity, roles, locale, prefs	slash command	Whoami: prefix
/silence <duration>	Mute notifications for the calling subscriber for the given duration	slash command	Silence: <duration> prefix
/unsilence	Cancel a silence	slash command	Unsilence: prefix
/subscribe <category>	Opt the calling subscriber into a category	slash command	Subscribe: <category> prefix
/unsubscribe <category>	Opt out	slash command	Unsubscribe: <category> prefix
/prefs	Open a modal/form to edit PreferenceSet (channels supporting modals); otherwise return current prefs as JSON	slash command + modal	Prefs: prefix returns JSON only

Reaction-based quick ops — for channels that support emoji reactions (Slack, Discord, Telegram, Lark). Adding a reaction to a Herald message triggers the action; removing the reaction undoes it where the action is reversible.

Emoji	Action	Reversible	Required role
👍 / 👌	Ack (acknowledge — silences re-fires for the alert fingerprint for the default window)	yes	reader+
🔇	Silence for the configured default duration ([interactions].default_silence, default 1 h)	yes	reader+
✅	Resolve (closes the workable item or marks the alert resolved)	no	operator
🚨	Escalate (bumps criticality up one level and re-pages)	no	operator
🔁	Reopen (only on resolved items)	no	operator
🐛	Reclassify as bug	yes	operator
📋	Reclassify as task	yes	operator
❓	Reclassify as query	yes	operator
🗑️	Mark as spam → moves to quarantined_messages for operator review	yes	operator

Reactions emit digital.vasic.herald.reply.reaction_applied CloudEvents so the diary + audit trail capture every interaction.

Interactive button actions — for channels that support clickable buttons (Slack Block Kit actions, Discord components, Telegram inline keyboards with callback_data, Teams Adaptive Card Action.OpenUrl / Action.Execute). Herald renders a per-flavor button palette on every "actionable" message; clicks call back through the channel's webhook to Herald's `/v1/interactions/<channel>` endpoint.

Modal forms — for Slack (`views.open`) and Discord (modal interactions), Herald can request structured input (e.g., "what's the reproduction steps for HRD-042?") instead of free-form chat. Falls back to a multi-message conversation on channels without modal support.

Thread / forum-topic affinity — incident-style flavors (`iherald`, `aherald`) pin all related messages to one thread / Telegram forum topic / Slack thread / Discord forum-channel post, so the conversation is self-contained for postmortem readers. Composes with §12 `ConversationRef` + §35 versioned reports.

Capability degradation — if a channel doesn't support a primitive, Herald MUST degrade gracefully (text instructions instead of buttons, etc.) — never silently drop the interaction. The §11.0 `Capabilities` struct declares per-channel support; the router consults it before rendering.

18.2 Project Herald (`pherald`)

Focus: Software-project development lifecycle. Integrates with VCS, code review, task tracking. Designed for AI-CLI-agent + developer-team workflows.

Event subtree: `digital.vasic.herald.project.*` and `digital.vasic.herald.reply.*`.

Sources:

- Git hooks (`post-commit`, `post-merge`, `pre-push`).
- VCS webhooks (GitHub, GitLab, GitFlic, GitVerse — `push`, `pull_request`, `review`, `issue`).
- AI CLI agents (Claude Code, OpenCode, Cursor, Aider) emitting structured progress events.
- Code-review-tool webhooks (CodeRabbit, Greptile, Sourcery).

Subscriber inbound commands (extends the V1 base set):

Command	Behaviour
`Bug:` / `Issue:`	Open an issue (writes to `docs/Issues.md` per Universal §11.4.12).
`Query:` / `Request:` / `Question:`	Request information; routes to LLM-driven research workflow if configured.
`Status:`	Reply with current project status from `docs/Status.md`.
`Continue:`	Pointer to `docs/CONTINUATION.md` for next agent.
`Done:`	Mark referenced item resolved (writes to `docs/Fixed.md`).
`Spec:`	Cite or modify spec section (subject to §23 spec-change rule).
`Run: <tool>`	Authorized operator only — execute an approved tool (gated by allowlist).

Quote-thread processing: Subscriber's reply automatically includes the full thread context (chained replies from bottom to thread start) — full chain parsed and processed per R-08 `ConversationRef`.

Security validation (extends §15):

- Subscriber commands accepted only from verified, allowlisted subscribers.
- `Run:` requires elevated subscriber privilege (`subscriber_aliases.verified_at` plus `roles` array).
- All commands logged to diary with subscriber identity and timestamp.

Derived workable-item prefix (per §8.2): if the consuming project hasn't set one, the algorithm runs over the project name (from `package.json`, `go.mod`, `pyproject.toml`, or the git remote name).

18.2.1 Investigation-before-Fixing flow

When a subscriber reports a `Bug:` / `Issue:` / `Implementation:` request, `pherald` MUST first create an **Investigation workable item** (type `investigation`, status `investigating`) before committing to a final bug/task row. This guards against runaway LLM-generated workable items that turn out to be duplicates, out-of-scope, or non-reproducible.

The full sequence (composes §32 inbound pipeline + §33 LLM dispatch + §8.3 lifecycle):

```
1. Subscriber DMs:  "Bug: telemetry pipe drops every hour"
                                        │
                                        ▼

2. §32 pipeline accepts + classifies → type=bug, criticality=middle
                                        │
                                        ▼  Reply A (queued)  →  "📨 Received. Queue
3. pherald allocates HRD-INV-NN as an investigation item:
       INSERT workable_items (type='investigation', status='investigating'
                              parent_request=<inbound_msg_id>, criticality
       Write row to docs/Issues.md with type='investigation'
                                        │
                                        ▼  Reply B (processing)  →  "⏳ Investigat
4. §33 dispatcher invokes Claude Code with the §33.3 prompt envelope.
   Claude Code analyzes:
     - Reproduces locally? (where: <project> session, working dir)
     - Identifies affected code paths.
     - Classifies root-cause area.
     - Estimates effort + dependencies.
   Returns <<<HERALD-REPLY>>> JSON.
                                        │
                                        ▼

5. pherald reads outcome:
   • validated → allocate HRD-NNN final bug item, link parent_investigatio
                 close investigation as "validated", emit task.opened even
                 attach Claude's investigation summary as multi-format bun
   • needs_more_info → keep investigation open, ask subscriber follow-ups
   • rejected (duplicate/out-of-scope/known) → close investigation with re
                                                no final item, Reply C exp
   • cannot reproduce → keep investigation open with "operator-blocked"
                        status per Universal §11.4.21, page operator-on-ca
                                        │
                                        ▼  Reply C (final)
                                           "✅ Created HRD-042 (bug, criticality=m
                                           "   Investigation: HRD-INV-007 (validat
```

```
"    Affected: pherald/internal/telemetr
"    Repo: github.com/<org>/<project>/bl
[+ multi-format attachment bundle per §
```

Persistence: every investigation has its own file under `docs/Investigations/<HRD-INV-NNN>.md` carrying the Claude-Code dispatch transcript + classification metadata + the validation decision. Composes with Universal §11.4.55 (Reopens-history) for the case where a previously-rejected investigation is reopened.

18.2.2 Criticality determination

Criticality is set in `inbound_messages.classification` by the §32.6 classifier and stored on the investigation + final workable items. Levels match Universal item-type vocabulary:

Level	Triggers	SLA replies	SLA investigation
`critical`	explicit `critical:` prefix, OR LLM keyword-confidence ≥ 0.85 on outage/data-loss/security-breach terms, OR sender has `oncall` role and message arrives during the configured incident window	Reply A ≤ 30 s, Reply B ≤ 2 min, Reply C ≤ 5 min	< 1 h
`high`	explicit `high:`, OR LLM 0.7–0.85	A ≤ 1 min, B ≤ 5 min, C ≤ 30 min	< 4 h
`middle`	default for `bug`/`issue`/`task`	A ≤ 5 s, B ≤ 5 min, C ≤ 2 h	< 24 h
`low`	explicit `low:` or `nice-to-have:`	A ≤ 5 s, B ≤ 30 min, C ≤ 24 h	< 1 week

Operators MAY override mid-flight: `Override: HRD-042 criticality=critical` re-dispatches to LLM with the new SLA and re-pages affected `oncall` tags.

18.2.3 Type classification (Universal §11.4.16 mapping)

Inbound type tokens map to Universal §11.4.16 item types one-to-one. The classifier emits the exact Universal vocabulary so downstream `Issues.md` rows pass §11.4.16-PARITY gate without translation. Full mapping in §32.6.

When an inbound is ambiguous (e.g., `Hey, the build keeps failing` — no prefix), the classifier:

1. Runs the deterministic keyword pass first (verbs like *fails*, *broken*, *crashed* → tentative `bug`).
2. If confidence < 0.7, escalates to LLM for type-only dispatch (§33.3 prompt envelope, `task=classify-only`).
3. If LLM confidence still < 0.7, replies ❓ `Could you tag this as Bug: / Issue: / Task: / Query: /` …? and stages the message as `needs-classification` (NOT processed further).

18.2.4 Attachment validation + storage

Subscriber-supplied attachments referenced by an inbound investigation/bug/task MUST follow this storage convention (mandatory, per the user's V3 requirement):

```
<consuming-project-root>/
└── issues/
    └── users/
        └── attachments/
            └── HRD-042/                          ← WORKABLE_ITEM_ID directory
                ├── 01_stack-trace.log           ← original filename, prefixed
                ├── 02_screenshot.png
                └── 03_repro-script.sh
```

Where `HRD-042` is the canonical workable-item ID. For investigations the directory is named after the investigation ID (`HRD-INV-007/`); on transition to a final workable item, files are moved to the final ID's directory and `HRD-INV-007/` becomes an empty placeholder kept for audit (Universal §11.4.55).

The investigation/bug `Issues.md` row carries a `Attachments:` column referencing the directory. Multi-format attachment generation (§36) does NOT apply to inbound user attachments — they are stored as-is in their original format. (§36 multi-format is for OUTBOUND attachments Herald itself produces.)

Pre-storage validation pipeline (per §32.4):

1. Each attachment downloaded into a sandbox directory (configurable; default `/tmp/herald-staging/<random>/`).
2. ClamAV scan (if configured) — quarantine on hit.
3. Magic-byte + MIME match — reject mismatched.
4. Extension allowlist — reject `.exe`, `.dll`, `.so`, `.dylib`, `.bat`, `.cmd`, `.ps1`, `.scr`, `.jar`, `.class` by default.
5. Per-file size cap (`[attachments].in_max_mib`, default 25 MiB).
6. Per-message total cap (`[attachments].in_total_max_mib`, default 100 MiB).
7. If all pass → move to `issues/users/attachments/<WORKABLE_ITEM_ID>/`.
8. If any fail → record reason in `quarantined_messages`, do NOT move to canonical path, Reply C cites the validation error.

The `attachments_index.md` file inside each WORKABLE_ITEM_ID directory documents:

```markdown
# Attachments for HRD-042

| Order | Filename | MIME | Size (B) | SHA-256 | Uploaded by | Uploaded at | Notes |
|---|---|---|---|---|---|---|---|
| 01 | stack-trace.log | text/plain | 12_348 | <hex> | alice@tgram:42 | 2026-05-20T18:30:12Z | repro context |
| 02 | screenshot.png | image/png | 348_551 | <hex> | alice@tgram:42 | 2026-05-20T18:30:14Z | UI state |
| 03 | repro-script.sh | text/x-shellscript | 1_204 | <hex> | alice@tgram:42 | 2026-05-20T18:30:17Z | runs in 10 s |
```

The index file IS the source of truth for "what's attached to this workable item" — Herald regenerates it on every attachment add/remove (per §11.4.61 freshness rules; if the index drifts from the directory contents, the §32 validator FAILs with `attachment_index_drift`).

18.2.5 Claude Code project-session integration

Per §33.2 the Claude Code session for the consuming project is anchored at `<consuming-project>/.herald/claude-code/sessions/<project_name>.session`. For Project Herald specifically:

- The `[herald].project_name` config value (e.g. `ATMOSphere`) drives the anchor name.
- Each invocation runs `claude --resume <UUID> --print "<§33.3 envelope>"` with `cwd=<consuming-project-root>`.
- The session anchor is read-only to Herald; manual deletion resets the session (operators MAY want this when Claude's context degrades — e.g. after a long-running incident produces a noisy session).
- The flavor binary refuses to start if `[herald].project_name` is empty (Claude Code dispatch is non-optional in V3 r1).

18.2.6 Channel interactions

In addition to the §18.1.1 cross-flavor primitives, `pherald` ships these flavor-specific interactive surfaces:

Surface	Channels	What it does
`/investigate <HRD-NNN>` slash command	Slack, Discord, Telegram	Re-runs the investigation phase for an existing item (useful after the subscriber adds more attachments to the thread).
`/promote <HRD-INV-NNN>` slash command	Slack, Discord	Operator-only — force-promotes an investigation to a final workable item without waiting for LLM validation.
`/reject <HRD-INV-NNN> <reason>` slash command	Slack, Discord	Operator-only — closes an investigation with a documented reason.
`Investigation summary modal`	Slack `views.open`, Discord modal	When a subscriber clicks "Add reproduction steps" on Reply B, opens a structured modal asking for: repro steps, expected vs actual, environment, attachments.
Investigation status buttons	Slack Block Kit `actions`, Telegram inline keyboard, Discord buttons	On Reply C of an investigation: `[Validate] [Reject] [Need more info] [Reassign]`.
Workable-item card	All channels with rich messaging	A pinned card with workable item summary + criticality badge + assignee + status — refreshed via §35 versioned-report mechanic on every status change.

18.3 System Herald (`sherald`)

Focus: OS/host/service health and security. Designed for systems-administration teams + on-call rotation.

Event subtree: `digital.vasic.herald.system.*`.

Sources:

- `journalctl` watcher (configurable `_SYSTEMD_UNIT` filters).
- `auditd` events.
- Prometheus Alertmanager webhook receiver.
- Loki/Grafana log alerts.
- File-integrity-monitoring (AIDE, Tripwire) hooks.
- SNMP traps (via a small SNMP-to-CloudEvent adapter).
- Kernel events (oom-kill, panics) — via `journald` filter.

Event types:

- `digital.vasic.herald.system.host.cpu_high` (threshold-crossing).
- `digital.vasic.herald.system.host.disk_full` (filesystem).
- `digital.vasic.herald.system.host.memory_pressure`.
- `digital.vasic.herald.system.service.restarted`.
- `digital.vasic.herald.system.service.crashed`.
- `digital.vasic.herald.system.service.flapping` (≥3 restarts in 5min).
- `digital.vasic.herald.system.security.login_anomaly` (failed SSH burst, unusual IP).
- `digital.vasic.herald.system.security.privilege_escalation`.
- `digital.vasic.herald.system.cert.expiring` (X.509 certificate ≤30d).
- `digital.vasic.herald.system.backup.missed`.

Subscriber commands:

- `Ack:` — acknowledge an alert (silences future fires of the same fingerprint for a window).
- `Silence: <fingerprint> for <duration>` — manual silencing.
- `Resolve: <fingerprint>` — manual resolution.
- `Status:` — current open-alerts snapshot.
- `Runbook: <fingerprint>` — link to the runbook for this alert type.

Integrations: `sherald` is often paired with `aherald` and `iherald` — the three flavors share the `commons_alert` package.

18.3.1 Channel interactions

Surface	Channels	What it does
Quick-action buttons on every alert	Slack, Discord, Telegram, Teams	`[Ack] [Silence 1h] [Silence 24h] [Resolve] [Runbook] [Escalate]`
🔇 reaction	Slack, Discord, Telegram, Lark	One-tap silence for the alert's fingerprint for the default window (1 h) — most-common-action wins the easiest UX.
`/silence-similar` slash command	Slack, Discord	Silences not just the current fingerprint but every alert sharing the same `service` label + severity.

Surface	Channels	What it does
`/runbook <fingerprint>` slash command	Slack, Discord, Telegram	Posts the runbook URL inline (saves operators from copy-pasting).
Service-health card	All rich channels	Per-service status card with last-N alerts, MTTR, current open count — refreshed via §35 versioned-report mechanic.
Slack `views.open` modal	Slack	Triggered by `[Silence custom…]` button — modal asks for duration + reason + scope (just fingerprint vs whole service).
Forum-topic per service	Telegram (forum group)	Each service gets its own forum topic; all alerts for that service land there → less noise in the main channel.

18.4 Build Herald (`bherald`)

Focus: CI/CD build lifecycle events. Designed for development teams + release engineers.

Event subtree: `digital.vasic.herald.ci.*`.

Sources:

- GitHub Actions workflow webhook (`workflow_run`, `check_suite`).
- GitLab CI webhook (`Pipeline Hook`).
- Jenkins post-build steps invoking `bherald send`.
- CircleCI / Drone / Buildkite via their respective webhooks.
- Tekton / Argo Workflows via CloudEvents emitters (native).

Event types:

- `digital.vasic.herald.ci.build.queued | started | succeeded | failed | cancelled`.
- `digital.vasic.herald.ci.test.passed | failed | flaky | skipped`.
- `digital.vasic.herald.ci.security_scan.finding` (Semgrep, Snyk, Trivy, gitleaks).
- `digital.vasic.herald.ci.dependency.outdated | vulnerable`.
- `digital.vasic.herald.ci.coverage.dropped`.
- `digital.vasic.herald.ci.lint.failed`.

Routing logic:

- Failed builds with @-mentions of the author (via `subscriber_aliases` mapping git-author-email → subscriber).
- Flaky tests batched into a daily digest (per §7.3 throttling).
- Security findings of CRITICAL severity bypass quiet hours.

Subscriber commands:

- `Retry:` — re-trigger the failed build (if integration grants permission).
- `Snooze: <duration>` — silence a flaky test.
- `Triage: <finding>` — mark a security finding for review.

18.4.1 Channel interactions

Surface	Channels	What it does
[Retry] [Open logs] [Blame] buttons	Slack, Discord, Telegram, Teams	Posted alongside every failed-build message; one-click access to the most common follow-ups.
[Snooze flaky 1d] [Snooze flaky forever] buttons	Slack, Discord	Posted on flaky-test events to throttle noise without needing prefix commands.
`/triage {accepted	wont-fix	false-positive}` slash command
🐛 reaction	Slack, Discord, Telegram	Promotes a failed-build event into a pherald bug investigation (HRD-INV-NNN) — one-emoji cross-flavor handoff.
Coverage delta card	Slack, Discord, Teams	Posted on every PR's CI completion; shows new-vs-base coverage + which files dropped + jump-to-blame.
Build digest forum topic	Telegram (forum group)	All build events for the same branch land in one topic; auto-archives on branch deletion.

18.5 Deploy Herald (dherald)

Focus: Release / deployment / rollout events.

Event subtree: digital.vasic.herald.deploy.*.

Sources:

- ArgoCD / Flux webhook (application synced, health degraded).
- Spinnaker pipeline notifications.
- Kubernetes Event watcher (Deployment rolled out, ReplicaSet failed).
- Custom deploy scripts invoking dherald send.

Event types:

- digital.vasic.herald.deploy.started | succeeded | failed | rolled_back.
- digital.vasic.herald.deploy.canary.promoted | canary.aborted.
- digital.vasic.herald.deploy.feature_flag.toggled.
- digital.vasic.herald.deploy.config.drift_detected.

Routing:

- Production deploys notify #prod-deploys + an Email to release-eng.
- Failed deploys with rollback option button (Slack Block Kit Action).

Subscriber commands:

- Rollback: <deploy_id> — initiate rollback (gated by elevated privilege).
- Hold: <env> — freeze further deploys to env.
- Status: <env> — current state of env.

Composes with rherald for the full release pipeline.

18.5.1 Channel interactions

Surface	Channels	What it does
[Rollback] [Promote canary] [Hold env] [Open dashboard] buttons	Slack, Discord, Telegram, Teams	Posted on every deploy event with elevated-role check before execution. Confirmation modal required for Rollback and Hold env (destructive).
🚨 reaction on a deploy event	Slack, Discord	Pages the on-call engineer + freezes further deploys to that env (one-emoji shortcut for "rollback in progress").
/promote <deploy_id> slash command	Slack, Discord	Promotes a canary to full rollout (operator role required).
/hold <env> <duration> [reason] slash command	Slack, Discord	Freezes deploys to an env; bidirectional sync to deploy tool.
Environment status card	All rich channels	One card per env (dev / staging / canary / prod) with current version + last deploy + health-check status — refreshed via §35 versioned-report mechanic on every event.
Slack views.open modal	Slack	[Rollback] button opens modal asking for: target version, scope (full vs canary), reason, on-call notify list.

18.6 Alert Herald (aherald)

Focus: Monitoring-alert routing. Sits between alert producers (Alertmanager, Datadog, Grafana) and human/on-call channels (PagerDuty, OpsGenie, Slack). Provides smarter routing than Alertmanager's native receivers.

Event subtree: digital.vasic.herald.alert.*.

Sources:

- Prometheus Alertmanager webhook.
- Grafana alert webhook.
- Datadog webhook.
- OpenTelemetry-native alert sources.
- Generic CloudEvents alert producers.

Routing features:

- **De-duplication**: same alert fingerprint within window → single notification.
- **Grouping**: related alerts (same service label + severity) collapse into a digest.
- **Inhibition**: parent alert silences child alerts (per Alertmanager's inhibit_rules).
- **Escalation**: integrates with Temporal workflow for time-based escalation chains.
- **Quiet hours** override only for severity=critical + category=incidents.

Subscriber commands:

- Ack:, Silence:, Resolve: (same as sherald).
- Escalate: — bump severity, route to on-call.

18.6.1 Channel interactions

Surface	Channels	What it does
`[Ack]` `[Silence 1h]` `[Silence 4h]` `[Resolve]` `[Escalate]` buttons	Slack, Discord, Telegram, Teams	Posted on every alert; one-click. `[Ack]` carries an implicit time-window (acknowledgement claim lasts for the alert's `escalation_window`, default 15 min — if not resolved by then, alert re-fires and escalation re-engages).
👍/🔕/🚨 reactions	Slack, Discord, Telegram, Lark	Cross-flavor primitives (per §18.1.1) wired to ack/silence/escalate.
`/aherald group <service>` slash command	Slack, Discord	Shows the current grouping for a service: which alerts are collapsed into which digest.
`/aherald inhibit <parent> <child>` slash command	Slack, Discord	Operator-only — creates a runtime inhibition rule without editing Alertmanager config (synced back via API).
Alert digest card	All rich channels	Refreshed via §35 versioned-report mechanic — shows currently-firing group with member-alert chips; each chip is clickable for drill-down.
Telegram inline-keyboard "expand" button	Telegram	Collapses long alert descriptions; one-tap to expand. Avoids hitting Telegram's message-length limit on big alert payloads.
Email reply-by-keyword	Email	Subscribers may reply `ack`, `silence 4h`, `resolve`, `escalate` in the email body; Herald parses keywords + applies the action even without buttons (email is button-less).

18.7 Schedule Herald (`scherald`)

Focus: Scheduled jobs + recurring notifications (cron-like).

Event subtree: `digital.vasic.herald.schedule.*`.

Sources:

- River queue periodic jobs.
- `scherald serve` runs an internal cron-style scheduler.
- External cron jobs invoking `scherald send`.

Event types:

- `digital.vasic.herald.schedule.job.started | succeeded | failed | skipped`.
- `digital.vasic.herald.schedule.digest.daily | weekly | monthly`.
- `digital.vasic.herald.schedule.reminder.due` (scheduled human reminders).

Built-in digest builders:

- Daily ops digest (yesterday's alerts + deploys + failed builds).
- Weekly project digest (PRs merged, issues opened/closed, contributors).
- Monthly compliance digest (per `cherald` if installed).

Subscriber commands:

- `Snooze: <reminder_id> for <duration>`.
- `Cancel: <reminder_id>`.
- `Remind me: <prose>` — parse and create a one-shot reminder.

18.7.1 Channel interactions

Surface	Channels	What it does
`[Snooze 1h] [Snooze 1d] [Snooze custom] [Cancel]` buttons	Slack, Discord, Telegram, Teams	One-click on every reminder. `[Snooze custom]` opens a modal (Slack/Discord) or DM-prompt fallback for duration entry.
`/remind <subject> in <duration>` slash command	Slack, Discord, Telegram	Native cross-platform reminder creation; parses `<duration>` permissively (`5m`, `2 hours`, `tomorrow 9am`, `next Mon 14:00 UTC`).
`/digest <daily	weekly	monthly>` slash command
🔁 reaction on a reminder	Slack, Discord, Telegram	Re-schedules for the same delta from now (e.g., subscriber gets reminder at 10:00, clicks 🔁 at 10:30 → next reminder at 11:00).
Digest "[I read this]" reaction button	Slack, Discord, Teams	Tracks readership; in subsequent digests Herald can deprioritise sections fewer subscribers read (closes feedback loop).
Forum-topic per recurring series	Telegram (forum group)	Each weekly project digest gets its own forum topic so readers can subscribe per series, not per channel.

18.8 Incident Herald (`iherald`)

Focus: Incident lifecycle — declare, escalate, resolve, postmortem.

Event subtree: `digital.vasic.herald.incident.*`.

Sources:

- PagerDuty / OpsGenie webhooks (incident open/escalated/resolved).
- Internal `iherald send` from on-call tools.
- Auto-escalation triggered by `aherald` after N min unacknowledged.

Event types:

- `digital.vasic.herald.incident.opened | escalated | resolved | reopened`.
- `digital.vasic.herald.incident.commander.assigned`.
- `digital.vasic.herald.incident.update.posted`.
- `digital.vasic.herald.incident.postmortem.due | published`.

Workflow (Temporal):

- On open: page primary on-call → wait 5 min → if no ack, escalate to secondary → wait 10 min → page manager.
- On resolve: schedule postmortem reminder 24h out.
- On postmortem.published: notify stakeholders.

Subscriber commands:

- `Page: <person>` — manual escalation.
- `IC:` (incident-commander) — assign IC role.
- `Update: <prose>` — post a public update.
- `Resolve:` — close the incident.

18.8.1 Channel interactions

Surface	Channels	What it does
Incident command room	Slack thread / Discord forum-channel post / Telegram forum-topic / Teams channel	Every incident gets a dedicated **command room** — all updates, page-out logs, IC handoffs, action items, and resolution land in one place. Auto-archived 30 days post-resolution.
`[Take IC] [Page sec] [Open runbook] [Update] [Resolve]` buttons	Slack, Discord, Telegram, Teams	Posted on every incident.opened. `[Take IC]` is a single-tap takeover (atomic compare-and-swap on `incidents.commander_id`).
`[Acknowledge page]` button	Slack, Discord, Telegram, Teams, **Email**, **WhatsApp**	Posted on every page-out — the engineer's ack short-circuits the 5/10-min escalation. Email implementation uses a one-click signed URL (Universal §11.4.10 credentials never tracked → URL signed with `[interactions].ack_signing_key`).
`/ic <handle>` slash command	Slack, Discord	Assign IC role to another subscriber; both parties get a DM confirming the handoff.
`/postmortem` slash command	Slack, Discord	Generates a postmortem template (using §36 multi-format export) seeded with the incident timeline pulled from `incident_events` table; opens a Slack modal / Discord thread for in-place authoring.
Status-update reaction kbd	Slack	Authoring an update in the command room uses Slack's native message composer; a `[Post as public status]` button publishes the message to subscribers outside the command room.
Real-time timer	All rich channels	Refreshed via §35 versioned-report mechanic every 60 s — shows time-since-open, time-since-last-update, time-to-postmortem-due.

18.9 Release Herald (`rherald`)

Focus: Release lifecycle — tags, changelogs, dependency notifications.

Event subtree: `digital.vasic.herald.release.*`.

Sources:

- Git tag webhooks.
- GoReleaser / Release-Please / semantic-release pipeline hooks.
- Renovate / Dependabot PR webhooks (`dependency.update.available`).
- OCI registry push events (`oci.image.pushed`).

Event types:

- `digital.vasic.herald.release.tagged | published`.
- `digital.vasic.herald.release.changelog.generated`.
- `digital.vasic.herald.release.dependency.update.available | major.update.available`.
- `digital.vasic.herald.release.sbom.generated | provenance.signed`.
- `digital.vasic.herald.release.security_advisory.published` (CVE for a project dep).

Composes with `dherald` — release publish event triggers deploy pipeline notification chain.

Subscriber commands:

- `Promote: <version> to <env>`.
- `Approve: <dep_update>` — approve a Renovate PR via authorised channel.

18.9.1 Channel interactions

Surface	Channels	What it does
Release card	All rich channels	One pinned card per release tag with version, SBOM checksum, signed-provenance link, changelog excerpt, breaking-changes flag. Refreshed via §35 versioned-report mechanic on every related event.
`[Promote to staging]` `[Promote to prod]` `[Hold]` `[Open changelog]` buttons	Slack, Discord, Telegram, Teams	One-click promotion path; `[Promote to prod]` requires a Slack modal confirmation with `release_note_signoff` field (recorded in audit log per §11.4.10).
`[Approve]` `[Reject]` `[Defer 7d]` buttons	Slack, Discord, Telegram	Posted on every dependency-update event (Renovate / Dependabot PR). Approval routes back to the source via PR-comment API.
`/cve <CVE-id>` slash command	Slack, Discord	Returns CVSS score + affected versions in the project + remediation guidance.
`/release-notes` slash command	Slack, Discord, Telegram	Generates a §36 multi-format release notes bundle from `docs/changelogs/` since last tag.
Compliance badge reactions	Slack, Discord	Operator reactions ✅ / ❌ on SBOM/SLSA-provenance events drive `cherald` triage queue.
Breaking-changes thread	Slack thread / Telegram forum-topic	Each release with `breaking_changes=true` opens a thread for migration-question Q&A; auto-archive 30 days post-release.

18.10 Compliance Herald (`cherald`)

Focus: Audit + compliance + governance events. For regulated environments (SOC2, HIPAA, PCI-DSS, GDPR).

Event subtree: `digital.vasic.herald.compliance.*`.

Sources:

- Cloud audit logs (AWS CloudTrail, GCP Cloud Audit Logs, Azure Activity Log) via OpenTelemetry / native exporters.
- IAM change webhooks (Okta, Auth0, AzureAD).
- Vault / KMS access logs.
- Database audit log streamers.
- GitGuardian / TruffleHog secret-scan webhooks.

Event types:

- `digital.vasic.herald.compliance.audit.access` (privileged action).
- `digital.vasic.herald.compliance.policy.violation` (e.g. unencrypted bucket, S3 public ACL).
- `digital.vasic.herald.compliance.cert.expiring | expired`.
- `digital.vasic.herald.compliance.license.expiring | non_compliant`.
- `digital.vasic.herald.compliance.access.review.due` (quarterly access reviews).
- `digital.vasic.herald.compliance.secret.exposed`.

Routing:

- All events archived to immutable storage (`docs/herald/audit/` with WORM bit if filesystem supports).
- High-severity events routed to security + legal channels.
- Quarterly summary digest emailed to compliance officer.

Constitution composition: `cherald` events plug into the parent project's Constitution §11.4.10 (credentials never tracked) and §11.4.18 (audit-log discipline).

18.10.1 Channel interactions

Surface	Channels	What it does
Restricted audience by default	Slack private channel, Discord role-gated channel, Telegram private group, Teams private team	All `cherald` channel addresses default to `restricted=true` in `channel_addresses.metadata`; messages are *not* visible to unprivileged subscribers regardless of category.
`[Acknowledge violation]` `[Mark false-positive]` `[Open`	Slack, Discord, Teams	Posted on every policy-violation event. `[Open ticket]` cross-flavor handoff into `pherald` (creates an investigation in the security workable-items track).

Surface	Channels	What it does
`ticket]` buttons		
`/audit <subscriber-id>` slash command	Slack (operator-only)	Returns the audit trail for the named subscriber — every event they touched in the last N days. Operator-gated; emits a `digital.vasic.herald.compliance.audit.review` event so audits-of-audits are themselves audited.
🔒 reaction	Slack, Discord (operator-only)	Marks an event as `sensitive`; subsequent re-publish suppresses message body in non-restricted channels, leaving only the event ID + "details in restricted channel" pointer.
Compliance digest modal	Slack `views.open`	Quarterly review modal lists overdue access-reviews + expiring certs + license issues in a single composable form for the compliance officer's sign-off.
Email-as-system-of-record	Email	Every `cherald` event is *also* emailed (per the §35 versioned-report mechanic) so the immutable mail archive is the legal-grade backup; subscribers reply with `keep` / `ignore` keywords to drive triage.
Forum-topic per compliance domain	Telegram (forum group)	One topic per domain (SOC2 / HIPAA / PCI-DSS / GDPR / internal-policy) so auditors can scope their review by clicking one topic.

18.11 Future flavors

Identified candidates (NOT in V2 scope; documented to lock-in event-subtree namespaces):

- `fherald` — Finance Herald (billing events, invoice paid/failed, MRR alerts) — `digital.vasic.herald.finance.*`.
- `mherald` — Marketing Herald (campaign sent, A/B test winner) — `digital.vasic.herald.marketing.*`.
- `gherald` — Game/Server Herald (multiplayer match events, anti-cheat reports) — `digital.vasic.herald.game.*`.
- `xherald` — Experiment Herald (feature-flag experiment results) — `digital.vasic.herald.experiment.*`.
- `hherald` — Hardware Herald (IoT device telemetry alerts) — `digital.vasic.herald.hardware.*`.
- `lherald` — Legal Herald (contract renewals, NDA expirations) — `digital.vasic.herald.legal.*`.
- `vherald` — Vendor Herald (third-party service status, SLA breaches) — `digital.vasic.herald.vendor.*`.

---

§19. Diary

`docs/herald/diary/main.md` (+ `main.pdf` + `main.html`) is the **append-only conversation log** of every inbound and outbound message Herald processes.

19.1 Diary path resolution (per R-20)

Resolved relative to the **operator-specified working directory**:

- Standalone Herald: defaults to Herald's own repository root.

- Herald as submodule: defaults to the parent project's root (discovered via the same parent-walk as `find_constitution.sh`).
- Override: `--diary-root <path>` flag, or `[herald].diary_root` in `config.toml`.

19.2 Schema (Markdown)

```markdown
## 2026-05-19T18:30:00Z · pherald · digital.vasic.herald.ci.failed

| Field | Value |
|---|---|
| Tenant | `00000000-…-0001` |
| Event ID | `01931a7c-…-5678` |
| Source | `github.com/vasic-digital/Herald/actions/runs/4231` |
| Channels | tgram (delivered), slack (delivered), email (queued) |

**Body:**

> Build pao u `main`: 3/5 testova nije prošlo. Pogledaj log:
> https://github.com/vasic-digital/Herald/actions/runs/4231

**Subscribers reached:** alice (tgram + slack), bob (email).
```

19.3 Sync strategy (per R-03)

- **Pandoc** (Markdown → HTML5) + **WeasyPrint** (HTML → PDF) via Pandoc's `--pdf-engine=weasyprint` flag.
- **Triggers**:
  - Under `<flavor>herald serve`: **fsnotify-watched debounced builds** (500 ms idle, 2 s ceiling). A single editor save fires 4–12 events; debouncing avoids redundant rebuilds.
  - Under one-shot `<flavor>herald send`: post-write hook in the diary writer.
- **Manifest** at `docs/herald/diary/.sync.json` records `{md_sha256, html_sha256, pdf_sha256, source_md_sha256_at_build, built_at}`.
- **Anti-bluff gate** (`CM-DIARY-SYNC` per Universal §11.4.61):
  1. Read current `main.md` SHA-256.
  2. Assert it equals `source_md_sha256_at_build` in the manifest.
  3. For HTML: re-run Pandoc to a temp file; byte-equality with on-disk HTML.
  4. For PDF: extract text (`pdftotext main.pdf -`) and compare to deterministic extraction of the HTML body (byte-equal PDF comparison fails due to embedded timestamps + non-deterministic font subsetting).

---

§20. Extensibility

Per research Topic 6 (operations): two-tier extensibility model.

20.1 Tier A — in-tree adapters (default for V2)

Every channel adapter lives under `commons_messaging/channels/<name>` and implements the `Channel` interface (§11). Out-of-tree authors fork or vendor.

20.2 Tier B — subprocess + manifest (deferred to V2.1)

Spec'd now, implementation deferred. Herald discovers external channel binaries through a manifest file (`heraldchannel.yaml`):

```yaml
name: my-custom-channel
exec: /usr/local/lib/herald/channels/my-channel
supports:
  - text
  - markdown
  - attachments
env:
  required:
    - MY_CHANNEL_TOKEN
protocol_version: 1
```

Communication: stdin/stdout NDJSON with versioned envelope. No in-process linkage — adapters can be written in any language, sandboxed with OS tooling. Mirrors Apprise / Mattermost / git credential-helper.

Explicitly rejected for V2:

- Go `plugin` stdlib (toolchain-version brittleness, no Windows, breaks `-trimpath`).
- HashiCorp `go-plugin` (RPC surface + per-plugin process mgmt — overhead vs. subprocess+JSON).
- Wazero/WASM (promising; WASI sockets still incomplete; network-bound adapter can't run usefully today). Marked as **V3 roadmap candidate**.

---

§21. Supply chain & release engineering

Per research Topic 5 (operations) + R-18:

21.1 Multi-binary versioning

`go.work` for local dev (git-ignored). Per-flavor `go.mod`. Lockstep release: one git tag → GoReleaser builds all flavors + tags `commons/vX.Y.Z`. Release-Please in `manifest` mode bumps all modules from Conventional Commits.

21.2 Reproducible builds

Mandatory flags: `CGO_ENABLED=0 GOFLAGS=-trimpath`. Pinned `-ldflags "-buildid= -s -w"`. Record `GOTOOLCHAIN`.

21.3 SBOMs

Every binary + every container image generates **CycloneDX JSON + SPDX JSON** via [`syft`](). Attached as release assets and as OCI referrers on the image.

21.4 Signing

Keyless `cosign` (Sigstore OIDC via GitHub Actions) for:

- Binaries: `cosign sign-blob`.
- OCI images: `cosign sign`.

21.5 SLSA provenance (target: Build L3)

GitHub `actions/attest-build-provenance` emits **in-toto** statements signed via Sigstore. Move the build into a **reusable workflow** shared across mirrors to reach **SLSA Build L3**.

21.6 Distribution channels

Channel	Tool	Path
Homebrew tap	GoReleaser	`vasic-digital/homebrew-herald`
Scoop bucket	GoReleaser	`vasic-digital/scoop-herald`
`.deb` / `.rpm`	nFPM	`vasic-digital/herald-packages` apt+yum repos
Container images	GoReleaser → Docker	GHCR ghcr.io/vasic-digital/<flavor>herald
Source release	GitHub Releases	Multi-mirror per §103

Verification UX: `scripts/verify-release.sh` runs `cosign verify` + `cosign verify-attestation` against the published cert identity (`https://github.com/vasic-digital/Herald/.github/workflows/release.yml@refs/tags/*`).

---

§22. Constitution integration

Once Herald is fully implemented and verified, **promote candidate universal rules** into the root Constitution, `AGENTS.md`, and `CLAUDE.md` **via a HelixConstitution PR** (audited per Universal §11.4 + §11.4.17 — universal-vs-project classification), never by editing the parent `constitution` Submodule from inside Herald (per R-09; see `CONSTITUTION_INHERITANCE.md` §"Promoting Herald rules into the constitution"). Each Flavor presents its Constitution extensions through the same promotion process.

> **Note:** Herald's implementation MUST BE in direct connection with the `constitution` Submodule — discovery via `find_constitution.sh` parent-walk per §103 / §105 of Herald Constitution.

See [`../../guides/HERALD_CONSTITUTION.md`]() and [`../../guides/CONSTITUTION_INHERITANCE.md`]().

22.1 How `constitution` rules are extended via `constitutable/`

A parent project may drop `Constitution.md` / `CLAUDE.md` / `AGENTS.md` (in `constitutable/`, `constitutable/<flavor>/`, `constitutable/<flavor>/<variant>/`, etc.) to layer extensions or overrides on top of the discovered `constitution/` Submodule.

Load priority (per `constitutable/` mechanism):

`constitution` Submodule → `constitutable/**` extensions/overrides for Constitution / `CLAUDE.md` / `AGENTS.md` → Project + Submodule definition files.

Each `constitutable/<path>` MUST contain at least one of: `Constitution.md`, `CLAUDE.md`, or `AGENTS.md`.

> **Note:** Tests in the `constitution` Submodule MUST be properly extended and updated as `constitutable/` content changes.
>
> **Note:** Herald is **primarily** consumed as a Submodule of another Project; in that case access to the `constitution` Submodule is through the root of that project (cloned under `project_root/constitution` or a configured alternative). For **standalone development** of Herald, the `constitution` is cloned **alongside** Herald (sibling-clone) — current development setup uses this layout. See [`../../guides/CONSTITUTION_INHERITANCE.md` §"Standalone development"]() and R-10.
>
> **Note:** Carefully investigate the codebase of the `constitution` Submodule before any changes are applied. Comprehensive analysis precedes any extension or promotion.

---

§23. Specification documents (change rule)

We MUST keep the following rule / mandatory constraint in `Constitution.md` (parent), `AGENTS.md`, `CLAUDE.md`, and `HERALD_CONSTITUTION.md` §106:

> **IMPORTANT:** Whenever this document (`docs/specs/mvp/specification.V4.md`) or any file under `docs/specs/` (any depth) is modified, **comprehensive planning and implementation of all changes is MANDATORY**. This rule does NOT apply to creating or renaming files; for those, explicitly tell the worker (CLI agent) what to do with the new path. Treat every spec edit as a project-wide ripple, not a doc tweak.

The rule's enforcement anchor is the phrase `comprehensive planning and implementation` — inheritance-gate invariants **I7a–c** assert its presence in `CLAUDE.md`, `AGENTS.md`, and `HERALD_CONSTITUTION.md`. Path references in those propagated files MUST point at the currently-active spec file (V4 as of 2026-05-31; bump in lockstep when V5 supersedes).

---

§24. Documentation

README.md MUST be fully updated with all relevant project details. All user guides and manuals are properly linked from README.md, including (when present):

- docs/flavors/<flavor>/ — per-flavor user manual.
- docs/channels/<channel>/ — per-channel setup guide (credentials, webhooks, signing).
- docs/operations/ — deployment + doctor + backups + upgrades.
- docs/security/ — credential handling, signing keys, secret rotation.
- docs/api/ — machine-readable API specifications (see §24.1).
- docs/migration/ — operator guides for migrating from other notification stacks (Apprise, Gotify, Mattermost-bridge, etc.).

We MUST have **mandatory documentation up to the smallest details**: full user guides, manuals, diagrams, schemes in all major formats (Markdown source + PDF + HTML siblings per §11.4.61 + §11.4.65) and other relevant materials.

24.1 Machine-readable API specifications

Herald ships its own API contracts as machine-readable schemas so client tooling (SDK generators, mock servers, schema-validating proxies, API gateways) doesn't have to reverse-engineer them.

Spec	Location	Format	Generated by
HTTP ingress (/v1/events, /v1/send, /v1/subscribers, /v1/deadletters, /v1/channels)	docs/api/openapi.v1.yaml	OpenAPI 3.1	hand-authored from the canonical Go handler signatures in commons_http/*; CI gate CM-OPENAPI-DRIFT (planned) verifies handler?spec parity.
Webhook ingest (signed, per-source)	docs/api/openapi.v1.yaml (sub-tree under /webhooks/)	OpenAPI 3.1	same source as above.
Event taxonomy (digital.vasic.herald.* event types per §4.2)	docs/api/asyncapi.v1.yaml	AsyncAPI 2.6	machine-generated from commons/events.go event-type registry.
Channel adapter Go interface (§11.0)	commons/types.go	Go source	the source IS the spec; pkg.go.dev rendering is the human view.
Database schema	commons_storage/migrations/*.sql	PostgreSQL DDL	the migration files ARE the schema spec; <flavor>herald schema dump exports current state.

CLI helpers:

- `<flavor>herald openapi` — print the embedded OpenAPI spec (so operators don't need the repo).
- `<flavor>herald asyncapi` — same for AsyncAPI.
- `<flavor>herald schema dump [--format=sql|markdown]` — emit current DB schema.

Documentation cross-link rule (§11.4.59 + §11.4.61 composed): every change to a `commons_http/` handler MUST update `docs/api/openapi.v1.yaml`, MUST add an entry to `docs/changelogs/`, MUST regenerate `docs/api/openapi.v1.html` (rendered via Redoc or stoplight-elements at build time). Drift is a release blocker.

---

§25. Testing

Whole project and all derivatives MUST follow testing rules from the root Constitution (`Constitution.md`), `CLAUDE.md`, `AGENTS.md` in the `constitution` Submodule. Specifically:

- **No bluffing** — every PASS carries positive evidence (§11.4).
- **Paired mutation gates** — every new gate has a paired mutation proving it catches regressions (§1.1).
- **Captured evidence** for test logs (§11.4.2).
- **Test pyramid**: unit tests in each module, integration tests against ephemeral Postgres + Redis (testcontainers-go), end-to-end tests with real-channel sandboxes.
- **No fakes beyond unit tests** (Universal §11.4.27) — integration + end-to-end tests use real Postgres, real Redis, real channel-sandbox or test-bot endpoints.

CI runs the inheritance gate (`tests/test_constitution_inheritance.sh`) as a precondition to any other test.

---

§26. Operations

26.1 Deployment

Reference deployment topologies:

- **Single-host Docker Compose** — Herald flavor + Postgres + Redis on one host. Suitable for small teams.
- **Kubernetes Deployment** — Herald flavor as a Deployment, Postgres + Redis as separate StatefulSets (or external managed). HPA on `messaging.client.operation.duration` p95.
- **Serverless / Lambda** — `<flavor>herald send` invoked as a Lambda function (one-shot mode); `serve` mode NOT supported in Lambda due to long-running needs.

26.2 Backups

- Postgres: daily logical backup (`pg_dump`) + WAL archiving for PITR.
- Redis: AOF rewrite + RDB snapshots.
- Diary: standard git history is the backup (everything is in `docs/herald/diary/main.md`).

26.3 Upgrades

- Lockstep across flavors via §21.1 release model.
- Database migrations are **forward-compatible** for 2 minor versions (so a rolling restart works during a deploy).
- `<flavor>herald migrate` is idempotent and safe to run multiple times.

26.4 Operator runbooks

`docs/operations/runbooks/` contains one runbook per common operational scenario:

- "Postgres connection storm" → check `herald_queue_depth` + `pg_stat_activity`.
- "Telegram bot suspended" → rotate token, update `.env`, run `<flavor>herald doctor`.
- "Dead-letter spike" → query `dead_letters`, identify pattern, replay or purge.
- "DKIM verification failing" → DNS check via `<flavor>herald doctor email`.

26.5 Operator quickstart (5-minute Docker Compose)

Goal: a working `pherald` ingesting one webhook and fanning out to Telegram + Email in five minutes on a fresh laptop. This is the canonical "hello world" deployment.

```yaml
# docker-compose.quickstart.yml — referenced from quickstart/
version: "3.8"
services:
  postgres:
    image: postgres:16-alpine
    container_name: herald-postgres
    environment:
      POSTGRES_USER: herald
      POSTGRES_PASSWORD: ${HERALD_DB_PASSWORD}
      POSTGRES_DB: herald
    ports:
      - "24100:5432"
    volumes:
      - herald-pg:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U herald"]
      interval: 5s
  redis:
    image: redis:7-alpine
    container_name: herald-redis
    command: ["redis-server", "--requirepass", "$
        {HERALD_REDIS_PASSWORD}"]
    ports:
      - "24200:6379"
    volumes:
      - herald-redis:/data
  otel-collector:
```

```yaml
    image: otel/opentelemetry-collector-contrib:latest
    container_name: herald-otel
    command: ["--config=/etc/otel-config.yaml"]
    volumes:
      - ./otel-config.yaml:/etc/otel-config.yaml:ro
    ports:
      - "24417:4317"
  pherald:
    image: ghcr.io/vasic-digital/pherald:latest
    container_name: herald-pherald
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_started }
    environment:
      HERALD_POSTGRES_DSN: "postgres://herald:$
        {HERALD_DB_PASSWORD}@postgres:5432/herald?
        sslmode=disable"
      HERALD_REDIS_ADDR:    "redis:6379"
      HERALD_REDIS_USER:    "default"
      HERALD_REDIS_PASSWORD: "${HERALD_REDIS_PASSWORD}"
      OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
      OTEL_RESOURCE_ATTRIBUTES:
        "deployment.environment=quickstart"
      TELEGRAM_BOT_TOKEN:   "${TELEGRAM_BOT_TOKEN}"
      TELEGRAM_CHAT_ID:     "${TELEGRAM_CHAT_ID}"
    ports:
      - "24091:24091"   # webhook ingress
      - "24090:24090"   # admin (/livez, /readyz, /metrics)
    command: ["serve"]
volumes:
  herald-pg:
  herald-redis:
```

Steps:

```bash
# 1. Clone Herald + containers submodule
git clone git@github.com:vasic-digital/Herald.git && cd Herald
git submodule update --init

# 2. Copy & fill the example .env
cp .env.example .env
$EDITOR .env   # set HERALD_DB_PASSWORD, HERALD_REDIS_PASSWORD,
        TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID

# 3. Boot
docker compose -f quickstart/docker-compose.quickstart.yml up -d

# 4. Wait for ready
```

```
curl --retry 30 --retry-delay 2 --retry-connrefused http://
        localhost:24090/readyz

# 5. Send a test event
curl -X POST http://localhost:24091/v1/events \
  -H "Content-Type: application/cloudevents+json" \
  -d '{
    "specversion":"1.0",
    "id":"01931a7c-3f4e-7000-9abc-def012345678",
    "source":"https://example.com/quickstart",
    "type":"digital.vasic.herald.system.host.cpu_high",
    "subject":"tag:*",
    "data":{"host":"laptop","cpu_pct":97}
  }'

# 6. Verify
docker logs herald-pherald --tail=20
cat docs/herald/diary/main.md | tail -30
```

A successful run delivers a Telegram message to the configured chat within ~3 seconds, appends a diary entry, and emits OTel traces visible at `http://localhost:24417` (Collector logs).

26.6 Disaster recovery

Recovery objectives (V2 baseline; per-tenant overrides allowed):

Class	RPO target	RTO target	Mechanism
Postgres data	5 min	30 min	WAL archiving + PITR (`pgbackrest` recommended)
Redis state	5 min	5 min	AOF rewrite + RDB snapshot every 5 min; warm replica optional
Diary	0 (git)	minutes	`git push` fan-out to 4 mirrors per §103; restore via `git clone <any mirror>`
Credentials	0 (out-of-band)	depends on operator	Vault / 1Password / OS keychain; Herald NEVER stores plaintext credentials
Configuration	0 (git)	minutes	`config.toml` is git-committed (no secrets); `.env` is git-ignored, backed up out-of-band by operator
Workflow state (Temporal opt-in)	5 min	30 min	Temporal's own backup/restore; out of Herald scope

Cold-start recovery procedure:

1. Restore Postgres from latest WAL position.
2. Restore Redis from latest RDB (transient state is fine to lose; rate-limit buckets repopulate).
3. `git clone` Herald + `containers` submodule from any mirror.
4. Re-provision `.env` from out-of-band credential store.
5. `docker compose up -d`.

6. <flavor>herald doctor — confirm all green.
7. <flavor>herald deadletter list — replay any in-flight messages caught at the time of failure.

Tested scenarios (each MUST have an integration test under `tests/dr/`):

- Postgres primary loss → failover to replica.
- Redis loss → cold restart (token buckets reset, no data corruption).
- Single Herald replica crash → other replicas continue (multi-instance deployments).
- Whole-host loss → cold-start from backups.
- All four git mirrors momentarily unreachable → push deferred, queued locally, retried.

---

§27. Roadmap

27.1 V2 → V3 candidates

Candidate	Notes
WASM channel adapters (wazero)	Once WASI sockets stabilise — Topic 6 (operations)
LLM-driven message rendering	Auto-summarize long events to channel-appropriate brevity
Slack workflow builder steps	Beyond V2 advanced tier
WhatsApp Business custom-template approval workflow	Once Cloud API matures
Mini-Apps inside Telegram	Subscriber-side UI
Adaptive Card Designer integration for Teams	UX for non-developer template authors
First-class Matrix protocol channel	If demand materializes
Mattermost adapter	Open-source alternative to Slack
Rocket.Chat adapter	Self-hosted alternative

27.2 Deferred V1 Recommendations

V1 R-NN items that V2 did not yet implement (with intended landing):

R	Status in V2	Landing
R-01	✅ Applied (24XXX ports in §9.4)	—
R-02	✅ Applied (single binary, two modes §3.1)	—
R-03	✅ Specified (§19.3)	First implementation cycle
R-04	✅ Applied (§3.3)	—
R-05	✅ Specified (delivery evidence enum §11.13)	First adapter cycle
R-06	✅ Specified (§11.2 Max behind build tag)	When sanctions advisory drafted
R-07	✅ Applied (§7.1)	—
R-08	✅ Applied (§12 `ConversationRef`)	—
	✅ Applied	—

R	Status in V2	Landing
R-09 / R-10 / R-14 / R-15 / R-19 / R-20 / R-21 / R-22		
R-11 / R-12	✅ Specified (§9.5)	When first SDK / `containers` lands
R-13	Deferred — `constitutable/` loader + I8/I9 gates	Next constitution PR
R-16	✅ Specified (§15)	First implementation cycle
R-17	✅ Specified (§8.2 algorithm)	First `commons_prefix` commit
R-18	✅ Specified (§21.1)	First release pipeline

27.3 Cost considerations

Order-of-magnitude indicative pricing as of 2026-Q2 (operators MUST re-verify at deployment time; tiers change frequently). Herald itself is open-source / no licence fee — these costs are external dependencies an operator pays for in production.

Infrastructure (per single-region deployment):

Component	Self-host minimum	Managed (illustrative)
Postgres 16 (2 vCPU / 4 GiB / 20 GiB)	~$15/mo (Hetzner CX21, DO Basic)	RDS db.t4g.small ~$35/mo
Redis 7 (1 GiB)	~$10/mo (same node as Postgres for small tenants)	ElastiCache cache.t4g.micro ~$15/mo
Herald binary host (1 vCPU / 1 GiB)	~$5/mo	ECS Fargate 0.25 vCPU ~$10/mo
OTel Collector	sidecar / shared	Grafana Cloud free tier or self-host
Single-tenant baseline	**~$30/mo**	**~$60–80/mo**

Transactional email provider (ESP) — pick one based on volume + deliverability needs:

Provider	Free tier	Paid tier entry	Bounce/complaint webhook	Notes
[Resend]()	3 000 emails/mo, 100/day	$20/mo for 50k	✓ (Event Webhook)	Best DX of the three; Idempotency-Key header support.
[Postmark]()	none — paid only	$15/mo for 10k	✓ (Bounce / Spam / Open / Click streams)	Best deliverability reputation for transactional traffic; HTTP 406 for blocked recipients.
[SendGrid]()	100 emails/ day free forever	$19.95/mo for 50k	✓ (Event Webhook)	Widest ecosystem; complex pricing tiers.
Self-host SMTP	$0	infra + IP-reputation labour	DSN parsing only	Only if operator has DNS / reverse-DNS / warm IPs already; otherwise deliverability tanks.

Messaging platform fees:

Channel	Cost model
Telegram Bot API	Free, no fee per message.
Slack	Free for public/private channel webhooks; paid plans only required for full Slack workspace, not for the integration.
Discord	Free webhooks; free bot.
Microsoft Teams	Free incoming webhooks; Bot Framework requires Azure Bot Service ~$0.50/1 k messages.
Lark / Feishu	Free for chatbots within an org.
WhatsApp Business Cloud API (Meta direct)	Per-conversation pricing, ~0.005–0.15 depending on country + category.
WhatsApp Business via Twilio	Twilio markup on top of Meta price.
Viber	Free for service messages within 30 days of user interaction; promotional pricing varies.
Max	Free Bot API at the time of writing (verify at deploy).
ntfy / Gotify	Self-hosted = $0; ntfy.sh free public instance.

Supply-chain tooling:

Tool	Cost
GitHub Actions (public repo)	Free unlimited minutes.
GitHub Actions (private repo)	2 000 free minutes/mo on Linux; $0.008/min after.
Sigstore / cosign	Free (keyless via OIDC).
syft (SBOM)	Free / OSS.
GoReleaser OSS	Free.
GoReleaser Pro	$19/mo if Herald needs the per-subproject `tag_prefix` feature (R-18 alternative — V2 default does not require Pro).

OpenTelemetry backend:

Backend	Free tier
Grafana Cloud	10 k series metrics, 50 GiB logs, 50 GB traces /mo
Honeycomb	20 M events/mo
Datadog	5 hosts free for 14 days
Self-host Prometheus + Tempo + Loki	$0 + operator time

Total order-of-magnitude TCO for a small team running `pherald` + `sherald` self-hosted on Hetzner with Resend free tier: **~$30–60/mo** of recurring infrastructure. Operator labour for upgrades, monitoring, and incident response is the larger real cost — design choices in §17 (observability) and §26 (operations) exist specifically to minimise that labour.

---

§28. Notes & open questions

- The list of integrated messengers in §11 is the V2 commitment; additional channels may be added in later iterations.

- Whether to ship a **first-party web UI** for subscriber preference management is an open question — current direction is "no UI in V2; expose REST API only; let third parties build UIs".
- Whether `aherald` should incorporate **machine-learning-based alert grouping** (similar to BigPanda, Moogsoft) is deferred to V3.
- Whether to support **end-to-end encryption** of messages (Signal Protocol via olm/megolm) is deferred — only Matrix bridges currently demand this.
- The **rate-limit cap** for AI-CLI-agent subscribers needs operator-level tuning per deployment to avoid runaway invocation; default suggested 60 messages/minute per agent token.

---

§29. Changelog

29.1 V1 → V2 changes (2026-05-19)

V1 → V2 changes (2026-05-19):

- **Renamed** `specification.md` → `specification.V1.md` (preserved as historical record).
- **Created** `specification.V2.md` (this document).
- **New sections** (no V1 counterpart): §2.2 What Herald is NOT, §2.3 Architecture diagram, §4 Event model & wire format (CloudEvents), §5 Architecture overview, §6 Channel addressing & routing (Apprise-style URLs + tags), §7 Subscriber model (identity, preferences, quiet hours, locale), §13 Templating, §14 Localization, §15 Security model (three-layer), §16 Multi-tenancy & isolation (Postgres RLS + Redis ACL), §17 Observability & SLOs (OpenTelemetry), §20 Extensibility (Tier A + Tier B), §21 Supply chain & release engineering (SLSA L3), §26 Operations, §27 Roadmap.
- **Populated V1 TBDs**:
    - `System Herald` (was TBD) → fully specified §18.3.
    - `Project Herald's Constitution rules` → folded into §18.2 subscriber-command contract.
    - `Input commands` → expanded per-flavor in §18.
    - `Input attachments` → §15.3 content-level scanning + size limits.
    - `Others and misc` → §18.4–18.11 (7 new flavors + future-flavor namespace reservations).
- **New flavors** beyond V1's `pherald` + `sherald`: `bherald` (Build), `dherald` (Deploy), `aherald` (Alert), `scherald` (Schedule), `iherald` (Incident), `rherald` (Release), `cherald` (Compliance) — plus reserved namespaces for `fherald` / `mherald` / `gherald` / `xherald` / `hherald` / `lherald` / `vherald`.
- **Game-changing architecture adoptions** (from research):
    - **CloudEvents v1.0** as canonical wire format.
    - **Watermill** as routing core.
    - **River + Postgres** as default queue (transactional enqueue); NATS JetStream as opt-in.
    - **Apprise URL+tag** channel model.
    - **Knock-style preferences** matrix.
    - **OpenTelemetry** end-to-end + Prometheus semantic conventions for messaging.
    - **PostgreSQL RLS** + Redis ACL for multi-tenancy.
    - **SLSA L3** supply chain (cosign + syft SBOM + in-toto provenance).
    - **MJML** for email templates, compiled at author time.
    - `nicksnyder/go-i18n` with CLDR plurals.

- **All V1 text-level Recommendations** (R-01 / R-04 / R-09 / R-10 / R-15 / R-19 / R-20 / R-21 / R-22) carried forward and re-expressed in V2 context.
- **Per-channel feature matrix** (§11.13) — every channel now has documented V1-minimum, V2-advanced, out-of-scope tiers + delivery-evidence ceiling.
- **Email deep dive** (§11.9) — DKIM signing (RFC 6376) + RFC 8058 one-click unsubscribe + DSN bounce parsing + suppression list + `doctor email` DNS verification, addressing Gmail/Yahoo 2024 sender requirements.
- **Removed** the V1 Review section — its findings are folded into the V2 body or marked applied/deferred in §27.2. V1's full Review remains in `specification.V1.md` for traceability.

V2 r2 → r3 (within V2 lifecycle, all detailed in §30): added §11.0 Go type contract for the Channel interface, defined `webhook_sources` / `channel_addresses` / `thread_refs` / `quarantined_messages` schemas, pinned Go ≥ 1.22 + license, added §16.1 Data retention & GDPR, §17.1 OpenTelemetry env-var table, §26.5 Operator quickstart, §26.6 Disaster recovery, §27.3 Cost considerations. R3 added the remaining undefined types (`Subscriber`, `CloudEventEnvelope`, `TraceContext`, `Branding`, `ChannelID`, `PreferenceSet`), §9.6 Database migration tooling, §3.4 Worker pools + SIGHUP hot-reload, §5.7 Ingress API URLs, §7.5 AI-agent subscribers, §8.3 Workable-item lifecycle, §11.14 `null://` sandbox channel, §17.4.1 Per-channel SLO budgets, §24.1 Machine-readable API specs, §5.4.1 Outbound idempotency, §3.5 Time/clock abstraction.

29.2 V2 → V3 changes (2026-05-20)

- **Created** `specification.V3.md` (this document). `specification.V2.md` marked `Status=superseded` and kept as historical record alongside `specification.V1.md`. V1 + V2 + V3 stack constitutes the spec evolution chain.
- **NEW top-level sections** (no V2 counterpart):
  - **§31 Project integration contract** — explicit contract for what a consuming project provides + what Herald provides in return; composition with eight named Universal Constitution mandates (§11.4.12, .15, .16, .21, .32, .44, .55, .59, .60, .61, .65); project-side `config.toml` template; mandatory `test_project_integration.sh` audit gate.
  - **§32 Inbound processing pipeline** — full lifecycle from `channel.Subscribe` to diary append. 30 s polling cadence mandate (per-channel mechanic table). FIFO ordering per `(tenant, channel_address)` via Postgres advisory locks. Seven Worker stages (validation → safety → anti-spam → classification → dispatch → materialization → reply). `inbound_messages` schema. Anti-spam at four layers (per-sender rate, burst detection, reputation, channel-level frequency).
  - **§33 LLM/agent dispatch** — Claude Code as first integration. `resolve_session(project_name)` algorithm anchored at `<consuming-project>/.herald/claude-code/sessions/<project>.session`. Structured `<<<HERALD-DISPATCH-v1>>>` request envelope. JSON `<<<HERALD-REPLY>>>` response schema (outcome enum, summary, details, affected_paths, reproduction_steps, estimated_effort, workable_item_proposed, follow_up_questions). Pluggable `Dispatcher` interface — V3 r1 ships only `claude-code`; spec hooks for OpenCode/Aider/Gemini/Cursor/Managed Agents.
  - **§34 Reply protocol** — three mandatory replies per inbound message: (A) queued ack with quoted original ≤ 5 s, (B) processing-started ≤ 5 s of dispatch (edit-in-place where channel supports), (C) final result within criticality SLA. Failure replies carry precise reason + diagnostics. Per-channel quote/edit matrix for all 13 channels. Operator-tunable verbosity (`minimal | normal | verbose`).
  - **§35 Versioned reports + Git linkage** — report-aware event types (`.report.created/.updated/.archived`) with `update_key` for

correlation. Per-channel update mechanic (edit-in-place vs re-post). Git commit SHA + URL + diff URL embedded in fan-out. `report_publications` schema with `(content_sha256, git_commit_sha)` dedup. Uncommitted-changes warning.
    - **§36 Outbound multi-format attachments** — every Herald-produced attachment ships in four formats (`.md` + `.html` + `.pdf` + `.docx`) so subscribers pick their preferred. Pandoc generation pipeline with `--reproducible` flag. Cache at `<diary_root>/.herald/format-cache/<sha256>/`. Per-channel delivery rules + per-subscriber `PreferenceSet.attachment_formats` overrides. Bundle size cap (default 50 MiB).
- **Expanded §18.2 Project Herald** with the full **Investigation-before-Fixing flow** (§18.2.1): every bug/issue/implementation request first creates a `HRD-INV-NNN` investigation item; LLM (§33) analyzes reproducibility + affected paths + effort; only validated investigations promote to final workable items. Adds §18.2.2 criticality determination table (with explicit SLA windows per level), §18.2.3 Universal §11.4.16 type-classification mapping, §18.2.4 attachment storage at `issues/users/attachments/<WORKABLE_ITEM_ID>/` with `attachments_index.md` source-of-truth file, §18.2.5 Claude Code project-session integration (anchor file at `.herald/claude-code/sessions/<project>.session`).
- `§29` **retitled** "Changelog" (was "Changelog (V1 → V2)") so V1→V2 / V2→V3 / future changelogs accumulate under one logical section.

V2 sections that V3 does NOT change (still authoritative — V3 just extends): §1–§28 architectural baseline, §11.0 channel contract types, all flavor sections §18.3–§18.11 (refinement scheduled for V3 r2 per the metadata `Continuation` row).

29.3 V3 → V4 changes (2026-05-31)

- **Created** `specification.V4.md` (this document). `specification.V3.md` moved to `docs/specs/mvp/archive/specification.V3.md`, marked `Status=superseded`, and kept as a historical record alongside V1 + V2. V1 + V2 + V3 + V4 constitute the spec evolution chain.

- V4 is a strict **superset** — every V3 section is preserved; V4 only **adds**. Two operator-mandated features (2026-05-31) are woven into the existing structure:

  Feature 1 — Participant identity + workable-item attribution + notification @-tagging (normative: `docs/design/PARTICIPANT_ATTRIBUTION.md`; constitution: inherited §11.4.104):

    - **§7.6 Participant identity** — the logical `subscribers` + per-channel `subscriber_aliases.username` model (one person → a different `@username` per messenger); the `Participant` / `IdentityResolver` / `MemoryResolver` Go contract; the `HERALD_<CHANNEL>_OPERATOR_USERNAME` operator designation (env-var, not a DB flag); the `Claude` system-agent sentinel handle. Migration `000015` adds `subscriber_aliases.username`. **As-built: LIVE.**
    - **§8.4 Workable-item attribution** — `created_by` + `assigned_to` (canonical-handle strings) on every item, with the attribution rules (CLI-prompt→Operator / system-detected→`Claude` / received-message→@sender; `assigned_to` default = Operator). SQLite SSoT + `commons_workable` + Markdown-tracker column additions. **As-built: LIVE.**

    - **§32.6.A Inbound attribution** — how Stage 6 sets `created_by`/`assigned_to` from the resolved sender (never overwriting LLM-supplied values; nil-resolver = Wave-6 behaviour). **As-built: LIVE.**
    - **§34.7 Notification @-tagging matrix** — the outbound rules (tag `assigned_to` if human≠Operator; tag `created_by` if human≠Operator≠`Claude`; Operator + `Claude` never tagged; skip if no alias on the target channel), via `commons.MentionsFor` + per-channel mention rendering. **As-built: LIVE.**

  Feature 2 — Natural-language intent recognition + clarify-and-tag (normative: `docs/design/INTENT_RECOGNITION.md`; constitution: inherited §11.4.105):

    - **§32.6.B Three-tier intent resolution** — Tier-1 deterministic `CommandRecognizer` (no prefix required; conservative), Tier-2 LLM intent inference, Tier-3 clarify-and-tag; the recognized command set; "no command syntax required / never guess / never ignore". The legacy §32.6 trigger-prefix table is retained as the special case where the user uses a recognizable keyword.
    - **§33.8 Tier-2 envelope instruction + the `clarify` action** — the additive INTENT RECOGNITION envelope block; the new `<<<HERALD-REPLY>>> action: "clarify"` + `Question` field (action set now seven values).
    - **§34.8 Tier-3 clarify reply** — the tag-and-ask fallback: reply to the original, tag the sender `@username` (via §7.6), ask a precise question naming the candidate intents.
    - **As-built:** the Tier-1 `CommandRecognizer`, Tier-2 envelope instruction, and Tier-3 `clarify` action/handler are implemented and being landed in a parallel implementation stream; V4 describes them per the design contract (the normal role of a spec) and annotates each subsection inline.

- **Header table** gains `Supersedes` + `What changed from V3` rows; the V3 status-summary is carried verbatim under `V3 status summary (carried)` for traceability.

- **ToC** gains §7.6, §8.4, §32.6.A, §32.6.B, §33.8, §34.7, §34.8, §29.3.

V3 sections that V4 does NOT change (still authoritative — V4 only extends): §1–§7.5, §8.1–§8.3, §9–§32.6 (legacy prefix table retained), §32.7–§33.7, §34.1–§34.6, §35–§44. The two new features compose with the existing §32 inbound pipeline, §33 dispatch, §34 reply protocol, §35/§38 outbound, and the §107 anti-bluff covenant without altering any of them.

---

§31. Project integration contract

> **First-version focus.** §31 makes Herald a load-bearing piece of any consuming project that already follows the Helix Universal Constitution. The contract is symmetric: the consuming project gets a fully-instrumented event-to-channel pipeline; Herald gets a discoverable place to live and the constitution's rules to operate under.

31.1 Where Herald lives in a consuming project

Herald is consumed as a Git submodule. Conventional path:

```
<consuming-project>/
├── constitution/                  # Helix Universal Constitution (also
├── herald/                        # this repo, as submodule
│   ├── pherald/cmd/pherald/main.go   # built per §21
```

```
│   ├── sherald/cmd/sherald/main.go
│   └── …
├── containers/                        # docker/podman compose (also a subm
└── docs/
    ├── Issues.md
    ├── Issues_Summary.md
    ├── Fixed.md
    ├── Fixed_Summary.md
    ├── CONTINUATION.md
    ├── Status.md
    └── herald/
        └── diary/main.{md,html,pdf,docx}
```

The consuming project's CI invokes `<flavor>herald send` from build steps and runs `<flavor>herald serve` as a daemon (Docker Compose / Kubernetes). Subscribers DM the configured bots; Herald processes those inbound messages, opens workable items, and replies in-thread.

31.2 Mandatory integration points

A consuming project that adopts Herald MUST:

1. **Add Herald as a Git submodule** at `<consuming-project>/herald/`. Run `install_upstreams.sh` per Universal §11.4.36.
2. **Register Herald in the** `Owned-submodule set` of the project's `Constitution.md` extension (per Universal §4 / §11.4.28).
3. **Pin the Herald version** via submodule SHA. Bumps follow Universal §11.4.32 (post-pull validation).
4. **Provide credentials** via `.env` + shell-exported variables (per §3.3). `.env` is git-ignored; `.env.example` is committed and templates every variable Herald needs.
5. **Wire the channel addresses** by populating `channel_addresses` (per §6) with the project's bot tokens / webhook URLs. Tags follow project conventions — common defaults: `prod`, `staging`, `oncall`, `audit`, `compliance`.
6. **Mount the diary path** so `<consuming-project>/docs/herald/diary/main.{md,html,pdf,docx}` is writable by Herald (per §19).
7. **Honor the spec-change rule** — any change to `<consuming-project>/herald/docs/specs/` triggers comprehensive planning per Universal §11.4 + Herald Constitution §106 (read order documented in `CLAUDE.md`).
8. **Add Herald CI hooks** — at minimum, post-commit hook calling `<flavor>herald send --type digital.vasic.herald.ci.commit.pushed` … and CI-build webhook configured per §5.5.

31.3 Event-type registration

The consuming project MAY introduce its own event subtree under `digital.vasic.herald.<project_subtree>.*` and register the schema in the AsyncAPI spec (§24.1). The schema MUST be committed in `<consuming-project>/herald/docs/api/asyncapi.<project>.yaml`. Herald validates inbound events against the schema and rejects malformed payloads with HTTP 422 + a per-channel error reply.

31.4 Composition with existing constitution mandates

Universal mandate	Herald V3 contribution
§11.4.12 Auto-generated docs sync	Herald emits events on every Issues / Issues_Summary / Fixed / Fixed_Summary / Status / Status_Summary change so subscribers get real-time visibility.
§11.4.15 Item-status tracking	Status transitions trigger `digital.vasic.herald.project.task.*` events.
§11.4.16 Item-type tracking	Type classifier (§32.4) maps inbound `Bug:`/`Issue:`/`Task:`/`Implementation:` commands to the correct item type.
§11.4.21 Operator-blocked status	When a workable item enters `operator-blocked`, Herald fan-outs to the configured `oncall` tag with the unblock-question payload.
§11.4.32 Post-Constitution-Pull validation	Herald's submodule bumps are validated against the constitution; Herald's gate (`tests/test_constitution_inheritance.sh`) is part of that validation.
§11.4.44 Document Revision Header	Herald respects §11.4.61 superseding format (table) when reading project docs.
§11.4.55 Reopens-history	Reopen flow (§8.3) writes to docs/Reopens/<HRD-NNN>.md.
§11.4.59 README always-sync	Herald reports README out-of-sync as an event (`digital.vasic.herald.compliance.doc.stale`).
§11.4.60 Documentation composite covenant	Herald's diary + multi-format exports per §36 satisfy the composite gate.
§11.4.61 Markdown metadata + ToC	Herald's diary follows the canonical metadata table; every diary entry carries Revision + Last modified per §19.
§11.4.65 Universal Markdown export	§36 multi-format pipeline is the implementation of this mandate at attachment-time.

31.5 Project-side configuration template

```toml
# <consuming-project>/herald/config.toml — committed (no secrets)
[herald]
flavor       = "pherald"
project_name = "ATMOSphere"                    # used for Claude Code session resolution (§33.2)
tenant_id    = "00000000-0000-0000-0000-000000000001"
diary_root   = ""                              # blank → parent-walk discovery (§19.1)
admin_port   = 24090
http_port    = 24091

[ingest]
```

```
poll_interval_seconds = 30                          # §32.2 —
        mandatory 30s cadence
fifo_strict           = true                        # §32.3 strict
        FIFO

[security]
spam_max_per_minute_per_sender = 10               # §32.5 anti-spam
        thresholds
spam_max_per_hour_per_sender   = 100
quarantine_unknown_sender      = true

[llm]
default_agent       = "claude-code"                # §33.1 default
        dispatch target
claude_code_binary = "claude"                      # path / name on
        PATH
session_strategy    = "project-cwd"                # §33.2 strategy
        enum

[attachments]
out_format_set      = ["md", "html", "pdf", "docx"]  # §36 multi-
        format default
in_storage_root     = "issues/users/attachments"      # §18.2
        inbound attachment path
in_max_mib          = 25
in_total_max_mib    = 100
```

31.6 Project audit gate

`<consuming-project>/herald/tests/test_project_integration.sh` (planned) asserts:

- `constitution/` submodule discoverable via parent-walk.
- `containers/` submodule present and version-pinned.
- `.env.example` committed; `.env` git-ignored.
- `channel_addresses` table non-empty for the project's default tenant.
- Project's `Owned-submodule set` lists `herald/` and `containers/`.
- Diary path exists and is writable.
- `pherald doctor` exits zero.

Paired §1.1 mutation: rename `.env` to track-ed — gate FAILs.

---

§32. Inbound processing pipeline

32.1 Pipeline overview

Every message a subscriber sends back into Herald (DM reply, channel mention, email reply with Herald's `Message-ID` in `In-Reply-To`, ntfy poll) enters the **inbound queue** for sequential processing by Workers. The pipeline is FIFO per tenant per channel-address, with explicit anti-spam, multi-stage validation, and three user-visible reply checkpoints (§34).

```
channel adapter Subscribe() loop
   ↓
InboundEvent → ingress dedup (§4.3) → enqueue `inbound_messages` table
   ↓
 ┌──────────────────────────────────────────────────────────────────────┐
 │ Inbound Worker (River queue, 30s tick OR LISTEN/NOTIFY)              │
 │ pulls 1 item at a time per tenant per channel (FIFO):                │
 │                                                                      │
 │ Stage 1: Validation (§32.3) → reply: queued ack (quote orig)         │
 │ Stage 2: Safety/anti-malware (§32.4)                                 │
 │ Stage 3: Anti-spam check (§32.5)                                     │
 │ Stage 4: Classification (§32.6) → type + criticality                 │
 │ Stage 5: Dispatch to LLM/agent (§33) → reply: processing-started     │
 │ Stage 6: Materialize side-effects (open workable item, …)            │
 │ Stage 7: Reply with final result (§34) → workable item ID, links     │
 └──────────────────────────────────────────────────────────────────────┘
   ↓
diary append (§19); OTel span closed
```

32.1.1 Implementation map (Wave 3b)

Each §32 stage of the **outbound** ingest path (POST /v1/events) is implemented by a concrete struct in `pherald/internal/runner/`. Stages communicate exclusively via `RunCtx` — no shared interface; the orchestrator (`runner.Runner.Run`) holds stage instances as fields and calls them in fixed order:

Stage	Responsibility	Implementation
1. EventParser	Parse + validate inbound CloudEvent; extract Trace context + idempotency key	`pherald/internal/runner/event_parser.go`
2. IdempotencyChecker	Redis SETNX + PG `events_processed` fallback; sets `rc.Duplicate`	`pherald/internal/runner/idempotency.go`
3. TenantResolver	Bind tenant GUC on the transaction for RLS	`pherald/internal/runner/tenant.go`
4. PolicyGate	Run registered constitution evaluators; sets `rc.PolicyDecision`	`pherald/internal/runner/policy.go`
5. SubscriberResolver	List subscribers + aliases for the tenant	`pherald/internal/runner/subscriber.go`
6. ChannelDispatcher	Fan out to the channel adapters per recipient alias	`pherald/internal/runner/dispatcher.go`
7. OutcomeRecorder	Persist `outbound_delivery_evidence` rows + `events_processed` archive	`pherald/internal/runner/outcome.go`

Short-circuits:

- Stage 2 duplicate → cached Receipt returned with `WasReplay=true`; no dispatch.
- Stage 4 `DecisionFail` → `OutcomeRecorder.RecordDenied` writes a single `policy_denied` evidence row + archive row; no dispatch.

The HTTP adapter is `pherald/internal/http/events.go` (`EventsHandler(r *runner.Runner)`).

Wave 3b ships a permissive `commons_constitution.Registry` by default (no evaluators registered) — flavor binaries that need policy enforcement register evaluators on top of the same Runner.

32.2 Polling cadence (30 s mandate)

Every flavor's Subscribe loop MUST check upstream for new messages **at least every 30 seconds**. Upstream-specific implementation:

Channel	Mechanism	Effective check rate
Telegram	Bot API `getUpdates` long-poll (timeout 25 s) — **LIVE (Wave 6 / Wave 7)**	continuous, but a 30 s safety-net timer also fires `getUpdates` if the long-poll thread stalled
Slack	Socket Mode (WebSocket) OR Events API webhook — **LIVE (Wave 7 T6 — Socket Mode)**	continuous on Socket Mode; 30 s timer as keepalive ping
Discord	Gateway WebSocket	continuous; 30 s timer asserts heartbeat OK
MS Teams	Bot Framework activity webhook	webhook-driven; 30 s timer asserts subscription validity
Lark	Event subscription webhook	same
WhatsApp Cloud	Webhook	same
Viber	Webhook	same
Email	IMAP IDLE preferred; fall back to 30 s IMAP poll	IDLE = continuous; non-IDLE = 30 s
ntfy	WebSocket/SSE	continuous
Gotify	WebSocket	continuous
Max	Bot API poll	30 s

Configurable via `[ingest].poll_interval_seconds` (default 30); operators MAY tighten to 10 s for high-volume tenants but MUST NOT loosen beyond 60 s without operator-explicit override (`--allow-slow-poll`).

32.3 FIFO order + Worker selection

Inbound items are queued in `inbound_messages` with `enqueued_at` timestamp + UUIDv7 id. The River queue is partitioned by `(tenant_id, channel_address_id)` so:

- Multiple senders on the same channel are processed in strict arrival order.
- Different channels for the same tenant run in parallel.
- Different tenants run in parallel.

Schema:

```
CREATE TABLE inbound_messages (
    id                    UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id             UUID NOT NULL,
    channel               TEXT NOT NULL,
    channel_address_id    UUID NOT NULL REFERENCES
        channel_addresses(id),
    sender_channel_user_id TEXT NOT NULL,
    received_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    cloudevent_jsonb      JSONB NOT NULL,
    attachments_jsonb     JSONB NOT NULL DEFAULT '[]'::jsonb,
    stage                 TEXT NOT NULL DEFAULT 'queued',      --
        queued|validating|safety|anti_spam|classifying|
        dispatched|completed|failed
    stage_started_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    classification        JSONB,                               --
        type+criticality+confidence
    workable_item_id      TEXT,                                --
        after successful materialization
    last_reply_at         TIMESTAMPTZ,
    failure_reason        TEXT,
    failure_details       JSONB
);
CREATE INDEX inbound_fifo_idx ON inbound_messages (tenant_id,
        channel_address_id, received_at)
    WHERE stage NOT IN ('completed', 'failed');
ALTER TABLE inbound_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE inbound_messages FORCE ROW LEVEL SECURITY;
CREATE POLICY inb_isolation ON inbound_messages
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

The Worker holds a per-`(tenant_id, channel_address_id)` advisory lock for the duration of one item's pipeline so strict FIFO is preserved across the seven stages.

32.4 Stages 1–2: validation + safety

Stage 1: Validation. Asserts:

- CloudEvents `id` is unique within idempotency window (§4.3 hit/miss).
- `sender_channel_user_id` matches an entry in `subscriber_aliases` OR is allowlisted as a verified webhook source.
- Message body is well-formed UTF-8 ≤ 8 KiB per command + ≤ 256 KiB total.
- Attachments declared in `attachments_jsonb` MUST be retrievable from the source channel and MUST be ≤ `[attachments].in_max_mib` each (default 25 MiB) and ≤ `[attachments].in_total_max_mib` total (default 100 MiB).

Failure → reply with precise reason (§34.4), stage = `failed`, NO further processing.

Stage 2: Safety / anti-malware. Each attachment is scanned:

- ClamAV (when commons_security.clamav.enabled=true and the daemon is reachable) — definitive engine.
- Magic-byte type check vs declared MIME (rejects MIME spoofing).
- Filename allowlist by extension (default: .txt .md .json .yaml .yml .csv .pdf .docx .html .png .jpg .jpeg .webp
- Optional: per-tenant additional scanner (configurable hook command receives the file path, returns 0 = clean, non-zero = quarantine).

Inbound text body is scanned for:

- URL allowlist / blocklist per tenant.
- Prompt-injection markers (heuristics — short list of known patterns: Ignore previous instructions, BEGIN SYSTEM, <|im_start|>, etc.). Hit → quarantine, NOT auto-reject (operator review).

Failure → reply with reason, stage = failed, attachment quarantined to quarantined_messages.attachments_jsonb.

32.5 Stage 3: anti-spam

Spam prevention is layered:

1. **Per-sender rate limits** (configurable per tenant via [security].spam_max_*):
    - spam_max_per_minute_per_sender (default 10)
    - spam_max_per_hour_per_sender (default 100)
    - spam_max_per_day_per_sender (default 500) Exceeded → message accepted, but reply downgraded to 429 Too Many Requests with the cooldown window.
2. **Burst detection** — a sender that posts > N similar messages (Levenshtein distance < 5%) within window T is throttled; if pattern continues, sender enters quarantine.
3. **Sender reputation** — per-(tenant, sender_channel_user_id) score in Redis (t:<id>:rep:<sender> integer), incremented on successful interaction, decremented on validation/safety failures. Negative score → mandatory operator review before processing.
4. **Channel-level frequency** — if channel_addresses[address_id] receives > rate_floor messages/minute from distinct senders, channel-wide soft rate-limit kicks in.
5. **Known-bot allowlist** — agent tokens (§7.5) bypass per-minute limits but obey per-hour caps.

All spam decisions logged to spam_audit (Redis stream, 7-day retention) for tuning.

32.6 Stage 4: classification

The classifier (running as a Watermill handler in commons_messaging) determines:

Item type (per Universal §11.4.16 + Project Herald subscriber-command vocabulary):

Inbound trigger	Type	Maps to
Bug: / Issue: / Issue: <prose>	bug	docs/Issues.md row + digital.vasic.herald.project.task.ope
Task:	task	same docs flow, different type column

Inbound trigger	Type	Maps to
`Implementation:` / `Implement:` / `Feature:`	`implementation`	same
`Query:` / `Question:` / `Q:` / `?`	`query`	LLM-only path (no workable item — research/info requ
`Request:` / `Req:`	`request`	LLM-routed; outcome may produce a workable item if approved
`Investigation:` / `Investigate:`	`investigation`	intermediate state (§32.7)
`Status:`	`status_request`	reply with current state from `docs/Status.md`
`Continue:`	`continuation_request`	reply with `docs/CONTINUATION.md` pointer
`Done:` / `Resolve:` (operator role only)	`closure`	close workable item, migrate Issues→Fixed
`Reopen:` (operator role only)	`reopen`	per Universal §11.4.55 reopen flow
`Ack:` / `Silence:` / `Snooze:`	`ops_command`	per-flavor ops handlers
`Spec:`	`spec_change_request`	gated by §23 spec-change rule

Criticality (independent of type; required for `bug`, `issue`, `task`, `implementation`, `investigation`):

Level	Trigger	SLA
`critical`	explicit `critical:` prefix in subject line, OR LLM classifier confidence ≥ 0.85 on production-outage keywords	acknowledge within 5 min; first investigation within 1 h
`high`	explicit `high:` OR LLM 0.7–0.85	within 30 min; investigation within 4 h
`middle`	default	within 2 h; investigation within 24 h
`low`	explicit `low:` or `nice-to-have:`	within 24 h; investigation within 1 week

The classifier itself is **deterministic-first** (keyword/regex), falling back to LLM (§33) only when the deterministic step returns ambiguous. Operators MAY override classifications via `Override: HRD-NNN type=task criticality=high`.

> **V4 note.** The `Bug:` / `Task:` / `Status:` etc. trigger-prefix table above is the **legacy explicit-command surface** — it remains valid and authoritative. V4 adds, on top of it, the **natural-language intent recognition** layer (§32.6.B): a subscriber no longer NEEDS to know the prefixes; a plain-language message is resolved to the same actions. The prefix table is now the special case where the user happens to use a recognizable keyword; the three-tier resolver generalizes it.

32.6.A Inbound attribution — setting `created_by` / `assigned_to` — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/PARTICIPANT_ATTRIBUTION.md` §2 + §5. Constitution authority: inherited

§11.4.104. **As-built:** LIVE — `pherald/internal/inbound/attribution.go` + `dispatcher.go` + `item_mutator.go`.

When Stage 6 materialization (§33.4) opens or updates a workable item as a result of an inbound message, the dispatcher sets the §8.4 attribution fields:

- `created_by` ← `IdentityResolver.ResolveSender(channel, channelUserID, senderUsername)` for a subscriber-originated item; or `commons.SystemAgentHandle` (`Claude`) on the system-detected path.
- `assigned_to` ← `IdentityResolver.OperatorHandle()` by default; overridden when the message carries an explicit `assign: @bob` directive (or the Tier-1 `assign … to @bob` intent, §32.6.B).
- An LLM-supplied value (from the `<<<HERALD-REPLY>>>` payload) is **never overwritten** by the defaulting logic.
- When no `IdentityResolver` is configured the inbound path preserves Wave-6 behaviour (attribution left empty) — the feature is additive, never a hard dependency.

32.6.B Natural-language intent recognition (three-tier resolution) — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/INTENT_RECOGNITION.md` §1–§2 + §5. Constitution authority: inherited §11.4.105. **As-built:** Tier-1 `CommandRecognizer` + Tier-2 envelope instruction + Tier-3 `clarify` action are implemented (`pherald/internal/inbound/command_recognizer.go`, `commons_messaging/dispatch/claude_code` envelope, `reply.go` `clarify`); the Tier-1 recognizer + Tier-3 clarify handler are landing in a parallel implementation stream and are described here per the contract (the normal role of a spec).

Operator mandate: users MUST NOT need to know command syntax (no `COMMAND:` prefix). They send a clear natural-language message; the System determines the intent. It recognizes the commands it has; if none matches it infers the exact intent; if it is *totally unable* it **replies, tags the user (`@user …`), and asks to clarify precisely**. We always do our best to determine the exact intent so end users are never annoyed. This is a CORE part of the System.

Every inbound subscriber message is resolved to exactly one action via three tiers, in order — the first that succeeds wins:

```
TIER 1  command recognition  — a deterministic CommandRecognizer maps clea
                               natural-language commands to a structured a
                               WITHOUT an LLM round-trip (fast-path; no pr
                               required). Confident match → that action.
TIER 2  intent inference     — when no command matches, the Claude Code di
                               (§33) infers the intent and returns a
                               <<<HERALD-REPLY>>> action. The envelope (§3
                               INSTRUCTS the LLM to recognize Herald's com
                               set, map plain language to the right action
                               NEVER guess.
TIER 3  clarify (fallback)   — when neither a command nor a confident inte
                               be determined, the action is "clarify": the
                               replies to the original message, tags the s
                               (@username, via the §7.6 IdentityResolver),
                               asks a precise clarifying question. No gues
                               no silent drop (§34.8).
```

The command set Tier 1 recognizes (no prefix; case-insensitive; tolerant of phrasing; maps to the EXISTING inbound actions):

Natural-language intent (examples)	action	fields
"close ATM-123", "mark ATM-5 fixed/ done/resolved"	`item.update`	status=closed/fixed
"set ATM-9 to in progress", "ATM-9 is blocked"	`item.update`	status=`<parsed>`
"assign ATM-5 to @bob", "give ATM-5 to @bob"	`item.update`	assigned_to=@bob
"open a bug: `<title>`", "create a task: `<title>`", "new feature request: `<title>`"	`issue.open`	type+title (created_by=sender, §32.6.A)
"investigate ATM-7", "look into ATM-7"	`investigation.start`	atm_id
"status of ATM-9?", "what's ATM-9?"	`reply` (query)	atm_id
anything conversational / a question	`reply`	—
ambiguous / unparseable intent	`clarify` (§34.8)	question

The recognizer is deliberately **CONSERVATIVE**: it only fast-paths a match it is confident about (a clear imperative verb + a resolvable target). A status *query* ("status of ATM-9?") is never misread as a mutating *set-status* command. Everything else falls through to Tier 2 (the LLM); only genuine ambiguity reaches Tier 3. A false command-match is worse than a deferral to the LLM, so when in doubt the recognizer returns "no match". Tier 1 runs BEFORE the LLM dispatch and skips the round-trip on a confident match.

32.7 Stage 5: Investigation-before-Fixing intermediate state

When a `bug` / `issue` / `implementation` arrives, Herald first creates an **Investigation workable item** (type `investigation`, status `investigating`), NOT the final bug/task row. The investigation:

1. Stores in `workable_items` table with type `investigation` and `parent_request` referencing the inbound message id.
2. Dispatches to the LLM (§33) for **reproducibility analysis** — can the LLM identify the affected code path? Reproduce locally? Estimate effort?
3. Stores LLM findings as a comment in `docs/Investigations/<HRD-NNN>.md`.

Once investigation concludes:

- **Validated** → Herald creates the *final* `bug` / `task` / `implementation` workable item, atomically migrates the investigation row to "investigation completed" status, links the new item via `parent_investigation`.
- **Cannot reproduce / not actionable** → investigation row stays; replies with reason + asks for more info; subscriber's follow-up reply re-runs the investigation.
- **Rejected (out of scope / duplicate / known)** → investigation closed with that reason; no final item created.

The investigation gate is a strong guard against runaway LLM-generated workable items.

32.8 Spam audit + observability

Per §17.2 the inbound pipeline emits:

- `herald_inbound_received_total{tenant,channel,outcome}` — `outcome` ∈ `enqueued | rejected_validation | quarantined_safety | rate_limited | classified | dispatched | completed | failed`.
- `herald_inbound_stage_duration_seconds{stage}` histogram.
- `herald_inbound_classification{type,criticality,confidence_bucket}` counter.
- `herald_spam_block_total{tenant,reason}` counter.
- Span name `herald.inbound.process` with child spans per stage.

32.9 Bot self-filter (Wave 6, anti-echo, operator-mandated 2026-05-22)

Inbound message handlers MUST drop messages where `msg.Sender.IsBot && msg.Sender.Username == bot.Me.Username`. Cross-bot messages (a different bot in the same chat) are KEPT — they represent legitimate multi-bot collaboration. If `bot.Me.Username` is unset after channel construction, the runtime MUST refuse to start (fail-closed: an echo-loop-prone runtime ships only when the self-identity is known).

Implementation: `commons_messaging/channels/tgram/subscribe.go` captures `bot.Me.Username` at `Subscribe` time and applies `shouldDropBotSelf(msg, selfUsername)` before dispatching each `OnText` / `OnPhoto` / `OnDocument` / `OnVoice` handler.

§107 anti-bluff: T12 mutation gate (M1) blanks the filter; paired detector is `TestSubscribeBotSelfFilter` (assertion on the cross-bot-kept case, not just "drop all bots"). When the closed-loop runs against real Telegram, an echo loop would manifest as pherald's own reply re-entering `getUpdates` and triggering a second CC dispatch within ~25s — observable in `tests/test_wave6_live_loop.sh`.

---

§33. LLM / agent dispatch

33.1 First target: Claude Code

V3's first LLM dispatch target is **Claude Code** (the Anthropic CLI). The dispatcher lives in `commons_messaging/dispatch/claude_code/`.

Design constraints:

- Claude Code sessions are scoped to a **working directory**, not a freestanding "name". V3 introduces a **project-named session resolution algorithm** (§33.2) that maps the consuming project's name to a stable working-directory anchor.
- The CLI is invoked in **non-interactive batch mode** for each request: `claude --resume <session> --print "<prompt>"`.
- Background invocation: Herald spawns the CLI in a separate process group with timeout (default 5 min); stdout is collected; non-zero exit becomes a `failed` stage with reason recorded.

- Concurrency: per-project session is single-writer (one in-flight Claude Code invocation at a time). Multiple inbound requests for the same project queue behind each other (River job priority preserves arrival order).

33.2 Session resolution algorithm

For project named `ATMOSphere`:

```
function resolve_session(project_name) -> session_id:
    1. compute working_dir = config[herald.session_workdir]
       (default: <consuming-project-root> discovered via parent-walk)
    2. compute session_anchor_path = working_dir/.herald/claude-code/sessi
    3. if session_anchor_path exists AND contains a session UUID
         AND `claude --resume <uuid> --print "ping"` returns 0:
          → return that UUID
    4. else:
          spawn `claude --print "Initializing Herald session for project:
          in working_dir
          capture the new session UUID emitted by Claude Code stdout
          write UUID to session_anchor_path
          → return new UUID
```

The `.herald/claude-code/sessions/` directory is `.gitignore`'d. Session anchors persist across Herald restarts. Operators can manually reset by deleting the anchor file (next dispatch will create a fresh session).

33.3 Request envelope

Every dispatch sends a structured prompt:

```
<<<HERALD-DISPATCH-v1>>>
Project:        <project_name>
Inbound ID:     <UUIDv7>
Sender:         <channel>:<subscriber.handle> (verified: yes|no, roles: [o
Channel:        <ChannelID>
Received at:    <RFC 3339>
Classification: type=<type> criticality=<level> confidence=<0.0-1.0>
Conversation:   <thread-of-quoted-replies, full chain bottom-to-top>
Attachments:    [<name>:<mime>:<size_bytes>, …]

User message:

<verbatim user text>

────────────────────────────────────────────────────────────
HERALD TASK (run in background along with mainstream work):

<task verb derived from classification>:
- For `bug`/`issue`/`investigation`: reproduce + identify affected code pa
  classify root-cause area + propose validation steps.
- For `query`/`question`: research + answer; cite project docs if relevant
  short answers preferred (subscribers see this directly).
- For `request`/`implementation`: scope effort + propose approach + flag p
```

```
  + estimate workable-item dependencies.
- For `spec_change_request`: invoke Herald Constitution §106 spec-change r

Reply with a JSON object on a single line, prefixed with `<<<HERALD-REPLY>

{
  "outcome": "validated"|"rejected"|"needs_more_info"|"answered",
  "summary": "<short summary for the subscriber>",
  "details": "<longer markdown body for the diary>",
  "affected_paths": ["<file>:<line>", …],          // optional
  "reproduction_steps": ["…", …],                   // for bug/investigat
  "estimated_effort": "S|M|L|XL",                   // for request/implem
  "workable_item_proposed": {                       // optional
    "type": "bug|task|implementation|investigation",
    "criticality": "critical|high|middle|low",
    "title": "…",
    "labels": ["…"]
  },
  "follow_up_questions": ["…"]                      // when outcome=needs
}

DO NOT modify project files unless the subscriber explicitly asked you to.
DO NOT commit. DO NOT push.
<<<END-HERALD-DISPATCH>>>
```

Herald parses the `<<<HERALD-REPLY>>>` JSON line out of Claude Code's stdout, validates against the response schema, and proceeds to Stage 6 materialization.

33.4 Materialization (Stage 6)

Based on the `outcome` and `workable_item_proposed`:

- `validated` + workable_item_proposed → allocate next `HRD-NNN`, write to `docs/Issues.md` + emit `digital.vasic.herald.project.task.opened` (per §8.3 lifecycle) + link investigation parent (§32.7).
- `answered` (query/question) → no workable item; the reply IS the answer.
- `needs_more_info` → no workable item; reply asks subscriber for clarifications listed in `follow_up_questions`.
- `rejected` → no workable item; reply states rejection reason.

33.5 Other LLM/agent targets (spec hooks, V3+ implementation)

The dispatcher interface is provider-agnostic:

```go
type Dispatcher interface {
    Name() string
    Capabilities() DispatcherCapabilities
    Dispatch(ctx context.Context, req DispatchRequest)
        (DispatchResponse, error)
}
```

V3 ships only `claude-code` dispatcher. Spec hooks reserved for:

- `opencode` — same CLI pattern.
- `aider` — file-modifying dispatcher (extra safety gates required; not enabled by default).
- `gemini-cli` — when GA.
- `cursor-cli` — pending product availability.
- `agentic-claude-api` — direct Anthropic API (Managed Agents) for hosted deployments where a CLI isn't desirable.

Operators select per-tenant via `[llm].default_agent`; per-classification-type override via `[llm.routing]` table.

33.6 Envelope pre-text (Wave 6, operator-mandated 2026-05-22)

Every Claude Code envelope MUST be prefixed with the verbatim operator wording:

```
We have received new message from our communication channel <channel-name>
The message has been classified as "<type>" with "<criticality>" criticali
Sender: <channel>:<handle>. Inbound ID: <ULID>.
Attached materials:
  - <filename> (<mime>, <bytes> bytes)
[or:] No attached materials.

<<<HERALD-DISPATCH-v1>>>
... existing structured block (§33.3) unchanged byte-for-byte ...
```

The pre-text is human-language; the LLM reads natural-language context BEFORE the structured schema. The existing `<<<HERALD-DISPATCH-v1>>>` block remains byte-identical to §33.3 — pre-text is additive.

Implementation: `(*claude_code.Dispatcher).FormatEnvelopeWithPreText(req, channelName string) string`. Production `buildCmd` calls the new formatter; `FormatEnvelope` (§33.3) is preserved as the structured-only renderer for tests + tooling. §107 anchor: T11 invariant E66 greps the captured envelope bytes (from `--qa-out-dir`) for the literal opener prefix `"We have received new message from our communication channel "`.

33.7 Model pinning (Wave 6, operator-mandated 2026-05-22)

`pherald listen` (and any other call site that spawns Claude Code through the dispatcher) MUST spawn `claude --model claude-opus-4-7` (literal argv flag). No fallback to default model selection. No silent downgrade to Sonnet or Haiku. The model flag is two contiguous argv entries (`["--model", "claude-opus-4-7"]`), NOT a joined `"--model claude-opus-4-7"` string.

Implementation: `commons_messaging/dispatch/claude_code/dispatch.go::buildCmd` inserts the flag between `--resume <UUID>` and `--print <envelope>`. §107 anchor: T2's `TestDispatchCommandIncludesOpusModel` inspects `*exec.Cmd.Args` directly (NOT env vars, NOT stdin). T12 mutation gate (M2) swaps to `claude-sonnet-4-6` and asserts the detector FAILs — proving the assertion is load-bearing.

33.8 Tier-2 envelope intent instruction + the clarify action — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: docs/design/INTENT_RECOGNITION.md §3 + §4. Constitution authority: inherited §11.4.105. **As-built:** FormatEnvelope gains the intent-instruction block; <<<HERALD-REPLY>>> gains action: "clarify" + Question field (pherald/internal/inbound/reply.go).

Tier-2 envelope instruction (the LLM intent-inference wiring). When Tier 1 (§32.6.B) returns "no match", the inbound message is dispatched to Claude Code (§33). The <<<HERALD-DISPATCH-v1>>> envelope (§33.3) gains a short, additive **INTENT RECOGNITION** instruction block telling the LLM:

1. the user speaks plain language — **no command syntax** is required;
2. recognize Herald's command set (§32.6.B) and map the message to the right action;
3. if you cannot determine the intent **with confidence**, return action=clarify with a **precise** clarifying question that *names the candidate intents* — **do NOT guess** (§11.4.6 no-guessing: a wrong action is worse than a clarifying question).

This block is additive to the existing reply-format note; the rest of the §33.3 envelope (and the §33.6 pre-text) is unchanged.

The clarify action (new). <<<HERALD-REPLY>>> gains a seventh action value, clarify, with a question field:

```
<<<HERALD-REPLY>>> {"action":"clarify","question":"did you want
        to close ATM-9, reassign it, or just get its status?"}
```

The full action set is now reply (default) | issue.open | event.emit | item.update | item.delete | investigation.start | clarify. The Reply.Question field carries the precise clarifying question. The handler routing for clarify is specified in §34.8.

---

§34. Reply protocol (queued → processing → result)

Every inbound message MUST receive **three replies** unless the inbound itself was rejected at Stage 1 (in which case one reply explaining the rejection).

34.1 Reply A — Queued ack

Within ≤ 5 s of receipt:

- Quote the original message (per-channel thread mechanic per §12 ConversationRef).
- Body: 📨 Received. Queued as #INB-<short-id>. I'll process this momentarily.
- Carries Herald-Reply-Stage: queued extension header for adapters that support custom headers.

34.2 Reply B — Processing started

Within ≤ 5 s of Stage 5 dispatch:

- Quote the original (same thread).

- Body: ⏳ `Processing as <classification.type> (priority: <criticality>). Investigating…`
- Edit-in-place where the channel supports it (Telegram `editMessageText`, Slack `chat.update`, Discord webhook edit); otherwise post a new message in the thread.

34.3 Reply C — Final result

Within the SLA window for the classified criticality (§32.6):

- Quote the original (same thread).
- Body varies by outcome:
  - **Validated + workable item created**:
    ✅ `Created HRD-042 (bug, criticality=high). Investigation summary attached. Link: <Issues.md row anchor>. Repo: <git URL>`
    With multi-format attachment bundle per §36 containing the investigation summary.
  - **Answered (query)**:
    💡 `<answer>`
    With cited documents attached as multi-format if relevant.
  - **Needs more info**:
    ❓ `Need more detail: 1) <question> 2) <question>`
  - **Rejected**:
    🚫 `<reason>. <suggested next step>.`

34.4 Failure replies

If validation, safety, anti-spam, or classification fails:

- Quote the original message.
- Body: ❌ `<stage>: <precise reason>. Details: <link-to-diary-entry-with-stack-trace>.`
- Body MUST contain the exact rejection cause (not "internal error" — anti-bluff per Universal §11.4).
- Stage = `failed` in `inbound_messages`; original receipt is preserved.

Examples:

- ❌ `validation: attachment 'malware.exe' exceeds size limit (30 MiB > 25 MiB max). Details: …`
- ❌ `safety: attachment 'invoice.pdf' triggered ClamAV signature 'EICAR-Test-File'. Quarantined. Details: …`
- ❌ `anti-spam: rate limit (12 msg/min) exceeded. Cooldown until <RFC 3339 UTC>. Details: …`
- ❌ `classification: ambiguous type. Suggested prefixes: 'Bug:', 'Query:', 'Request:'. Details: …`

34.5 Per-channel quoting/edit-in-place matrix

Channel	Quote original	Edit-in-place (B → C)	Notes
Telegram	`reply_to_message_id`	`editMessageText` ✓	Reply B edited into Reply C if same message id
Slack	`thread_ts` (parent ts)	`chat.update` ✓	edits the queued ack

Channel	Quote original	Edit-in-place (B → C)	Notes
Discord	`?thread_id=` + `webhook?wait=true`	webhook edit ✓	requires bot token for non-webhook edit
MS Teams	message reference	Adaptive Card refresh × (per-message replacement only)	post new message in conversation
Lark	reply API	message edit ✓	
WhatsApp Cloud	`context.message_id`	not supported	post new message
Viber	`tracking_data` thread	not supported	post new message
Email	`In-Reply-To` + `References` headers	not supported	post new message in thread
ntfy	not natively threaded	not supported	new message with `X-Tag: same-as-original`
Diary	parent-id anchor	append + cross-link	

When edit-in-place is unavailable, Herald posts a NEW message and Reply B+C are distinct.

34.6 Configurable verbosity

Operators MAY tune reply verbosity per tenant via `[replies].verbosity = minimal | normal | verbose`:

- `minimal` — only Reply A (queued ack) + Reply C (final result); no Reply B.
- `normal` (default) — A + B + C as described.
- `verbose` — A + B + C + per-stage progress (Stage 3 done, Stage 4 done, …).

34.7 Notification @-tagging matrix (outbound participant mentions) — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/PARTICIPANT_ATTRIBUTION.md` §3. Constitution authority: inherited §11.4.104. **As-built:** LIVE — `commons.MentionsFor` (the matrix) + `commons_messaging/channels/tgram/mentions.go` (`RenderMentions`/`PrependMentions`) + the outbound wiring `pherald/internal/workflow` `NewTaggingNotifier`.

On any workable-item event (§35 / §38), the outbound notification dispatched to each messenger channel/group **@-tags** the participant(s) who must be aware, each resolved to that channel's `@username` (§7.6). The matrix:

```
mentions = {}
if assigned_to is a human handle AND assigned_to != Operator:   mentions +
if created_by  is a human handle AND created_by  != Operator AND created_b
                                                                 mentions +
# "Claude" (commons.SystemAgentHandle) is NEVER tagged — it is the system.
# Operator is NEVER tagged — the operator drives the system; no self-ping.
# de-dup; order = assigned_to before created_by; for each mention resolve
# UsernameFor(handle, channel) — SKIP if the participant has no alias on t
```

This exactly satisfies the operator's stated rules:

- **assigned to Operator → no tag** (both `assigned_to==Operator` and `created_by==Operator` are skipped).
- **opened by Operator, assigned to another → tag the assignee** (`assigned_to≠Operator` is tagged).
- **opened by a non-Operator non-Claude subscriber → tag them** (`created_by` is tagged).
- **a participant not on the target messenger → not tagged there** (`UsernameFor` returns `ok=false`; you cannot tag someone who is not on that channel).

`commons.MentionsFor(createdBy, assignedTo, operatorHandle, channel, r IdentityResolver) []string` returns the canonical handles to mention (NOT the resolved `@usernames`), in stable de-duplicated order, having dropped `Claude`, the operator, and any handle with no alias on `channel`. Each channel adapter renders the resolved `@usernames` in its own mention syntax — Telegram prepends/appends a `cc: @username` line (`tgram.PrependMentions`); Slack `<@U…>` and other adapters render their native mention form (future). §107 anti-bluff: the outbound E2E asserts the captured dispatched body contains exactly the expected `@usernames`, with a NEGATIVE case proving the Operator is never self-tagged; a paired mutation that removes the operator-skip makes the E2E FAIL.

34.8 Tier-3 clarify reply (tag-and-ask fallback) — V4

> **New in V4** (operator mandate 2026-05-31). Normative contract: `docs/design/INTENT_RECOGNITION.md` §3 + §5. Constitution authority: inherited §11.4.105. **As-built:** the `clarify` handler is landing in the parallel intent-recognition stream; described here per the contract.

When intent resolution (§32.6.B) reaches Tier 3 — neither a Tier-1 command nor a confident Tier-2 LLM intent — the action is `clarify` (§33.8). The dispatcher's `clarify` handler:

1. Sends a **reply to the original message** (`Replier.SendReply`, quoting/threading the original per §12 `ConversationRef`).
2. The reply body is `@<sender-username> <question>` — the sender resolved to their per-channel `@username` via `IdentityResolver.UsernameFor(senderHandle, channel)` (falling back to the raw sender handle if there is no alias).
3. The `<question>` is **specific** — it names the candidate intents ("did you mean close, reassign, or status?"), never a generic "I didn't understand".

Tier 3 is the **anti-annoyance guarantee**: the user is never ignored and never has to learn syntax — at worst they get a friendly, precise "@user, did you mean X or Y?". The LLM is instructed (§33.8) to return `action=clarify` with a precise question rather than guess an action (§11.4.6). §107 anti-bluff: the Tier-3 E2E asserts the recording-sink reply body is exactly `@<sender> <specific question>` (proving the user is tagged + asked, not ignored); a NEGATIVE asserts a clear command does NOT trigger clarify; a paired mutation that drops the clarify tag or breaks the recognizer's confidence guard makes a test FAIL.

---

§35. Reports + state-tracking documents (versioned fan-out with Git linkage)

Some events carry **reports that change over time** — security-scan summaries, build-status digests, weekly project digests, compliance reports. When a report's source document changes, Herald MUST re-publish to subscribers with the updated file attached + a link to the Git commit that introduced the change.

35.1 Report-aware event types

CloudEvents types under `digital.vasic.herald.report.*` are special:

- `.report.updated` — content changed since last publish.
- `.report.created` — first publish.
- `.report.archived` — no longer being maintained.

Each carries an `update_key` extension attribute (stable identifier for the report) so Herald can correlate updates with previous deliveries.

35.2 Update mechanic per channel

When a `.report.updated` event arrives:

Channel	Mechanism
Telegram	Send new message in the channel with the updated file attached + caption `Updated: <git short-sha> — <commit message subject>`. If channel supports `editMessageMedia` for original messages, edit in place; otherwise re-post.
Slack	Edit original message via `chat.update` (using `channel_msg_id` from `report_publications` table); attach new file via `files.upload` and reference.
Discord	Edit webhook message; attach updated file.
Teams	Post new Adaptive Card with `previousVersionRef`; old card is left in place (no edit-in-place reliable).
Email	New message with `In-Reply-To: <previous-Message-ID>` + `References` chain; full updated attachment bundle.
Diary	Append new entry referencing previous; the report file at `docs/herald/reports/<update_key>/main.md` (and siblings) is the source of truth (overwritten on each update).

35.3 Git linkage

If the report's source `.md` is committed in the consuming project's repo, Herald includes:

- `git_commit_sha` — short SHA from `git log -1 --pretty=format:%h <file>` at publish time.
- `git_commit_url` — derived from `git remote get-url origin` + the commit SHA, formatted for the host (GitHub blob URL, GitLab blob URL, GitFlic/GitVerse equivalents).
- `git_diff_url` — URL to the diff against the previous publish's commit SHA.

If the source `.md` is NOT committed yet (working-tree only), Herald posts with a warning: ⚠️ `This report has uncommitted changes — link will become live once the change is committed.`

35.4 Persistence

```
CREATE TABLE report_publications (
    id                  UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id           UUID NOT NULL,
    update_key          TEXT NOT NULL,                    --
        stable identifier for the report
    channel             TEXT NOT NULL,
    channel_address_id  UUID NOT NULL REFERENCES
        channel_addresses(id),
    channel_msg_id      TEXT,
        -- so we can edit-in-place on next update
    content_sha256      BYTEA NOT NULL,                   -- of
        the source .md
    git_commit_sha      TEXT,                             --
        nullable for uncommitted
    published_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, update_key, channel, channel_address_id)
);
ALTER TABLE report_publications ENABLE ROW LEVEL SECURITY;
ALTER TABLE report_publications FORCE ROW LEVEL SECURITY;
CREATE POLICY rp_isolation ON report_publications
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

The `(content_sha256, git_commit_sha)` pair is the dedup key: if a re-publish event arrives but the source `.md` is byte-identical AND the commit SHA is the same, Herald skips the publish (logs `digital.vasic.herald.report.noop`).

---

§36. Outbound multi-format attachments (.md + .html + .pdf + .docx)

Every report, technical-documentation page, or research material that Herald sends as a subscriber-visible attachment MUST be attached in **four formats** so subscribers can pick the one they prefer for their use case.

36.1 Format set

Default `[attachments].out_format_set = ["md", "html", "pdf", "docx"]`. Per-channel overrides allowed via channel-address query param `format_set=md,pdf` (e.g. for bandwidth-constrained ntfy push).

36.2 Generation pipeline

For each source `.md` file Herald wants to attach:

```
source.md   (canonical, source of truth)
     ↓
pandoc -f gfm -t html5 -s -o source.html
     ↓
pandoc -f gfm --pdf-engine=weasyprint -o source.pdf
     ↓
pandoc -f gfm -o source.docx
```

`docx` generation requires Pandoc (already a dependency for HTML/PDF per §11.4.65); no extra runtime needed. WeasyPrint stays the PDF engine.

36.3 Pre-generation caching

Generation is deterministic — given the same source `.md`, the outputs are byte-stable (modulo PDF embedded timestamps, mitigated via Pandoc `--reproducible` flag in V3 r1).

Cache at `<diary_root>/.herald/format-cache/<sha256-of-md>/{source.md, source.html, source.pdf, source.docx, manifest.json}`. Cache hit avoids regeneration.

36.4 Per-channel delivery rules

Channel	Delivery
Telegram	Four `sendDocument` calls in sequence (Telegram's bot API has no native "file group"); the order is `.md`, `.html`, `.pdf`, `.docx` so the most-readable preview surfaces first.
Slack	Single `files.upload_v2` with all four files referenced in one message; preview thumbnail = PDF.
Discord	Multiple file attachments on one webhook message (up to 10 per message; we use 4); embed shows PDF preview.
MS Teams	Adaptive Card with four `Action.OpenUrl` links to per-format URLs in a Herald-hosted blob (channel doesn't support multi-file attachment natively).
Lark	Multi-file upload in one message.
WhatsApp Cloud	One message per file (WA Business doesn't support multi-attachment messages).
Viber	Carousel of links.
Email	One MIME multipart message with four `application/...` parts, each `Content-Disposition: attachment` and `filename="report.<ext>"`.
ntfy	Posts the `.md` inline; the other three via `X-Attach` URLs to Herald-hosted blobs.
Gotify	Same as ntfy.
Webhook (outbound)	CloudEvents payload with four `data_base64` entries OR four URLs depending on `attachment_mode` query param.
Diary	All four written to `docs/herald/diary/attachments/<message_id>/` and referenced from the diary entry.

36.5 Format-set override per subscriber

Subscribers MAY express format preferences via `PreferenceSet.metadata.attachment_formats` (per §7.2):

```
{
  "attachment_formats": ["md", "pdf"]    // only these two; skip html + do
}
```

If a subscriber's preference excludes a format that another subscriber on the same channel-address wants, Herald sends the union (or the channel's max bundle if multi-attachment is supported). On channels that allow only one attachment per message (WhatsApp, Viber), Herald sends per-subscriber DM with that subscriber's preferred format.

36.6 Total size limits

Combined size of the four formats per attachment-bundle MUST stay under `[attachments].out_bundle_max_mib` (default 50 MiB). If a single source `.md` produces > 50 MiB of total output (rare — typically only image-heavy reports), Herald posts a single message with a link to a Herald-hosted blob carrying the full bundle.

---

§37. Tracker-doc change events (Issues / Fixed / Status / Continuation)

> **Operator mandate (2026-05-20):** any modification of constitution-defined tracker documents MUST fire CloudEvents that fan out to all configured channels for all subscribed receivers. Subscribers see project state evolve in real time, not on the next digest cycle.

37.1 In-scope tracker docs

Every modification under `<consuming-project>/docs/` of any of these files triggers a CloudEvent (the same list as Universal §11.4.60's eight bound classes):

Doc	Triggers event type
`docs/Issues.md`	`digital.vasic.herald.tracker.issues.updated`
`docs/Issues_Summary.md`	`digital.vasic.herald.tracker.issues_summary.updated`
`docs/Fixed.md`	`digital.vasic.herald.tracker.fixed.updated`
`docs/Fixed_Summary.md`	`digital.vasic.herald.tracker.fixed_summary.updated`
`docs/Status.md`	`digital.vasic.herald.tracker.status.updated`
`docs/Status_Summary.md`	`digital.vasic.herald.tracker.status_summary.updated`
`docs/CONTINUATION.md`	`digital.vasic.herald.tracker.continuation.updated`
`docs/Reopens/<HRD-NNN>.md`	`digital.vasic.herald.tracker.reopens.updated`

Plus the high-fidelity item-level events that compose with §4.2:

Item-level event	Fired by
`digital.vasic.herald.project.task.opened`	Row added under `Issues.md` Open section
`digital.vasic.herald.project.task.in_progress`	Row's `Status` column transitions to `in_progress`
`digital.vasic.herald.project.task.blocked`	Row's `Status` transitions to `blocked` or `operator-blocked`
`digital.vasic.herald.project.task.closed`	Row migrates atomically from `Issues.md` to `Fixed.md`
`digital.vasic.herald.project.task.reopened`	Row migrates from `Fixed.md` back to `Issues.md` (per Universal §11.4.55)
`digital.vasic.herald.project.task.reclassified`	Type or criticality column changes in place

37.2 Detection mechanism

`<flavor>herald serve` runs an fsnotify-watched debounced (500 ms idle, 2 s ceiling) tracker-doc watcher over `<consuming-project>/docs/`. On every observed change:

1. Hash the file's new content (SHA-256).
2. Compare to `tracker_state.content_sha256` (a per-doc row in the new `tracker_state` table — schema below).
3. If new SHA differs, parse the file (Markdown table → row diff) to detect which rows changed; emit one **doc-level** event (always) plus one **item-level** event per row delta (when the diff is parseable).
4. Persist new SHA + `last_emitted_at`.

```
-- §37.2 tracker_state (added to migrations as a future
        000006_tracker_state.up.sql)
CREATE TABLE IF NOT EXISTS tracker_state (
    tenant_id        UUID NOT NULL,
    doc_path         TEXT NOT NULL,
    content_sha256   BYTEA NOT NULL,
    last_event_id    UUID,
    last_emitted_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, doc_path)
);
ALTER TABLE tracker_state ENABLE ROW LEVEL SECURITY;
ALTER TABLE tracker_state FORCE ROW LEVEL SECURITY;
CREATE POLICY ts_isolation ON tracker_state
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

37.3 Fan-out

Each tracker-doc event is routed via the standard §6 channel-addresses + tag-fan-out matching pipeline. By default tracker events carry the tag `tracker` so operators can scope subscribers per category:

- `tracker:issues` — subscribers who want every Issues.md change (verbose).
- `tracker:fixed` — closure announcements only.
- `tracker:status` — high-level state changes.
- `tracker:incidents` — Issues/Fixed entries with `Type=incident` only.

Subscribers opt in via `PreferenceSet.categories.tracker_*` (spec §7.2). The default for new tenants is `tracker:fixed` enabled, others muted — so subscribers aren't flooded by every in-progress edit.

37.4 Composition

- **§11.4.60 composite always-sync** — §37 is the *event-emission* layer; §11.4.60 ensures the `.md`/`.html`/`.pdf` siblings stay in sync. They're orthogonal but complementary: §11.4.60 prevents stale renders, §37 prevents stale subscriber awareness.
- **§35 versioned reports** — `.report.*` events are a *subset* of §37: any report doc that also lives in a tracker file gets BOTH a §37 tracker event AND a §35 report event (with shared `update_key`).
- **§32 inbound pipeline** — §37 events are pure-outbound; they don't go through the inbound worker pipeline.

---

§38. Workable-item announcement contract

> **Operator mandate (2026-05-20):** when a new workable item is added, send an announcement to all subscribers with: title, short description, and attachments — inline when small enough for the channel, otherwise via direct URL to the Git host's permalink. Whenever channel-side size constraints make the inline attachment infeasible, fall back to a URL pointing at the canonical attachment under `issues/users/attachments/<HRD-NNN>/<filename>` on the project's Git remote (GitHub / GitLab / GitFlic / GitVerse).

38.1 Announcement structure

Every `digital.vasic.herald.project.task.opened` event triggers a per-subscriber announcement message containing:

```
[<flavor icon>] <criticality badge> <Type>: <HRD-NNN>  —  <Title (≤ 100 ch

<Short description (≤ 280 chars, single paragraph)>

📎 Attachments: <inline summary OR list of permalinks>
🔗 Permalink: <Git host URL of the Issues.md row's anchor>
👤 Opened by: <subscriber.handle> on <channel>
🕒 <RFC 3339 timestamp>
```

The `<flavor icon>` is the flavor's emoji (📋 pherald, ⍰ sherald, 🔨 bherald, 🚀 dherald, 🔔 aherald, ⍰ scherald, 🚨 iherald, ⍰ rherald, 📜 cherald).

The `<criticality badge>` is colored / emoji-coded: 🚨 critical, 🟠 high, 🟡 middle, 🟢 low.

38.2 Attachment size policy

For each attachment associated with the new workable item, Herald computes per-channel inline-feasibility:

Channel	Inline-feasible if attachment ≤	Fallback
Telegram	50 MiB (Bot API limit per file)	Permalink
Slack	1 GiB (workspace plan-dependent; default soft-cap 100 MiB)	Permalink
Discord	25 MiB (free tier) / 100 MiB (Nitro) — Herald uses 25 MiB	Permalink
MS Teams	10 MiB (webhook restriction)	Permalink
Lark	30 MiB	Permalink
WhatsApp Cloud	16 MiB (image) / 100 MiB (document)	Permalink
Viber	200 MiB	Permalink
Email (SMTP)	`[attachments].email_inline_max_mib` (default 10 MiB)	Permalink
ntfy	`X-Attach` URL only — always permalink	Permalink
Gotify	extras link only — always permalink	Permalink
Webhook outbound	per-receiver — defaults to permalink	Permalink
Diary	always inline (filesystem-local)	n/a

Permalink construction. Herald reads the project's `git remote get-url origin` and derives the host's permalink format:

Host	URL pattern
GitHub	`https://github.com/<org>/<repo>/blob/<sha>/issues/users/attachments/<HRD-NNN>/<filename>`
GitLab	`https://gitlab.com/<org>/<repo>/-/blob/<sha>/issues/users/attachments/<HRD-NNN>/<filename>`
GitFlic	`https://gitflic.ru/project/<org>/<repo>/blob/<sha>/issues/users/attachments/<HRD-NNN>/<filename>`
GitVerse	`https://gitverse.ru/<org>/<repo>/content/<sha>/issues/users/attachments/<HRD-NNN>/<filename>`

The `<sha>` is captured at announcement time so the URL remains valid even after subsequent commits.

Uncommitted-attachments fallback. If the attachment hasn't been committed yet (working-tree only), the announcement includes the inline file PLUS a notice: `⚠️ Attachment not yet committed — permalink will become live once the next commit lands.` Subscribers can still receive the inline payload; the warning is for traceability.

38.3 Multi-format bundle composition

Per §36, every outbound Herald-produced attachment ships as a four-format bundle (`.md` + `.html` + `.pdf` + `.docx`). For *subscriber-uploaded* attachments (the ones at `issues/users/attachments/<HRD-NNN>/`), the multi-format expansion is **NOT** applied — those files are sent as-is in their original format (per §18.2.4). Only Herald-generated documents (investigation summaries, reports, weekly digests) get the multi-format bundle.

38.4 Status-transition follow-up announcements

Every subsequent transition (`task.in_progress` / `task.blocked` / `task.closed` / `task.reopened` / `task.reclassified`) fires a smaller follow-up announcement in the same thread (per §12 `ConversationRef`):

```
[<flavor icon>] HRD-NNN  →  <new status>  <transition reason / short note>
🔗 <permalink>
```

Subscribers MAY mute these via `PreferenceSet.workflows.….transitions = { muted: true }`.

---

§39. Message presentation + template standards

> **Operator mandate (2026-05-20):** messages sent to subscribers MUST be properly designed, nicely and clearly written, organized — AND adhere to **standardized templates** with strict rules.

39.1 Why standardize

Herald sends thousands of messages a day at scale. Even tiny stylistic drift between channels (different emoji choices, different field-order, different "permalink" wording) creates cognitive friction for subscribers who scan messages in seconds. §39 fixes the templates so every flavor + channel combination renders predictably; operators can tune *content* per tenant but the *shape* is constitution-level.

39.2 The Herald canonical template (HCT)

Every outbound subscriber-facing message is composed of FIVE blocks, in this order:

```
[1] HEADER       — flavor icon + criticality badge + type + ID + title (on
[2] LEAD         — short paragraph (≤ 280 chars). One thought; one CTA imp
[3] DETAILS      — optional. Key:value pairs, bullet lists, code blocks (w
[4] ATTACHMENTS  — optional. Inline thumbnails OR permalinks (per §38.2).
[5] FOOTER       — opened-by + timestamp + permalink + `<flavor>herald <ve
```

When the channel supports rich messaging (Slack Block Kit, Discord embeds, Adaptive Cards), each block maps to a native rich-message element; on plain channels (SMS, ntfy text-only), blocks degrade to delimited plain text.

39.3 Standardized template files

All templates live under `commons_messaging/templates/<event_type>/<channel>.tmpl` (per §13). The directory MUST follow this structure:

```
commons_messaging/templates/
├── _shared/
│   ├── header.tmpl                 # HCT block 1
│   ├── lead.tmpl                   # HCT block 2
```

```
│   ├── details.tmpl                 # HCT block 3
│   ├── attachments.tmpl             # HCT block 4
│   └── footer.tmpl                  # HCT block 5
├── digital.vasic.herald.project.task.opened/
│   ├── tgram.md.tmpl
│   ├── slack.json.tmpl              # Block Kit JSON
│   ├── discord.json.tmpl            # Embed JSON
│   ├── teams.json.tmpl              # Adaptive Card v1.5
│   ├── email.mjml                   # MJML source
│   ├── email.html                   # compiled MJML (committed alongside
│   ├── ntfy.txt.tmpl
│   ├── diary.md.tmpl
│   └── meta.toml                    # per-template metadata: i18n keys, h
└── digital.vasic.herald.tracker.issues.updated/
    └── …
```

39.4 Mandatory template properties

Each `meta.toml` declares (and Herald's CI gate `CM-TEMPLATE-CONFORMANCE`, planned, enforces):

Property	Required	Notes
`event_type`	yes	matches the parent directory
`channels`	yes	array of channels the template directory covers — gate flags missing channels
`i18n_keys`	yes	list of locale keys this template references; missing translations FAIL the gate
`header_max_chars`	yes	hard cap on HCT block 1 — default 100; SMS-style channels override to 70
`lead_max_chars`	yes	hard cap on HCT block 2 — default 280
`body_max_chars`	yes	hard cap on full rendered body before truncation
`attachment_policy`	yes	`inline
`helpers`	no	additional Sprig / custom helpers the template needs

39.5 Style rules (constitution-level)

Templates MUST follow these style rules (`CM-TEMPLATE-STYLE` gate, planned):

1. **One emoji per block** — header may have flavor-icon + criticality-badge; otherwise emojis are atomic.
2. **Sentence case** — no SCREAMING headlines except for 🚨 `CRITICAL` badges.
3. **Active voice** in the lead — "Build failed" not "A build has been failed by".
4. **Past-tense in event-confirmation messages, present-tense in status-update messages** — keeps tense aligned with the event semantics.
5. **No clipping mid-word** — body truncation always happens at a clause boundary; ellipsis (…) marks the cut.
6. **i18n-safe** — no string concatenation; only `{{tr "key" .Param}}` calls.
7. **No raw user input in HTML / shell context** — template MUST use the auto-escaping `html/template` or `text/template` per channel (spec §15.3).
8. **Permalink wording is always "🔗 Permalink:"** — never "Link:", "URL:", "See:", etc.

9. **Timestamps are RFC 3339 with explicit timezone** — never relative ("5 minutes ago") because diary entries lose context over time.
10. **Universal `<flavor>herald <version>` attribution in the footer** — operators can suppress for white-label deployments via `[branding].suppress_footer=true`.

39.6 Per-channel rendering tiers

Tier	Channels	HCT mapping
Rich (interactive)	Slack, Discord, Telegram, Teams, Lark, WhatsApp Cloud, Viber	Header → emphasised header element + criticality color; Lead → top text; Details → fields/blocks; Attachments → file attachments + previews; Footer → context block.
Rich (read-only)	Email (HTML), Gotify, ntfy with X-Markdown	Same blocks, no interactive components.
Plain	Email (plain), SMS-like channels, generic webhook with `format=text`	Blocks delimited by `\n\n`; emojis preserved; no rich formatting.
Structured	Webhook with `format=cloudevent`	Blocks serialised as JSON fields in the CloudEvent `data` payload.
Source	Diary	Markdown source — no transformation.

39.7 Template-evolution discipline

Changes to templates follow the §23 spec-change rule when they alter the HCT block layout. Pure copy edits (typos, i18n additions, color hex tweaks) DO NOT trigger the spec-change rule — they're catalogued in `docs/changelogs/templates/` (per Universal §11.4.18 script-companion-doc parallel) and audited by the template-conformance gate.

---

§40. Documentation + testing completeness mandate

> **Operator mandate (2026-05-20):** everything MUST be fully documented and covered with all possible test types and challenges.

40.1 Documentation completeness

Every module, package, type, and exported function MUST carry documentation. The CI gate `CM-DOC-COVERAGE` (planned) enforces:

Surface	Required documentation
Every `commons*` package	Package-level `// Package <name>` doc.go OR top-of-file comment naming the spec section it implements.
Every exported type	One-paragraph godoc that names purpose + invariants + spec section.
Every exported function/ method	godoc with parameters, return semantics, error conditions, spec section.
Every SQL migration	Header comment with spec section reference + rationale + irreversibility notes (if any).
Every CLI subcommand	`Short`, `Long`, per-flag `Description`; `--help` output is the source of truth for operators.

Surface	Required documentation
Every channel adapter	`docs/channels/<channel>/setup.md` with credential acquisition steps, webhook setup, troubleshooting.
Every flavor	`docs/flavors/<flavor>/<flavor>.md` with use cases, command palette, channel-interaction matrix.
Every Universal Constitution composition	The package/function MUST comment which Universal §X.Y mandate it implements.
Every channel adapter test fixture	`testdata/README.md` documenting recording conditions + redaction policy.

`CM-DOC-COVERAGE` is run by `go vet ./...` extension + a custom golangci-lint analyzer.

40.2 Test-type matrix

Herald MUST ship eight test tiers (per Universal §11.4.27 no-fakes-beyond-unit-tests + §11.4.39 per-feature on-device end-user validation):

Tier	Build tag	Required for	What it covers
Unit	(none)	every PR	pure-logic tests, no external services, in-memory mocks only of `commons` interfaces
Component	`component`	every PR	one module's exported surface against its own real dependencies (e.g. `commons_storage` against a real Postgres started via testcontainers-go)
Integration	`integration`	every PR (CI-only)	cross-module flows: `pherald send` → router → null:// adapter against real Postgres + Redis + River
Contract	`contract`	every PR	adapter tests against recorded HTTP fixtures (go-vcr cassettes) — proves the SDK invocation is byte-stable
End-to-end (sandbox)	`e2e_sandbox`	nightly CI	per-channel sandboxes: Telegram test bot, Slack test workspace, Email IMAP IDLE against a test mailbox
End-to-end (live)	`e2e_live`	release-gated	live bot tokens in a dedicated test tenant; runs against the published image
Mutation	`mutation`	every release	Universal §1.1 paired-mutation tests for every gate — anti-bluff invariant
Chaos	`chaos`	nightly	null:// `fail_rate` + latency-injection + worker-pool starvation + Postgres connection-storm

40.3 Test challenges (per Universal §11.4.39 + spec §40)

Beyond test tiers, Herald MUST ship **named challenges** — scenarios that exercise the boundaries of correctness:

- **C-01. 30-second poll cadence enforcement** — slow upstream means `getUpdates` long-poll occasionally stalls; the safety-net timer (§32.2) MUST fire `getUpdates` exactly at the 30 s mark. Test inserts an artificial stall + asserts second invocation.
- **C-02. FIFO under contention** — 100 inbound messages from 10 senders on one channel; assert per-sender processing order is preserved.

- **C-03. Anti-spam thresholds** — burst of 100 similar messages from one sender; assert reputation drops + quarantine triggers after threshold; assert legitimate senders unaffected.
- **C-04. Investigation-before-Fixing rejection path** — submit a duplicate `Bug:` for an already-fixed issue; assert LLM rejects + no new workable item; assert subscriber reply explains reason.
- **C-05. Attachment-size threshold** — submit attachment exactly at the channel limit, +1 byte, +1 MiB; assert correct inline vs permalink fallback.
- **C-06. Multi-format bundle determinism** — generate same source `.md` twice with `--reproducible`; assert byte-identical outputs.
- **C-07. RLS bypass attempt** — assert that omitting `SET LOCAL app.tenant_id` in a transaction yields zero rows on every multi-tenant table (fails closed).
- **C-08. Graceful shutdown drain** — fire 50 in-flight `Sends`, send SIGTERM, assert all 50 complete within `HERALD_SHUTDOWN_GRACE`; second SIGTERM forces immediate exit.
- **C-09. Tracker-doc fsnotify debounce** — write to `Issues.md` 12 times within 100 ms; assert exactly ONE …`tracker.issues.updated` event fires, not 12.
- **C-10. Idempotency replay** — POST the same CloudEvent twice with same idempotency key; assert second POST returns the cached response, NOT a duplicate fan-out.
- **C-11. Cross-channel reply threading** — Slack thread reply → Telegram-mirror message; assert both link back to the same `logical_thread_id`.
- **C-12. Channel-credential rotation mid-flight** — rotate Telegram bot token during a 30 s send burst; assert pending sends complete with old token, new sends use new token, no message loss.
- **C-13. Template style-rule violation** — submit a template containing emoji in two HCT blocks; assert `CM-TEMPLATE-STYLE` gate FAILs.
- **C-14. Investigation LLM timeout** — Claude Code session hangs > 5 min; assert Herald cancels, posts `failed` reply with reason, opens HRD-INV row as `operator-blocked`.
- **C-15. Disaster-recovery cold-start** — destroy Postgres + Redis containers; restore from latest backup; assert dead-letters table contents survive; assert in-flight messages replayed without duplicates.

40.4 Documentation MUST-include per file class

Each file class carries a documentation responsibility:

File	Doc must include
`*.go`	godoc per §40.1 + spec section reference in package comment
`*.sql`	header with migration number, spec section, rationale, idempotency notes
`Dockerfile*`	layer-by-layer rationale + build / runtime separation explained
`docker-compose*.yml`	bring-up instructions + assumed env vars + verification commands
`*.tmpl`	i18n-key inventory + max-length asserts + HCT block boundaries marked with comments
`tests/*.sh`	mandatory header explaining: what the gate checks, what the paired mutation does, exit code semantics
`*.md`	Universal §11.4.61 metadata table + ToC (when ≥ 2 H2s) + Universal §11.4.65 export siblings

40.5 Composition

§40 doesn't replace existing testing rules; it expands them:

- Universal §11.4.27 no-fakes-beyond-unit-tests is INHERITED — V3 doesn't loosen it.
- Universal §11.4.39 per-feature end-user validation MAPS to tier `e2e_live` here.
- Universal §11.4.2 captured-evidence applies to every tier ≥ component (recorded outputs MUST be attached to CI artefacts).
- Universal §1.1 paired-mutation tests are the `mutation` tier.

The §40 mandate is intentionally aspirational — V3 r1 ships the spec; V3 implementation lands the test tiers progressively per the HRD-008..HRD-015 first-implementation cycle. The `CM-DOC-COVERAGE` and `CM-TEMPLATE-CONFORMANCE` gates land alongside the modules they cover.

---

§41. REST API surface (Gin Gonic)

> **Operator mandate (2026-05-20):** every Herald flavor — where it makes sense for that flavor — MUST expose a REST API for triggering commands that operators would otherwise issue via the CLI. The REST surface lets applications (web UIs, mobile apps, third-party orchestrators) integrate with Herald without spawning subprocess invocations. The standard framework is **Gin Gonic** (`github.com/gin-gonic/gin`).

41.1 Which flavors expose REST

Flavor	REST?	Rationale
`pherald` (Project Herald)	✓ yes	Highest integration surface — IDE plugins, web dashboards, AI agents call REST to open investigations, fetch workable items, post replies.
`sherald` (System Herald)	✓ yes	Monitoring stacks (Grafana / Datadog / OpsGenie integrations) push events via REST instead of synthetic webhooks.
`bherald` (Build Herald)	✓ yes	CI tools without native webhook support call REST.
`dherald` (Deploy Herald)	✓ yes	Deploy orchestrators trigger rollback / hold / promote via REST.
`aherald` (Alert Herald)	✓ yes	Alert routers post events; subscribers (or front-ends) query open-alerts state.
`scherald` (Schedule Herald)	✓ yes	Reminder UIs create / snooze / cancel reminders.
`iherald` (Incident Herald)	✓ yes	Incident-management UIs page on-call, take IC, post updates.
`rherald` (Release Herald)	✓ yes	Release dashboards.
`cherald` (Compliance Herald)	✓ yes	Audit dashboards + IR tools.
Future flavors	per-flavor	Decided when each flavor is specified.

Every yes-flavor exposes `/v1/...` HTTP routes through Gin. **Flavors MAY opt out** of REST only when no realistic app/integration consumer exists — operators MUST justify the opt-out in the flavor's manual.

41.2 Framework: Gin Gonic

Why Gin — actively maintained (~80k stars), httprouter under the hood (so latency is ~5× lower than net/http defaults for high-fanout APIs), middleware ecosystem already covers OTel + CORS + rate-limit + recovery, idiomatic for Go developers reading Herald source. Alternatives (chi, echo, fiber) were considered; Gin wins on operator familiarity and the existing OTel + Prometheus middleware quality.

Dependency: `github.com/gin-gonic/gin` pinned via each flavor's `go.mod`. The Gin handler hand-off into `commons_messaging.Router` keeps adapter code Gin-agnostic — only the flavor's `internal/http/` wires Gin into the router.

41.3 Endpoint surface (common across all REST-enabled flavors)

Mounted under `/v1/` per spec §5.7 (REST is the *same* ingress port — `[server].http_port`, default 24091; the HTTP surface IS the REST API plus webhooks).

Method	Path	Behaviour
`POST`	`/v1/events`	CloudEvents v1.0 ingest (binary + structured) — same as §5.7.
`POST`	`/v1/send`	Convenience REST ingest (non-CloudEvents JSON body, wrapped server-side).
`GET`	`/v1/events/{id}`	Idempotent replay; 200 with cached response, 409 on body-mismatch.
`GET`	`/v1/items`	List workable items (paginated, filterable by `status`, `type`, `criticality`).
`GET`	`/v1/items/{id}`	Single workable item with full Markdown row + linked investigation + attachments.
`POST`	`/v1/items`	Open a new workable item (operator role required — alternative to `Bug:`/`Issue:` subscriber commands).
`PATCH`	`/v1/items/{id}`	Update status / type / criticality.
`POST`	`/v1/items/{id}/close`	Migrate row from `Issues.md` → `Fixed.md` (per §8.3).
`POST`	`/v1/items/{id}/reopen`	Reverse migration (per Universal §11.4.55).
`POST`	`/v1/items/{id}/attachments`	Upload an attachment (multipart) — Herald validates per §32.4 + stores at `issues/users/attachments/<HRD-NNN>/`.
`GET`	`/v1/subscribers` / `POST /v1/subscribers`	Subscriber CRUD.
`GET`	`/v1/channels`	List configured channel addresses (filtered by `enabled`/`tag`).
`GET`	`/v1/deadletters` / `POST /v1/deadletters/{id}/replay`	Dead-letter management.
`GET`	`/v1/status`	Same payload as `docs/Status_Summary.md` machine-readable JSON.

Method	Path	Behaviour
GET	/v1/healthz	Public-facing health (composite of /livez + /readyz).
Per-flavor	/v1/<flavor>/...	Flavor-specific endpoints (e.g. /v1/incident/{id}/take-ic on iherald; /v1/build/{id}/retry on bherald).

Wave 3b (r9) live-route flip — POST /v1/events flips from 501-stub to live 202 Accepted + Receipt JSON:

Method	Path	Owning flavor	Behaviour	HRD
POST	/v1/events	pherald	202 Accepted + Receipt JSON (CloudEvent ingestion through the §32 7-stage Runner pipeline). 200 + X-Herald-Replay: true on idempotency replay. 401 on missing/invalid JWT. 400 on parse failure. See §32.1.1 implementation map.	HRD-016 closed (Wave 3b)

Wave 2 (r8) per-flavor route additions — landed as 501-stub routes by Wave 2 (scaffold only; live wiring lands in Wave 3 under the cited HRDs):

Method	Path	Owning flavor	Behaviour	HRD
GET	/v1/compliance	cherald	Constitution-state pull surface. Returns the current constitution_state row per tenant (rule_id, severity_category, decision_result, transition_from/to, bundle_hash, last_decided_at). Composes with §42.7 + §44.6.	HRD-028
GET	/v1/safety_state	sherald	Daemon status surface: open .host.safety_breach events, current memory% (per §12.6 budget), last destructive-op log entry (per §9.1). Used by sherald serve daemon-mode.	HRD-098
POST	/v1/webhooks/page	iherald	PagerDuty/Opsgenie-compatible inbound webhook for paging events. Translates into the §42 digital.vasic.herald.alert.firing event class + fans out per §11 channel selection.	HRD-024

41.4 Authentication

Reuses the §5.7 auth model:

- HTTP Basic with per-tenant ingest_token (for simple integrations).
- JWT bearer (Authorization: Bearer …) validated against JWKS in [auth.oidc] (for OAuth-integrated apps).
- mTLS via reverse proxy (for high-security deployments).

REST endpoints requiring operator role check claims for roles: ["operator"] in the JWT (or look up the API key's owning subscriber's subscribers.roles).

41.5 Versioning

Same as §5.7 — /v1/ is the stable contract; breaking changes ship as /v2/ with ≥ 6 months of /v1/ co-existence. OpenAPI spec at docs/api/openapi.v1.yaml is the source of truth; CI gate CM-OPENAPI-DRIFT enforces handler ? spec parity per §24.1.

41.6 Containerization mandate (composes with §9.5)

Reinforced operator mandate (2026-05-20): every flavor binary MUST be distributed as a container image built via the containers Submodule (per V3 §9.5). The submodule provides:

- Per-flavor Dockerfile (multi-stage, distroless or alpine runtime).
- Compose / Kubernetes / Nomad manifests at the per-flavor reference deployment.
- Reproducible-build flags (CGO_ENABLED=0, GOFLAGS=-trimpath, deterministic -ldflags).
- SBOM + cosign signatures per V3 §21.

Operators MAY use quickstart/Dockerfile.pherald (shipped in this commit at HRD-008) as a *local development bridge* until the containers Submodule is added; once the submodule lands, the quickstart files migrate there and Herald's containers/ directory becomes a thin pointer.

41.7 OpenAPI generation

Per §24.1, docs/api/openapi.v1.yaml is the source-of-truth schema. Gin-based REST handlers MUST be tagged with openapi:"<operation_id>" comments (custom golangci-lint analyzer herald-openapi-tags will enforce); the spec generator scans tags + handler signatures to keep YAML and code in lockstep. Drift triggers CM-OPENAPI-DRIFT gate FAIL.

41.7.1 Content negotiation (TOON primary, JSON fallback — Wave 4b live)

Every /v1/* business route honors the Accept header per the W4b policy ladder (see §44.P for as-built evidence + the full operator decision rationale):

- Accept: application/toon → response is real TOON-encoded bytes (via digital.vasic.toon + toon-format/toon-go upstream). Content-Type: application/toon.
- Accept: application/json → JSON response. Content-Type: application/json.
- Accept: */* (or missing Accept) → server default. Herald default = TOON; HERALD_DEFAULT_RESPONSE_CODEC=json flips to JSON.
- Both with no q params → TOON wins (Herald-default tie-break).
- Both with q params → higher q wins.
- Unsupported Accept (e.g., application/xml) → JSON honest fallback. Content-Type: application/json — the bytes match what we advertise.

POST /v1/events additionally honors the inbound Content-Type header to decide how to decode the request body. TOON request bodies are transcoded to JSON before the Runner's EventParser (the Runner stays codec-agnostic — codec adaptation is a transport-layer concern). Encoder-failure fallback is observable: X-Herald-Codec-Fallback: encoder_error header is emitted when toon.Marshal fails, with the response falling back to JSON. Never a silent swap.

`/healthz`, `/readyz`, `/metrics` are NOT subject to content negotiation — they're registered BEFORE the TOON middleware in `commons/cli/serve.go` and remain in their canonical formats (JSON for healthz/readyz; Prometheus text exposition for metrics) regardless of Accept header. This is intentional: load-balancer probes + Prometheus scrapers don't negotiate, and forcing them to would create paper-cut bluffs.

The negotiation primitive lives in `commons/cli/contentnego.go` (pure functions over header strings + `MarshalChosen` / `UnmarshalChosen` codec dispatch). The Gin glue lives in `commons/cli/toon.go` (`TOONMiddleware` + `BindNegotiated`). Both are auto-wired into `cli.RunServe`, so every flavor binary inherits without per-flavor code changes.

§107 anti-bluff guards: TOON wire bytes never start with `{` or `[` (caught by E60 + the M2 mutation gate that plants the EXACT shape of the 2026-05-17 PASS-bluff). TOON bytes round-trip via `toon.Unmarshal` to non-empty Go maps (E58). TOON body is strictly smaller than JSON for the same payload (E59 — real compression, not header-swap bluff).

41.8 Per-app integration patterns

Two recommended integration patterns documented in `docs/integrations/`:

- **Embedded SDK** — apps in Go vendor the relevant Herald client (planned: `clients/go/`); other languages call REST directly. SDK generators from the OpenAPI spec are documented (TypeScript via openapi-typescript, Python via openapi-python-client).
- **Reverse-proxy fan-out** — apps mount Herald's REST under their own `/api/herald/*` namespace, applying app-side auth before forwarding.

`pherald` r1 shipped REST scaffolding under `pherald/internal/http/`; Wave 3b (V3 r9) lands `POST /v1/events` live — see §32.1.1 implementation map + §44.N Wave 3b milestone.

---

§42. Constitution-flavor binding catalogue

> **Operator mandate (2026-05-20):** every Helix Universal Constitution rule that produces a *runtime-observable state change* in a governed project MUST be bound to the natural Herald flavor that owns it — so subscribers learn about violations, transitions, and compliance state through the same channels they already use for project events. Process-only rules (agent self-discipline like §11.4.6 no-guessing, §11.4.20 subagent-by-default) are deliberately excluded; binding them adds noise without value.

§42 is the **single canonical mapping** from constitution clauses to Herald event types + owning flavors. It composes with §37 (tracker-doc events), §32 (inbound pipeline), and §34 (reply protocol).

42.1 Binding architecture (event envelope, mode ladder, replayability)

Synthesised from a deep survey of how real governance systems emit events (OPA Gatekeeper, OPA Decision Logs, Kyverno PolicyReports, AWS CloudTrail + Config Rules, GCP Cloud Audit Logs, GitHub branch-protection, NIST SSDF, SLSA VSA, Sigstore policy-controller, Anthropic Constitutional Classifiers). Five design rules are universal across all of them:

§42.1.1 Three-axis envelope. Every constitution-rule event carries the triple `(rule_id, severity_category, decision_result)`. These are the three minimum required CloudEvent extension attributes on `digital.vasic.herald.constitution.*` events:

Extension attribute	Example	Notes
`heraldconstitutionrule`	`§11.4.10`	Stable rule identifier; matches the §X.Y reference in Constitution.md.
`heraldconstitutionseverity`	`critical/high/ middle/low`	Per spec §18.2.2 criticality vocabulary.
`heraldconstitutiondecision`	`pass/warn/fail/ error/skip`	Kyverno-aligned (`scored=false` ⇒ warn).

The CloudEvents `type` is the **event class**, not the rule ID — e.g. `digital.vasic.herald.constitution.gate.failed` carries `heraldconstitutionrule=§11.4.61` as an attribute, NOT in the `type` itself. This keeps the event-type tree finite (~12 leaf classes) and lets routers fan out by rule via filter expressions without exploding subject namespaces.

§42.1.2 Transitions, not states. Herald MUST emit a constitution-rule event ONLY on state *change* — `pass → fail`, `fail → pass`, `unknown → fail`. Continuous "still compliant" pings are forbidden; they drown the audit channel and train operators to mute it. This mirrors AWS Config's `Config Rules Compliance Change` event (which carries both `newEvaluationResult` and `previousEvaluationResult`) and Kyverno's `summary` transitions. Implementation: every rule-evaluation result is hashed and stored in a Postgres `constitution_state` table; events fire only when the hash changes.

```
-- §42.1.2 constitution_state (planned
        000006_constitution_state.up.sql).
CREATE TABLE IF NOT EXISTS constitution_state (
    tenant_id        UUID NOT NULL,
    rule_id          TEXT NOT NULL,                    --
        "§11.4.10", "§12.1", ...
    subject          TEXT NOT NULL,                    -- file
        path / repo / branch / commit SHA / tenant scope
    last_decision    TEXT NOT NULL,                    --
        'pass'|'warn'|'fail'|'error'|'skip'
    last_evaluated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_event_id    UUID,
    PRIMARY KEY (tenant_id, rule_id, subject)
);
ALTER TABLE constitution_state ENABLE ROW LEVEL SECURITY;
ALTER TABLE constitution_state FORCE ROW LEVEL SECURITY;
CREATE POLICY cs_isolation ON constitution_state
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

§42.1.3 Constitution-bundle hash (replayability). Every constitution-rule event MUST carry the `heraldconstitutionbundlesha` extension — the SHA-256 of the rendered `Constitution.md` at the time of evaluation. This is the OPA-Decision-Logs `bundles` field and the SLSA-VSA `policy.digest`: without it, "rule §X said deny at T" is unreplayable a year later when the constitution has moved on. The bundle hash is captured at *evaluation* time, not *event-emission* time, so even if Herald itself fails over mid-stream the receiver can still verify which constitution revision produced the verdict.

§42.1.4 Mode ladder (allow / warn / enforce). Every binding declares a `mode` per tenant:

- `allow` — evaluator runs, decision is computed, NO event fires. Used during initial rollout to gather baseline data.
- `warn` — events fire on `fail`/`warn` outcomes but routed to non-paging channels (digest / diary only; never SMS / page-out).
- `enforce` — events fire to all configured channels per §6 tag-fan-out; high-severity failures may page on-call.

The ladder mirrors Gatekeeper's `enforcementAction` field, Sigstore policy-controller's modes, and Anthropic's auto-mode safety thresholds. Operators MUST be able to roll a new binding through `allow → warn → enforce` without redeploying. Storage: per-binding row in `constitution_bindings` table (planned 000007_constitution_bindings.up.sql).

§42.1.5 Push + pull surfaces. Per the research (Gatekeeper CR `status` is pull; Gatekeeper log line is push; GitHub PR comment is push; GitHub branch-protection settings query is pull), Herald maintains BOTH:

- **Push** — CloudEvents on `digital.vasic.herald.constitution.*` topic fanned out to subscribed channels per the standard §6 + §32 pipeline.
- **Pull** — REST endpoint per §41 mounted at `/v1/compliance` on every REST-enabled flavor. Returns the current `constitution_state` rows for the calling tenant + pagination + filter (`?rule=§11.4.10`, `?subject=docs/Issues.md`, `?decision=fail`).

The pull surface lets auditors snapshot compliance posture without subscribing to fan-out; the push surface lets engineers learn about regressions in their channel of choice.

42.2 Canonical event-class taxonomy

CloudEvents `type` is one of these twelve leaf classes under `digital.vasic.herald.constitution.*`:

Event `type`	When emitted	Owning flavor (default)
`.gate.failed`	a CI/pre-commit gate keyed on a constitution rule FAILs (e.g. `CM-DOC-REVISION-HEADER-PRESENT`)	`bherald` (CI) or `cherald` (deep audit)
`.gate.recovered`	the same gate transitions back to PASS	same as `.gate.failed`
`.policy.violation`	a runtime evaluator detects a violation (e.g. naming rule §11.4.29, file-layout §11.4.11)	`cherald`
`.policy.cleared`	a previously-violating subject now passes	`cherald`
`.host.safety_breach`	a §12 host-safety rule violated (forbidden op attempted, mem budget exceeded)	`sherald`
`.repo.safety_breach`		

Event `type`	When emitted	Owning flavor (default)
	a §9 codebase-safety rule violated (force-push without authorization, missing backup)	`sherald` / `pherald` (both subscribe)
`.credential.leak`	a §11.4.10 / §11.4.10.A credential-handling violation	`cherald` + `iherald` (paged)
`.bundle.updated`	the constitution itself moves to a new SHA (post-pull validation §11.4.32 passed)	`sherald`
`.bundle.update_failed`	post-pull validation FAILed	`sherald` + `iherald` (paged)
`.release.gate_blocked`	a §11.4.40-class release gate refused to let the tag land	`rherald`
`.spec.revision_drift`	§11.4.73 spec-vs-Revision drift detected	`cherald`
`.catalogue.miss`	§11.4.74 catalogue-check missing from a PR	`cherald`

Adapter implementation notes: each event carries the standard three-axis envelope (§42.1.1) plus the bundle hash (§42.1.3) plus the OTel trace context (`heraldtraceparent` / `heraldtracestate`).

42.3 Master binding table (constitution rule → flavor)

Rules NOT listed are intentionally excluded from runtime binding — they're process-only / agent-discipline rules that have no observable target-system event surface. Excluded rules: §11.4.6 (no-guessing), §11.4.8 (deep-web-research), §11.4.20 + §11.4.70 (subagent-by-default), §11.4.35 (canonical-root clarity), §11.4.54 (ATM-NNN — ATMOSphere-specific), §11.4.58 (parallel-development methodology), §11.4.63 (workable-items procedure), §11.4.72 (audio top-priority — operator-scheduling rule), §8 (aspirational), §10 (enforcement meta), §11 (anchor only).

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
§1 Test coverage mandatory	`.gate.failed`	`bherald`	enforce	high	coverage drop or zero-test PR
§1.1 False-positive immunity (paired mutation)	`.gate.failed`	`bherald` + `cherald`	enforce	critical	meta-test fails to detect mutation
§2 Commit + push mechanics	`.repo.safety_breach`	`pherald` + `sherald`	enforce	high	wrong entrypoint OR partial fan-out
§3 Submodule changes propagate first	`.repo.safety_breach`	`pherald`	enforce	high	parent commit before submodule commit
§4 Tag mirroring	`.release.gate_blocked`	`rherald`	enforce	high	tag missing on owned submodule
§5 Changelog + multi-format export	`.policy.violation`	`rherald` + `cherald`	warn	middle	changelog missing or stale export

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
§7.1 NO-BLUFF positive-evidence	`.gate.failed`	`cherald`	enforce	critical	gate PASSes without captured evidence
§9.1 Destructive-op protocol	`.repo.safety_breach`	`sherald`	enforce	critical	`rm -rf` / `git reset --hard` without backup
§9.2 Force-push authorization	`.repo.safety_breach`	`sherald` + `pherald`	enforce	critical	force-push without explicit per-session auth
§9.3 Hardlinked backup before destructive	`.gate.recovered`	`sherald`	enforce	middle	backup-created event for audit trail
§9.4 Commit-message audit trail	`.policy.violation`	`cherald`	warn	low	history rewrite without audit line
§11.4.1 FAIL-bluffs forbidden	`.gate.failed`	`cherald`	enforce	critical	FAIL that's actually a fake-fail
§11.4.2 Recorded-evidence requirement	`.gate.failed`	`bherald`	enforce	high	test PASS without `tests/.captured-evidence/` artifact
§11.4.3 Per-environment-topology dispatch	`.gate.failed`	`bherald`	warn	middle	test ran in wrong environment
§11.4.4 Test-interrupt-on-discovery	`.gate.failed`	`bherald`	warn	middle	test continued past first failure
§11.4.5 Captured-evidence quality	`.gate.failed`	`bherald`	warn	middle	evidence file present but malformed/empty
§11.4.7 Demotion-evidence	`.gate.failed`	`bherald`	warn	middle	promotion lacks the captured demotion proof
§11.4.9 Batch-source-fixes-before-rebuild	`.policy.violation`	`bherald`	warn	low	per-fix rebuild loop detected
§11.4.10 Credentials-handling	`.credential.leak`	`cherald` + `iherald`	enforce	critical	tracked `.env`, plaintext credential in source
§11.4.10.A Pre-store leak audit	`.credential.leak`	`cherald` + `iherald`	enforce	critical	gate before committing credential storage
§11.4.11 File-layout discipline	`.policy.violation`	`pherald`	warn	low	file in wrong directory
§11.4.12 Auto-generated docs sync	`.policy.violation`	`cherald`	warn	low	covered by §37; reuses tracker events
§11.4.13 Out-of-band sink-side evidence	`.gate.failed`	`bherald`	warn	middle	log-side claim without sink-side proof
§11.4.14 Test playback cleanup	`.policy.violation`	`bherald`	warn	low	playback artifact left over after run

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
§11.4.15 Item-status tracking	`.policy.violation`	`pherald`	warn	low	covered by §37
§11.4.16 Item-type tracking	`.policy.violation`	`pherald`	warn	low	covered by §32.6 classifier
§11.4.17 Universal-vs-project rule promotion	`.policy.violation`	`cherald`	warn	low	rule promoted without §11.4 audit
§11.4.18 Script documentation mandate	`.policy.violation`	`cherald`	warn	low	shell script without companion `.md`
§11.4.19 Fixed-document column alignment	`.policy.violation`	`cherald`	warn	low	misaligned columns in `Fixed.md`
§11.4.21 Operator-blocked status	`.policy.violation`	`pherald` + `iherald`	enforce	high	item enters operator-blocked → page on-call
§11.4.22 Document-sync commit discipline	`.policy.violation`	`pherald`	warn	low	doc + code committed in separate commits
§11.4.23 Visual-cue & grouping for Issues docs	`.policy.violation`	`cherald`	warn	low	colorizer drift
§11.4.24 Build-resource stats tracking	`.gate.failed`	`bherald` + `sherald`	warn	middle	build stats missing
§11.4.25 Full-Automation-Coverage	`.policy.violation`	`cherald`	warn	low	manual step detected in operator runbook
§11.4.26 Constitution-Submodule Update Workflow	`.bundle.updated`	`sherald`	enforce	middle	constitution pulled successfully
§11.4.27 No-Fakes-Beyond-Unit-Tests	`.gate.failed`	`bherald`	enforce	high	fake/mock present in non-unit test
§11.4.28 Submodules-As-Equal-Codebase	`.policy.violation`	`cherald`	warn	low	submodule rule violated
§11.4.29 Lowercase-Snake_Case-Naming	`.policy.violation`	`cherald`	warn	low	naming convention violation
§11.4.30 No-Versioned-Build-Artifacts	`.repo.safety_breach`	`cherald` + `bherald`	enforce	high	build artifact committed
§11.4.31 Submodule-	`.policy.violation`	`cherald`	warn	low	manifest missing/stale

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
Dependency-Manifest					
§11.4.32 Post-Constitution-Pull Validation	`.bundle.updated` / `.bundle.update_failed`	`sherald`	enforce	high	post-pull gate result
§11.4.33 Type-aware closure-status vocabulary	`.policy.violation`	`pherald`	warn	low	covered by §32.6 classifier
§11.4.34 Reopened-source attribution	`.policy.violation`	`pherald`	warn	low	covered by §37 / §11.4.55 composition
§11.4.36 Mandatory install_upstreams	`.repo.safety_breach`	`sherald`	warn	middle	submodule missing upstream config
§11.4.37 Fetch-before-edit	`.repo.safety_breach`	`pherald`	warn	middle	edit on stale base
§11.4.38 Installable-Asset Evidence	`.release.gate_blocked`	`rherald`	enforce	high	release asset fails installability check
§11.4.39 Per-Feature On-Device Validation	`.gate.failed`	`bherald`	enforce	high	feature claimed done without on-device proof
§11.4.40 Full-suite retest before release tag	`.release.gate_blocked`	`rherald`	enforce	critical	tag attempted without full retest
§11.4.41 Pre-Force-Push Merge-First	`.repo.safety_breach`	`sherald` + `pherald`	enforce	critical	force-push without preceding merge
§11.4.42 Iteration-discipline	`.policy.violation`	`pherald`	warn	low	iteration cycle skipped
§11.4.43 TDD-Fix-Discipline	`.gate.failed`	`bherald` + `cherald`	enforce	high	fix without preceding red test
§11.4.44 Document Revision Header	`.policy.violation`	`cherald`	warn	low	doc missing revision header
§11.4.45 Integration-Status-Doc Maintenance	`.policy.violation`	`scherald` + `cherald`	warn	low	Status.md stale; composes with §37
§11.4.46 Validate-recent-work-before-post-flash	`.gate.failed`	`bherald`	warn	middle	post-flash tests ran on un-rebased base
§11.4.47 Firebase Data Review	`.policy.violation`	`sherald`	warn	low	data review skipped (project-specific)
§11.4.48 UI-Driven Video Testing	`.gate.failed`	`bherald`	warn	middle	UI test without video capture
	`.gate.failed`	`bherald`	warn	middle	

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
§11.4.49 Dual-Approach Testing					only one of unit/e2e present
§11.4.50 Deterministic Consistency	`.gate.failed`	`bherald`	enforce	high	flaky test detected (same input, different result)
§11.4.51 Live-ADB-First Maximization	`.policy.violation`	`bherald`	warn	low	proxy when live-ADB available
§11.4.52 Autonomous-Validation	`.gate.failed`	`bherald`	warn	middle	human-only validation step skipped
§11.4.53 Fixed_Summary parity	`.policy.violation`	`cherald`	warn	low	covered by §37
§11.4.55 Reopens-history tracking	`.policy.violation`	`pherald`	warn	low	reopen without Reopens/HRD-NNN.md
§11.4.56 Status_Summary parity	`.policy.violation`	`cherald`	warn	low	covered by §37
§11.4.57 README doc-link section	`.policy.violation`	`cherald`	warn	low	README missing doc-link table
§11.4.59 README always-sync	`.policy.violation`	`cherald`	warn	low	covered by §37
§11.4.60 Documentation composite covenant	`.gate.failed`	`cherald`	enforce	high	composite gate fail
§11.4.61 Markdown metadata + ToC	`.policy.violation`	`cherald`	warn	low	new structured doc missing metadata/ToC
§11.4.65 Universal Markdown export	`.policy.violation`	`cherald`	warn	low	.md edited without .html/.pdf regen
§11.4.66 Blocker-resolution clarification	`.policy.violation`	`pherald` + `iherald`	enforce	high	operator-blocked without clarification prompt
§11.4.67 Shell-script parseability	`.gate.failed`	`bherald`	warn	low	non-portable bashism detected
§11.4.68 Positive sink-side evidence	`.gate.failed`	`cherald`	enforce	high	downstream proof missing
§11.4.69 Sink-Side Positive-Evidence Taxonomy	`.gate.failed`	`cherald`	enforce	high	taxonomy violation
§11.4.71 Pre-Push Fetch +	`.repo.safety_breach`		enforce	high	

Constitution rule	Default event class	Owning flavor	Default mode	Severity	Notes
Investigate + Integrate		`pherald` + `sherald`			push without preceding fetch+integrate
§11.4.73 Main-spec versioning + revision discipline	`.spec.revision_drift`	`cherald`	enforce	high	spec touched without Revision bump
§11.4.74 Submodule-catalogue-first	`.catalogue.miss`	`cherald` + `pherald`	enforce	high	non-trivial PR missing Catalogue-Check line
§12.1 Forbidden host-session operations	`.host.safety_breach`	`sherald`	enforce	critical	suspend/hibernate/logout attempted
§12.2 Required safeguards	`.host.safety_breach`	`sherald`	enforce	high	heavy work without bounded scope
§12.3 Container hygiene	`.host.safety_breach`	`sherald` + `dherald`	enforce	high	container without mem_limit
§12.6 Memory-Budget 60% Ceiling	`.host.safety_breach`	`sherald`	enforce	critical	budget breach detected
§12.10 CONTINUATION sacred invariant	`.policy.violation`	`cherald`	enforce	high	CONTINUATION.md missing/stale

Total bindings: 65. Process-only rules excluded (no runtime event surface) — see prefix to this section.

42.4 Subscriber-facing payload shape

Every constitution event renders into a subscriber-facing message via the §39 Herald Canonical Template. The body block carries:

```
[<flavor icon>] <severity badge> Constitution rule violated: <rule_id>  —

<short description of the violation context (≤ 280 chars)>

Rule:          <rule_id> — <rule title from Constitution.md ToC>
Bundle:        <heraldconstitutionbundlesha short-form 12 chars>
Subject:       <file path / repo / commit SHA / tenant scope>
Decision:      <pass|warn|fail|error|skip>
Mode:          <allow|warn|enforce>
Evidence:      <link to evidence artifact or transparency log>
🔗 Constitution.md §<rule_id>: <permalink to that section on the active mi
👤 Caught by: <evaluator name>
🕒 <RFC 3339 timestamp>
```

For `allow` mode, the message is suppressed (only logged to diary + the pull surface). For `warn` mode, message routes to digest tag only. For `enforce`, full §6 tag-fan-out.

42.5 Why §42 is gated, not aspirational

The catalogue-row count (65) means landing every binding at once is impossible. Rollout discipline:

1. **First** — `commons_constitution` Go package (planned `commons_constitution/`) implementing the `Evaluator` interface + the 12-class event-emit helpers. **HRD-018**.
2. **Second** — `cherald` gets the bulk of bindings; HRD-019 implements the gate-result subscribers + the pull surface `/v1/compliance`. Initial mode for all bindings: `allow` (data gathering only).
3. **Third** — `sherald` host-safety + repo-safety bindings (§9, §12, §11.4.32, §11.4.36, §11.4.41, §11.4.71). **HRD-020**.
4. **Fourth** — `bherald` CI / test bindings (§1, §11.4.2 / .3 / .4 / .5 / .7 / .13 / .14 / .24 / .27 / .39 / .43 / .46 / .48–.52 / .67). **HRD-021**.
5. **Fifth** — `rherald` release bindings (§4, §5, §11.4.38, §11.4.40). **HRD-022**.
6. **Sixth** — `pherald` project bindings (§2, §3, §11.4.11 / .15 / .21 / .22 / .34 / .37 / .42 / .55 / .66 / .71 / .74). **HRD-023**.
7. **Seventh** — `iherald` incident bindings (§11.4.10 / .10.A escalation, §11.4.21 + .66 escalation). **HRD-024**.
8. **Eighth** — `scherald` scheduled audit bindings (§11.4.45 periodic audit + digests). **HRD-025**.
9. **Cross-cutting** — bundle-hash captureer (HRD-026), mode-ladder runtime config (HRD-027), pull surface `/v1/compliance` Gin handler (HRD-028).

Each HRD MUST carry a `Catalogue-Check:` per Universal §11.4.74 (since most of this work *might* already exist as a Submodule under `vasic-digital` / `HelixDevelopment` — the catalogue survey is HRD-018's first task).

42.6 Per-flavor cross-references

Each flavor's §18.X.2 "Constitution bindings" sub-section enumerates only the rules where that flavor is the *default owner* (the bold flavor in the master table above). Bindings where a flavor is a *secondary subscriber* (the second flavor in `cherald + iherald`-style rows) are implemented as filtered routes — the §6 channel router fans out to the secondary flavor's subscribers based on the `heraldconstitutionrule` extension attribute, not via a duplicate adapter.

The per-flavor sub-sections live at:

- **§18.2.7** `pherald` — Constitution bindings.
- **§18.3.2** `sherald` — Constitution bindings.
- **§18.4.2** `bherald` — Constitution bindings.
- **§18.5.2** `dherald` — Constitution bindings.
- **§18.6.2** `aherald` — Constitution bindings.
- **§18.7.2** `scherald` — Constitution bindings.
- **§18.8.2** `iherald` — Constitution bindings.
- **§18.9.2** `rherald` — Constitution bindings.
- **§18.10.2** `cherald` — Constitution bindings.

For brevity, the master table in §42.3 IS the source of truth; per-flavor sub-sections (added in §18.X.2 below by the implementation HRDs) point back to it and add only flavor-specific rendering / SLA notes that aren't captured by the table.

42.7 Composition + anti-bluff

§42 composes with:

- **§4 Event model** — constitution events use the same CloudEvents v1.0 envelope as every other Herald event.
- **§32 Inbound pipeline** — inbound subscriber acks/silences for constitution events use the standard pipeline (`Ack:` / `Silence: <duration>` / etc.).
- **§34 Reply protocol** — silencing a constitution event in `warn` mode emits the standard three replies.
- **§35 Versioned reports** — constitution-bundle updates emit `.bundle.updated` events that compose with §35 (the Constitution.md export bundle gets a new SHA).
- **§37 Tracker-doc change events** — rules that map onto tracker-doc state (§11.4.12, .15, .53, .56, .59, .60) are emitted via §37 with `heraldconstitutionrule` as an additional attribute — NOT duplicated as separate `.policy.violation` events.
- **§41 REST API** — every REST-enabled flavor mounts the `/v1/compliance` pull surface.

Anti-bluff (Universal §11.4 + §1.1):

- Every `.gate.failed` / `.gate.recovered` event MUST carry an `evidence_uri` per Universal §11.4.2.
- The `constitution_state` transition gate MUST itself have a paired §1.1 mutation: mutate the `last_decision` row to a different value while leaving the actual evaluator deterministic; expect the gate to detect the drift.

---

§43. Constitution-derived flavor commands + workflows

> **Operator mandate (2026-05-20):** the constitution analysis surfaces concrete *tooling integrations* — workflows + CLI subcommands + REST endpoints — that Herald flavors MUST ship as native capabilities, not as external scripts the operator stitches together. §43 enumerates those integrations. Each row is implementable as either (a) a Cobra subcommand on the owning flavor's binary, (b) a Gin endpoint under `/v1/<verb>` per §41, or both. Items in this catalogue are the *implementation* counterpart to §42's *event-binding* catalogue: §42 says "fan this event out"; §43 says "ship the tool that produces or acts on that event".

43.1 Why §43 is distinct from §42

§42 binds constitution *rules* to event types so violations + state changes surface to subscribers. §43 catalogues the *workflows* (often already named in the constitution itself — `install_upstreams`, `sync_issues_docs.sh`, the composite always-sync gate, etc.) that operators run today as standalone shell scripts. Herald wraps those workflows as flavor-native CLI subcommands + REST endpoints so:

- Operators get a single binary surface to call (`"pherald install-upstreams"` instead of "find the script in the constitution submodule and run it").
- The CLI/REST invocation emits the matching §42 event automatically.
- The workflow result lands in the diary + the §41 audit log + the §6 fan-out without bespoke instrumentation.

Every row in §43.2 below MUST land as its own HRD-NNN (already opened HRD-029..HRD-052 in `Issues.md`, with the catalogue-check field per Universal §11.4.74).

43.2 Master command catalogue (constitution rule → flavor command/ workflow)

Constitution rule	Flavor	CLI surface	REST surface (§41)	What it does
§2 Commit + push mechanics	`pherald`	`pherald commit-push [--scope=…]`	`POST /v1/commit-push`	Single-entrypoint lock multi-mirror push per `.repo.safety_br` entrypoint bypassed.
§3 Submodule propagation order	`pherald`	`pherald submodule propagate`	`POST /v1/submodule/propagate`	Walks owned-submod inner first, then paren propagation order.
§4 Tag mirroring	`rherald`	`rherald tag mirror <tag>`	`POST /v1/release/tag/mirror`	Asserts tag exists on e submodule before allo parent.
§5 Changelog discipline + multi-format export	`rherald`	`rherald changelog generate`	`POST /v1/release/changelog`	Generates `docs/cha` `<version>.md` fro Commits + exports to `.html`/`.pdf`/`.doc`
§9.1 Destructive-op protocol	`sherald`	`sherald destructive guard <op>`	`POST /v1/safety/destructive`	Wraps `rm -rf`/`git` `hard`/`git push` asserts the §9.1 prereq authorization).
§9.3 Hardlinked backup	`sherald`	`sherald backup snapshot <path>`	`POST /v1/safety/backup`	Creates a hardlinked s emits `.gate.recov`
§11.4.2 Recorded-evidence requirement	`bherald`	`bherald evidence capture <test_id>`	`POST /v1/build/evidence`	Records test stdout/st under `tests/.cap` `evidence/<test_` + §11.4.5.
§11.4.10 / .10.A Credentials handling	`cherald`	`cherald creds scan [--repo=…]`	`POST /v1/compliance/creds-scan`	Runs gitleaks / truffle working tree + history `.credential.lea`
§11.4.12 Auto-generated docs sync	`cherald`	`cherald docs sync`	`POST /v1/compliance/docs/sync`	Regenerates `Issues` `Fixed_Summary.r` `Status_Summary` canonical sources. Co
§11.4.18 Script-companion docs	`cherald`	`cherald script-docs check`	`POST /v1/compliance/script-docs`	Asserts every `scrip` has a sibling `.md` per
§11.4.19 / .23 Fixed-doc colorize + align	`cherald`	`cherald fixed align` + `cherald colorize`	`POST /v1/compliance/fixed/align`	Enforces §11.4.19 col §11.4.23 visual-cue H over Issues/Fixed doc
§11.4.26 Constitution-submodule update	`sherald`	`sherald constitution pull`	`POST /v1/safety/constitution/pull`	Wraps the §11.4.26 u fetch+rebase constitut gate per §11.4.32, em `.bundle.updated`
	`bherald`	`bherald test-tier verify`		

Constitution rule	Flavor	CLI surface	REST surface (§41)	What it does
§11.4.27 No-fakes + 100% test-type coverage			`POST /v1/build/tier-verify`	Walks the 8-tier test n emits `.gate.faile` tiers.
§11.4.31 Submodule-Dependency-Manifest	`cherald`	`cherald submanifest verify`	`POST /v1/compliance/submanifest`	Validates `docs/dep` `submodules.md` a `.gitmodules` + ac
§11.4.36 install_upstreams	`pherald`	`pherald install-upstreams`	`POST /v1/project/install-upstreams`	Reads `Upstreams/` + configures all 4 mir locally per §11.4.36.
§11.4.37 Fetch-before-edit	`pherald`	`pherald fetch-guard`	`POST /v1/project/fetch-guard`	Pre-edit hook: asserts rebased on `origin/` before any edit.
§11.4.40 Full-suite retest before tag	`rherald`	`rherald gate retest`	`POST /v1/release/retest-gate`	Pre-tag gate runs the asserts every tier (§40 `.release.gate_` FAIL.
§11.4.41 Pre-force-push merge-first	`sherald`	`sherald force-push gate`	`POST /v1/safety/force-push-gate`	Asserts the merge-firs explicit per-session au allowing force-push.
§11.4.45 / .56 Status + Status_Summary maintenance	`scherald`	`scherald status digest [--cadence=daily|weekly|monthly]`	`POST /v1/schedule/status-digest`	Periodic Status.md sw Status_Summary.md with §35 versioned re
§11.4.53 Fixed_Summary parity	`cherald`	`cherald fixed-summary sync`	`POST /v1/compliance/fixed-summary`	Auto-regen of `Fixed` whenever `Fixed.md` already a §37 trigger, standalone CLI for ba
§11.4.55 Reopens-history	`pherald`	`pherald reopen <HRD-NNN> [--reason="…"]`	`POST /v1/items/{id}/reopen` (§41 alignment)	Reverses an Issues→ writes `docs/Reope` `NNN>.md` per §11.4.
§11.4.59 README always-sync	`cherald`	`cherald readme sync`	`POST /v1/compliance/readme/sync`	Regenerates the REA section (§11.4.57) + r (§11.4.65).
§11.4.60 Composite always-sync covenant	`cherald`	`cherald composite-gate`	`POST /v1/compliance/composite-gate`	The canonical implen `DOCS-COMPOSITE` all 8 bound doc classe artefact-mtime invaria `.gate.failed` on
§11.4.65 Universal Markdown export	`cherald`	`cherald export <doc> [--formats=md,html,pdf,docx]`	`POST /v1/compliance/export`	Single-doc + bulk exp around Pandoc + Wea `.md/.html/.pdf/` formats.
§11.4.71 Pre-push fetch +	`pherald`	`pherald pre-push`	`POST /v1/project/pre-push`	Pre-push hook: fetch summarise incoming operator to acknowle

Constitution rule	Flavor	CLI surface	REST surface (§41)	What it does
investigate + integrate				plan before pushing. I `.repo.safety_br`
§11.4.73 Main-spec version + revision discipline	`cherald`	`cherald spec-version check`	`POST /v1/compliance/spec-version`	Asserts the main-spec `Revision` is current spec edits + project c `main_spec_path.` `.spec.revision_`
§11.4.74 Submodule-catalogue-first	`cherald`	`cherald catalogue-check <pr-url>`	`POST /v1/compliance/catalogue-check`	Scans a PR descriptio Issues.md row for the `Check:` line; survey- `vasic-digital` + `HelixDevelopmer` a search term. Emits `.catalogue.miss`
§12.6 Memory-budget ceiling	`sherald`	`sherald mem-budget watch` (daemon-mode subcommand)	`GET /v1/safety/mem-budget`	Continuous watcher e `.host.safety_br` threshold crossed. Co metrics.
Wave 6 operator mandate (2026-05-22) — inbound runtime	`pherald`	`pherald listen [--qa-out-dir <path>]`	(no REST; long-running CLI daemon)	Closed-loop subscribe runtime. Telegram `ge` poll + bot self-filter (§ content-addressed atta + Claude Code dispat pinned (§33.7) + enve (§33.6) + `<<<HERAL` action routing + `tgra` with `reply_to_me` Required env: `HERALD_TGRAM_BO` `HERALD_TGRAM_CH` Optional: `HERALD_F` (default: repo root fol `HERALD_CLAUDE_E` $PATH lookup). `--q` in journals every ever `transcript.json`
Wave 7 multi-channel (2026-05-28) — generic messenger framework	`pherald`	`pherald listen` (multi-channel mode)	(no REST; long-running CLI daemon)	`HERALD_CHANNELS` (default `tgram`) spin `Subscribe` goroutin channel, all fanning in `inbound.Dispatc` namespaced env: `HER` (Wave 6) + `HERALD_SLACK_BO` `HERALD_SLACK_AF` `HERALD_SLACK_CH` generalized self-filter `channels.IsSel` own echoes uniformly IdentityUsername (tgr IdentityUserID (slack

Constitution rule	Flavor	CLI surface	REST surface (§41)	What it does
				isolate under `~/.her` `<channel>/<sha` Slack adapter is Sock with tgram long-poll webhook required).

Total: **28 new flavor commands / workflows** — one per HRD-029 through HRD-056 plus HRD-100 (note: HRD-049 is `reopen` which maps onto an existing REST endpoint per §41 rather than a fresh `/v1/<verb>`; net new HRDs: 28).

Wave 2 (r8) addendum — shared CLI scaffold. The 28 commands above ship as Cobra subcommands on their owning `<prefix>herald` binary. Wave 2 landed the **shared CLI scaffold** (`commons/cli/`) that every flavor binary consumes — `NewRootCmd` / `VersionCmd` / `ServeCmd` / `StubCmd` plus `/v1/healthz`, `/v1/readyz`, `/v1/metrics`, and per-flavor 501-stub route registration. Tracked under **HRD-092** (Catalogue-Check: `no-match → vendor as Herald-internal package`; evidence in `docs/catalogue-checks/HRD-092-commons-cli.md`). HRD-092 is **Fixed (Wave 2, r8)** — every §43.2 command's Cobra surface is now built on top of this scaffold rather than each flavor reinventing root + version + serve. See §44.M for the full Wave 2 milestone.

43.3 Implementation gating

§43 is a *catalogue*; landing all 27 commands at once would freeze every other workstream. Order:

1. **First wave** — workflows the inheritance gate already depends on (no Herald implementation required to FUNCTION, but Herald wrapping them makes the operator surface uniform): `pherald install-upstreams` (HRD-043), `sherald constitution pull` (HRD-040), `cherald composite-gate` (HRD-051), `cherald export` (HRD-052).
2. **Second wave** — workflows that produce `.gate.failed` / `.policy.violation` events the §42 catalogue references but currently have no emitter: `cherald creds scan` (HRD-036), `cherald docs sync` (HRD-037), `cherald script-docs check` (HRD-038), `pherald fetch-guard` (HRD-044), `sherald force-push gate` (HRD-046).
3. **Third wave** — operator-quality-of-life CLIs: `pherald reopen` (HRD-049), `rherald changelog generate` (HRD-032), `rherald gate retest` (HRD-045), `scherald status digest` (HRD-047).
4. **Fourth wave** — the remainder.

The §11.4.74 catalogue-check applies: each HRD-029..-056 MUST start with a survey of `vasic-digital` + `HelixDevelopment` for an existing Submodule. Many of these workflows already exist as shell scripts in the constitution submodule itself (`install_upstreams.sh`, `sync_issues_docs.sh`, etc.); the Herald implementation wraps + extends them rather than reinventing.

43.4 Composition with §41 REST + §42 events + §39 templates

Every §43 command flows through the same three layers:

- **§41 REST** — every command exposes a matching `/v1/<verb>` endpoint accepting the same parameters as the CLI; apps + dashboards consume the REST surface.

- **§42 events** — invoking the command emits the matching event class (`.gate.failed` / `.policy.violation` / `.repo.safety_breach` / etc.) via `commons_constitution`; subscribers see the action through their configured channels.
- **§39 templates** — the subscriber-visible message uses the Herald Canonical Template per-flavor; outcomes (success / failure / partial) follow the §39.5 style rules.

This three-layer pattern makes adding a 28th workflow (or a project-specific Submodule's workflow per §11.4.74) trivial: define the command + register its event class + author the §39 template + open the HRD.

43.5 Boundary: what §43 does NOT add

- **Per-channel adapters** — those are §11 + §18.X.1, not §43.
- **General-purpose Go libraries** — `commons_*` packages are §10, not §43.
- **Subscribers' workflows** — `Bug:` / `Query:` / `Status:` style subscriber commands are §18.X (per-flavor inbound) + §32 (inbound pipeline), not §43. §43 is operator-facing (CLI + REST), not subscriber-facing (chat/email/etc.).

§43 is intentionally scoped to *operator workflows that the constitution itself names or implies*. Anything else (custom project tooling, integrations with external SaaS) is the consuming project's concern, not Herald's.

---

§44. Foundation implementation contract

> Added 2026-05-20 (V3 r7). Locks the contract for the **Foundation sub-project** — the first of eight implementation sub-projects derived from V3. Full design + rationale lives in `docs/superpowers/specs/2026-05-20-foundation-design.md`; this section is the contract the spec carries forward.

§44.1 Scope

Foundation composes HRD-018 (`commons_constitution`) + HRD-010 (`commons_storage` live wiring) + HRD-016 (Gin REST skeleton subset) + HRD-026 (bundle-hash captureer) + HRD-027 (mode-ladder runtime config) + HRD-028 (`/v1/compliance` pull surface) into a single, end-to-end-smokeable substrate. Every other sub-project (channel adapters, dispatchers, sweep daemon, REST surface expansion, multi-flavor binaries) builds on Foundation.

§44.2 Done criterion (locked)

The Quickstart compose stack accepts a real CloudEvent v1.0 on `POST /v1/events`, fans it out to the `null://` channel, writes a `constitution_state` row with the correct transition, and exposes that row via `GET /v1/compliance`. The full smoke is reproduced in `quickstart/` and runs in CI per §40.

§44.3 Three-milestone delivery (Approach B — bottom-up vertical slices)

Milestone	What lands	Smoke criterion
M1	`commons_constitution` package (Evaluator + 12 emit helpers + BundleHash +	`go test -race ./commons_constitution/...`

Milestone	What lands	Smoke criterion
	ModeLadder + ConstitutionStore + in-process EventBus) — all in-memory, no external deps.	proves an in-memory evaluator detects a transition → emits a `.policy.violation` → memory-pubsub listener counts it.
M2	`commons_storage` live: `digital.vasic.database` (pgx + migrations) + `digital.vasic.background` (Postgres job queue). New migrations 000006_constitution_state + 000007_constitution_bindings. Postgres backends for `ConstitutionStore` + `ModeLadder`.	`go test -tags=integration` against testcontainers Postgres + queue verifies UPSERT semantics, RLS isolation, transition detection, audit-row append, job enqueue + consume.
M3	`pherald/internal/http/` (Gin via `digital.vasic.middleware` + `digital.vasic.auth` + `digital.vasic.observability`) + `digital.vasic.cache` 60s read-cache for mode-ladder. Wire `pherald serve` to mount the Gin router.	Quickstart compose up; curl POST `/v1/events` → fans to `null://` → `constitution_state` row written → diary appended → curl GET `/v1/compliance` returns the row.

§44.4 Evaluator trigger model (locked)

Hybrid: push-trigger for critical-severity rules (§9 secret-handling, §11.4.10 root-cause, §12 ops invariants — re-evaluate immediately on the named CloudEvents type); pull-sweep every 5–15 min for lower-severity rules (sweep daemon itself lives in Sub-project 5, but the Registry building blocks land in Foundation M1).

§44.5 Mode-ladder storage (locked)

Postgres `constitution_bindings` table is the source of truth. Redis 60-second read-cache wraps it (mirrors §4.3 idempotency-keys pattern). Cache invalidation on `ModeLadder.Set` is published as a `mode-ladder.invalidated` event on the in-process EventBus so every running process invalidates its own cache (see Foundation design §7 open question 3 for the rationale).

§44.6 Three-axis governance envelope (locked)

Every emitted Constitution-CloudEvent carries the spec §42.1.1 envelope:

```
{
  "envelope": {
    "rule_id": "§11.4.10",
    "severity_category": "critical",
    "decision_result": "fail",
    "bundle_hash": "<sha256-hex>",
    "transition_from": "pass",
    "transition_to": "fail",
    "tenant_id": "<uuid>",
    "evidence_uri": "<uri>",
```

```
    "traceparent": "<W3C trace-context>"
  },
  "payload": { /* per-class typed payload */ }
}
```

On the wire (CloudEvent extension attributes), underscored keys are normalised to lowercase alphanumeric per CloudEvents v1.0 §3 (`rule_id` → `ruleid`, etc.). The internal Go API keeps the readable form; the boundary normaliser lives in `commons_constitution/cloudevents.go`.

§44.7 Catalogue-Check verdict (recorded 2026-05-20)

Per Universal §11.4.74, every Foundation capability surveyed for existing modules in `vasic-digital` + `HelixDevelopment` before code lands. Full evidence in `docs/catalogue-checks/HRD-018-foundation.md`. Summary:

- **9 of 12** capabilities `extend` existing Helix-stack modules.
- **3 of 12** capabilities `no-match` (bespoke, written new): Evaluator framework, BundleHash captureer, ModeLadder semantics.
- River queue **replaced** by `digital.vasic.background`; Watermill pub/sub **replaced** by `digital.vasic.eventbus`; raw Gin/JWT/OTel **composed** from `digital.vasic.middleware` + `auth` + `observability`; raw pgx + golang-migrate **wrapped** via `digital.vasic.database`.

§44.8 Anti-bluff testing mandate (continuous)

Per operator mandate 2026-05-20: every Foundation test (M1 unit + M2 integration + M3 e2e) MUST exercise the behavior it claims to verify — no pass-without-execution paths, no mock-only tests that don't round-trip, no `t.Skip` paths that mask broken features. Each test failure in CI MUST imply a real broken feature. This mandate propagates to all Helix-stack submodules consumed by Foundation (their existing tests + Challenges are audited at integration time per the codegraph milestone).

§44.9 M1 evidence (landed 2026-05-20)

The Foundation M1 milestone landed with the following observed deltas:

- **New module**: `github.com/vasic-digital/herald/commons_constitution` — 8 production files (~1.2k LOC) + 6 test files (~1.2k LOC).
- `go test -race -count=1 ./commons_constitution/...` — all 3 packages PASS (top, ladder/, state/).
- **Inheritance gate** — 12 PASS / 0 FAIL before + after.
- **Other workspace modules** — unchanged behavior (still PASS).
- **I6 gate-policy conflict surfaced** — current `I6` invariant in `tests/test_constitution_inheritance.sh` forbids any `.gitmodules` file (originally written to prevent re-embedding the parent `constitution/`). M1 sidesteps this by writing all Helix-stack modules' Go-equivalents in-package; M2/M3 will require either an I6 refinement (`.gitmodules` permitted iff it does NOT contain a `constitution/` entry) or a non-submodule install path. **Open: HRD-080** (gate refinement) tracks this.

When M2 closes, this section is appended with the M2 evidence. M3 closure appends the as-built CloudEvent example + RLS isolation proof + null:// ring-buffer trace.

§44.M Wave 2 — Flavor scaffolds (landed 2026-05-21)

Scope (per [`docs/superpowers/specs/2026-05-21-wave2-flavor-scaffolds-design.md`]):

- Shared `commons/cli/` package — `NewRootCmd`, `VersionCmd`, `ServeCmd` (with Middleware hook), `StubCmd`, `Healthz`/`Readyz`/`Metrics` handlers, `Route` + `StubRouteHandler`. ~400 LOC + 7 unit tests.
- `pherald` refactor — `cmd/pherald/main.go` + stubs consume `cli.NewRootCmd` + `cli.VersionCmd`; `serve.go` uses `cli.ServeCmd` with `pherald`'s `RequestIDMiddleware` + 2 flavor routes. Old `pherald/internal/http/server.go` (150 LOC) deleted; 5 `server_test.go` tests replaced by 4 focused middleware + routes tests.
- 6 new flavor binaries scaffolded: `sherald` (:24793 serving), `cherald` (:24792 serving), `iherald` (:24794 serving), `bherald` (CLI-only), `rherald` (CLI-only), `scherald` (CLI-only). Each `cmd/<flavor>/main.go` is ~25 LOC consuming the shared scaffold.
- `Branding` struct extended with 5 per-flavor identity fields (`Flavor`, `Prefix`, `DisplayName`, `DefaultPort`, `Mission`); `commons.DefaultBranding(<flavor>, version)` populates them for every flavor key.
- `scripts/e2e_bluff_hunt.sh` invariants grow 18 → 33: **E19–E24** cover the 6 new `version --json` shapes, **E25–E30** cover serving-flavor `/v1/healthz` + flavor-specific 501 routes, **E31** covers a representative §43 stub HRD-pointer surfacing, **E32** covers `sherald` SIGTERM graceful shutdown, **E33** is the `pherald` regression sentinel.
- Paired §1.1 mutation gate: `tests/test_wave2_mutation_meta.sh` (3 mutations + 1 post-flight, all PASS).

As-built evidence (2026-05-21):

- Commits: `7e0a614..37e348e` (13 Wave 2 commits) + `24b96f2` (branding polish, parallel work).
- `go build ./...` across 13 workspace modules: PASS clean.
- `go test -race -count=1 ./...`: PASS across all 13 modules.
- `scripts/e2e_bluff_hunt.sh`: **33 PASS / 0 FAIL / 3 SKIP** (live channel SKIPs documented per §11.4.3).
- `tests/test_wave2_mutation_meta.sh`: **4/4 PASS** (mutations correctly trigger e2e failures).
- `scripts/audit_antibluff.sh`: **16 PASS / 0 FAIL / 1 SKIP**.
- Inheritance gate: **15/15 PASS**; meta-test META-PASS.

Catalogue-Check verdict: `commons/cli/` is `no-match → vendor as Herald-internal package` per Universal §11.4.74. Tracked under HRD-092 (now Fixed in r8). Evidence: `docs/catalogue-checks/HRD-092-commons-cli.md`.

Open follow-ups (Wave 3+):

- **~~HRD-016~~** — `pherald /v1/events` Runner wiring → **closed in Wave 3b (r9); see §44.N below**.
- **HRD-024** — `iherald /v1/webhooks/page` live (Wave 3/4).
- **HRD-028** — `cherald /v1/compliance` constitution_state pull live (Wave 3).

- **HRD-098** — `sherald /v1/safety_state` live (Wave 3).
- **HRD-029..056** — the 28 §43 command bodies (Wave 4+).

§44.N Wave 3b — pherald Runner live + `/v1/events` 202 (landed 2026-05-22)

Scope (per [`docs/superpowers/plans/2026-05-22-wave3b-runner.md`]):

The §32 7-stage event-ingest pipeline lands as 7 concrete structs under `pherald/internal/runner/`, orchestrated by `runner.Runner.Run`:

1. `event_parser.go` — Stage 1 EventParser (CloudEvent v1.0 parse + tenant guard + idempotency-key extraction).
2. `idempotency.go` — Stage 2 IdempotencyChecker (Redis SETNX + PG fallback).
3. `tenant.go` — Stage 3 TenantResolver (binds `app.tenant_id` GUC on the per-request transaction context).
4. `policy.go` — Stage 4 PolicyGate (runs registered constitution evaluators; ships permissive by default).
5. `subscriber.go` — Stage 5 SubscriberResolver (lists subscribers + aliases for the tenant under RLS).
6. `dispatcher.go` — Stage 6 ChannelDispatcher (fan out to channel adapters per recipient alias).
7. `outcome.go` — Stage 7 OutcomeRecorder (persists `outbound_delivery_evidence` + `events_processed` archive).

`pherald/internal/http/events.go` exposes the live `POST /v1/events` handler. `pherald/cmd/pherald/{main,stubs}.go` lazily build the Runner + JWT verifier inside the serve subcommand's `RunE` so `pherald --help`, `version`, and `migrate` paths still work without `HERALD_PG_DSN` / `HERALD_AUTH_MODE`.

As-built evidence (2026-05-22):

- Commits: `eb9b2f4` (T1 runctx) → `40ad60f` (T2 EventParser) → `2fdb7b5` (T3 IdempotencyChecker) → `4c4af24` (T4 TenantResolver) → `3988114` (T5 PolicyGate) → `5468897` (T6 SubscriberResolver) → `6ea29a4` (T7 ChannelDispatcher) → `be71db1` (T8 OutcomeRecorder) → `c7b89a4` (T9 Runner orchestrator + 3 integration tests + real PG/Redis adapters) → **this commit** (T10 HTTP handler + main.go wiring + e2e E37-E42 + Issues→Fixed + 4-mirror push).
- `go build ./pherald/... ./commons/... ./commons_auth/... ./commons_storage/... ./commons_messaging/...`: PASS clean.
- `go test -race -count=1 ./pherald/internal/runner/...`: 23/23 PASS (per-stage units + 3 integration scenarios: happy / duplicate / deny).
- `scripts/e2e_bluff_hunt.sh`: **47 invariants** total (E1..E48; E37-E42 live in Wave 3b; E45 cross-binary still SKIP-with-reason — runtime gate: PG :24100 reachability).
- `tests/test_wave3_mutation_meta.sh`: M1+M2+M4+M5 prove their gates; M3+M6 SKIP-with-reason pending Wave 3c deny-evaluator e2e + cross-binary wiring; post-flight green after restores.

HTTP contract (`POST /v1/events`):

Scenario	Status	Body	Headers
Fresh CloudEvent	202 Accepted	Receipt JSON	—
Replay of prior idempotency key	200 OK	Receipt JSON	

Scenario	Status	Body	Headers
			X-Herald-Replay: true
Missing/invalid Bearer JWT	401 Unauthorized	{"error":"..."}	—
Malformed CloudEvent body	400 Bad Request	{"error":"event_parser: ..."}	—
Policy DecisionFail	202 Accepted	Receipt JSON (Recipients=0, denied evidence row)	—
Stage error (Redis outage etc.)	500 Internal Server Error	{"error":"<stage tag>: ..."}	—

Open follow-ups (Wave 3c+):

- Cross-binary E45 wiring (pherald deny POST → cherald GET /v1/compliance round-trip).
- Deny-evaluator e2e invariant (currently M3 SKIP-with-reason).
- Parallel-fan-out optimization in Stage 6 (current implementation is serial per recipient).
- OpenAPI surface for /v1/events per §41.7.
- ✅ docs/INTEGRATION.md consumer onboarding guide — landed in commit e89f509 post-Wave-3b.
- ✅ v0.1.0 release tag — issued post-Wave-3b.

§44.O Wave 4a — Transport substrate: HTTP/3 + Brotli + Alt-Svc + TLS 1.3 (landed 2026-05-22)

Scope (per [docs/superpowers/specs/2026-05-22-http3-brotli-toon-design.md] + [docs/superpowers/plans/2026-05-22-wave4a-http3-brotli.md]):

Every Herald serving binary (pherald, cherald, sherald, iherald) now binds a dual TCP/HTTP-2 + UDP/HTTP-3 listener on the same port. The Wave 4a transport contract:

- **HTTP/3 (QUIC) primary** on UDP via digital.vasic.http3 (vendored as submodules/http3 from vasic-digital/http3@1d0df7b), TLS 1.3 mandatory, ALPN h3.
- **HTTP/2 fallback** on TCP via net/http.Server.ListenAndServeTLS reusing the same tls.Certificate (no temp file), ALPN advertises h2+http/1.1.
- **Alt-Svc header** alt-svc: h3="<port>"; ma=2592000 (30-day cache) on every TCP response so capable clients migrate to HTTP/3 on the next request.
- **Brotli compression** via commons/cli/brotli.go wrapping digital.vasic.middleware/pkg/brotli, default quality 6 (HERALD_BROTLI_LEVEL env override clamped 0..11), MinLength=256B, falls through to identity for sub-threshold bodies. Honest passthrough behaviour: a request with Accept-Encoding: br that the middleware decides not to compress receives identical bytes to the Accept-Encoding: identity request.
- **TLS cert sourcing** — new commons_tls/ package (workspace module 15) with LoadOrGenerate(certPath, keyPath) + ResolveCertSource(Config):
    - **Dev mode** (HERALD_AUTH_MODE != "jwks") — auto-generate ECDSA P-256 self-signed cert at ~/.herald/dev-cert.pem + dev-key.pem, SAN list [localhost, 127.0.0.1, ::1], 365-day validity, key chmod 600. Source label autogen-dev.

- ◦ **Prod mode** (`HERALD_AUTH_MODE == "jwks"`) — fail-loud if neither `--tls-cert`/`--tls-key` flags nor `HERALD_TLS_CERT_PATH`/ `HERALD_TLS_KEY_PATH` env vars are set.
  - ◦ **Asymmetric guard** — `LoadOrGenerate` errors if exactly one of (cert, key) exists. Never auto-pairs an operator-provided cert with an auto-generated key.
- `HERALD_DISABLE_HTTP3=1` **escape hatch** — for environments where outbound UDP/ 443 (or the configured port) is blocked. TCP/HTTPS still binds; the Alt-Svc middleware becomes a no-op.
- **New flags** on every serve subcommand (auto-wired via `cli.BindServeFlags`): `--tls-cert`, `--tls-key`, `--no-brotli`. The dual-listener body is exported as `cli.RunServe(ctx, opts, port)` so flavors with lazy dependency construction (pherald — builds Runner only at serve-time) can call it directly without going through `cli.ServeCmd`.

As-built evidence (2026-05-22):

- Commits: `b16ff89` (T1 vendor http3 submodule) → `4ffc556` (T2 commons_tls) → `78e35de` (T3 h3.go listener + real quic-go round-trip test) → `548f354` (T4 AltSvcMiddleware) → `d1aaab9` (T5 BrotliMiddleware q6 + decode-and-compare test) → `c0f1068` (T6 serve.go dual-listener refactor) → `171e55b` (T7 flag wiring for cherald/ sherald/iherald) → `eebfe70` (T7.5 pherald unification onto `cli.RunServe`) → `8720c39` (T8 e2e E49-E55 + scheme flip http→https for E7-E47) → `966de65` (T9 `test_wave4_mutation_meta.sh`) → `d5bd360` (security fix restoring middleware.go from JWT-bypass mutation residue in 72e81ab) → **this commit** (T10 spec V3 r9 → r10 + Issues r12 → r13 + Fixed r11 → r12 + Status r13 → r14 + tag v0.2.0 + 4-mirror push).
- `go build` across all 15 workspace modules: PASS clean.
- `go test -race -count=1 ./...`: PASS workspace-wide.
- `scripts/e2e_bluff_hunt.sh`: **54 invariants** total (E1..E55). 37 PASS / 0 FAIL on a fresh shell without containers or root; E13-E18+E34+E37-E42+E45+E48+E55 SKIP-with-reason for environmental prereqs.
- `tests/test_wave4_mutation_meta.sh`: 4/4 PASS — M1 strip H3 → E50 (UDP listener absent) FAIL; M2 strip Brotli → E53 (Content-Encoding:br absent on ≥256B body) FAIL; M3 downgrade TLS 1.3 → 1.2 → upstream h3 server refuses to start, E49-E55 cascade FAIL with E54 captured; post-flight green after restores.
- Full anti-bluff battery (10 gates) green: inheritance (15/15) + meta-PASS; I6 (3/3); I8 (5/5); Wave 2 mutation (4/4); Wave 3 mutation (3/3); Wave 4 mutation (4/4); audit_antibluff green; codegraph_validate (7/0/2); e2e_bluff_hunt (37/0/SKIP).

Critical security fix lesson (commits `72e81ab` → `d5bd360`):

The `FIX-bg` subagent (logo pandoc-attr → HTML `<img>` swap) committed its working tree while a Wave 3 mutation gate was concurrently running. The mutation gate had mid-flight applied the M1 mutation to `commons_auth/middleware.go` (strip JWT verification, set fake all-zero tenant). The `FIX-bg` agent's `git add` captured that mutation state and pushed it to all four mirrors — every release built from commit `72e81ab` would have authenticated nothing. Commit `d5bd360` restores the file from the last good commit and patches `tests/test_wave2_mutation_meta.sh` to probe `https://` instead of `http://` (legitimate Wave 4a follow-up given the dual-listener default scheme flip).

Constitutional addition (proposed for next propagation pass): §107.x "Working-tree quiescence rule" — no subagent commits while a mutation gate is in flight; `git status --porcelain` must reconcile with the explicit expected change set; any unexplained file in the staging area MUST trigger an abort.

Operator-facing behavior changes from v0.1.0 → v0.2.0:

- Every flavor binary now serves HTTPS (self-signed dev cert by default) — operators must use `curl -k` against `https://` URLs (or set `--tls-cert`/`--tls-key` to a real cert).
- Existing operator scripts using `http://127.0.0.1:24791/v1/healthz` MUST update to `https://127.0.0.1:24791/v1/healthz` (or set `HERALD_DISABLE_HTTP3=1` if they want plaintext for legacy debugging).
- HTTP/3-aware clients (browsers, modern HTTP clients) automatically migrate to UDP on the second request after seeing the Alt-Svc header.

Open follow-ups (Wave 4b — TOON adoption): all closed; see §44.P below.

---

§44.P Wave 4b — TOON (`application/toon`) content negotiation (landed 2026-05-22)

Scope (per [`docs/superpowers/specs/2026-05-22-http3-brotli-toon-design.md`] §5 + [`docs/superpowers/plans/2026-05-22-wave4b-toon.md`] T1..T7):

Herald adopts TOON (Token-Oriented Object Notation) as the preferred wire format on every `/v1/*` business route, with JSON as the honest fallback. The negotiation is driven by the standard `Accept` header — clients in control. The `digital.vasic.toon` submodule (the prior PASS-bluff watershed surface) was extended in Wave 4b T2 with real `toon-format/toon-go` delegation; the sentinel-error scaffold from the anti-regression revert is replaced with a real codec.

- **TOON adoption per W4b operator decisions** — TOON is the server-default (clients sending `Accept: */*` or missing Accept get TOON); `HERALD_DEFAULT_RESPONSE_CODEC=json` flips the default; explicit `Accept: application/json` is honored; explicit `Accept: application/toon` is honored; both present without q-params → TOON wins (Herald-default tie-break); both with q-params → higher q wins; unsupported Accept (e.g., `application/xml`) → JSON safe fallback (honest header-body match, never advertise a content-type the server doesn't speak).
- **Inbound CloudEvents accept both** — `POST /v1/events` reads `Content-Type` and decodes TOON when it is `application/toon`, JSON when it is `application/json` (default for missing Content-Type, backwards-compat with curl `-d` usage). TOON request bodies are transcoded to JSON BEFORE the Runner's EventParser (the Runner stays codec-agnostic — transport-layer codec adaptation only).
- **Encoder-failure fallback is observable** — if `toon.Marshal` fails (Go type the codec can't represent), the server falls back to JSON AND emits `X-Herald-Codec-Fallback: encoder_error` header AND keeps `Content-Type: application/json` (honest header-body match). Never a silent swap.
- `/metrics` + `/healthz` + `/readyz` **stay JSON/text** — observability + LB probes don't negotiate; healthz registered BEFORE the TOON middleware in serve.go so it's structurally protected.
- **TOONMiddleware auto-wired into** `cli.RunServe` — every serving flavor binary inherits for free (cherald + sherald + bherald + rherald + iherald + scherald via `cli.ServeCmd`; pherald via its custom cobra wrapper that also calls `cli.RunServe`). Zero per-flavor code change beyond the existing chain registration.

As-built evidence (2026-05-22):

- Commits: `e81f175` (T1 vendor `digital.vasic.toon` submodule at the sentinel-error pin) → `cca1ee2` (T2 upstream PR — replace sentinel with `toon-format/toon-go` wire-up; new submodule SHA `9ae61db`) → `0a94fb3` (T3 `commons/cli/contentnego.go` — Accept-header negotiation + MarshalChosen/UnmarshalChosen) → `de59f0a` (T4 `commons/cli/toon.go` — Gin TOONMiddleware + BindNegotiated) → `9f1d378` (T5 pherald events.go TOON-aware — request transcode + response inheritance via middleware) → `a3164e5` (T6 auto-wire TOONMiddleware into `cli.RunServe`; cherald + sherald + every flavor inherits) → **this commit** (T7 e2e E56-E62 + mutation gate + spec V3 r10 → r11 + Issues r13 → r14 + Fixed r12 → r13 + Status r14 → r15 + tag v0.3.0 + 4-mirror push).
- `go build` across all 15 workspace modules: PASS clean.
- `go test -race -count=1 ./...`: PASS workspace-wide.
- `scripts/e2e_bluff_hunt.sh`: **61 invariants** total (E1..E62; E55 + E62 + container-gated invariants SKIP-with-reason on a fresh shell). 43 PASS / 0 FAIL / 18 SKIP on a fresh shell.
- `tests/test_wave4b_mutation_meta.sh`: 5/5 PASS — M1 strip TOONMiddleware auto-wire → E57 FAIL (response stays JSON); M2 swap `toon.Marshal` → `json.Marshal` (the 2026-05-17 PASS-bluff signature) → E60 FAIL (wire byte 0 = `{` under TOON Content-Type); M3 no-op `transcodeRequestBody` → pherald `TestEventsHandler_ContentTypeTOON_RequestDecoded` FAIL; working-tree quiescence guard (lesson from 72e81ab — `git status` MUST be clean of `MUTATED W4B-M*` markers BEFORE declaring PASS); post-flight e2e green after every restore.
- Full anti-bluff battery (11 gates) green: inheritance (15/15) + meta-PASS; I6 (3/3); I8 (5/5); Wave 2 mutation (4/4); Wave 3 mutation (3/3 + 3 SKIP); Wave 4 mutation (4/4); Wave 4b mutation (5/5); `audit_antibluff.sh` green; `codegraph_validate.sh` (7/0/2); `e2e_bluff_hunt.sh` (43/0/18).

§107 watershed lineage:

`digital.vasic.toon` is the operative cautionary tale of the entire Herald project — a prior revision silently delegated to `encoding/json` while claiming TOON encoding (caught by the round-27 audit). The revert to a sentinel-error scaffold (`ErrTOONEncodingNotImplemented`) was an anti-regression marker. Wave 4b T2 replaces the sentinel with REAL `toon-format/toon-go` delegation. The mutation gate at `tests/test_wave4b_mutation_meta.sh` plants the EXACT shape of the original bluff (M2: TOON branch silently calls json.Marshal) and asserts the e2e battery FAILs at the wire byte-0 check (E60). A recurrence cannot pass silently.

Operator-facing behavior changes from v0.2.0 → v0.3.0:

- `Accept: */*` (or missing Accept header) on any `/v1/*` business route now returns `application/toon`. Operators using `curl` to probe Herald MUST either set `Accept: application/json` explicitly or pipe through a TOON decoder. The shipped e2e probes (E44, E47) were updated in this commit to set `Accept: application/json` so they continue to validate the JSON-shaped response contract.
- `HERALD_DEFAULT_RESPONSE_CODEC=json` is the operator opt-out — flips the server default to JSON for transition periods or legacy client compatibility.
- `POST /v1/events` now accepts `Content-Type: application/toon` request bodies — agentic/CI callers can drop the JSON encoding step.

Token efficiency rationale (per §5.1 of the design doc):

TOON is a tabular-collection-optimized wire format that drops `[]map[string]any`-shaped collections into a header-and-row layout: `users[2]{id,name}: 1,alice 2,bob` instead of `{"users":[{"id":1,"name":"alice"},{"id":2,"name":"bob"}]}`. For Herald's tabular payloads (compliance results, safety samples, receipt logs, pagination pages), TOON saves 30-60% of bytes versus JSON. For tiny constant-shape payloads (e.g., a SafetyState snapshot), the savings are modest (~5%) but still positive. Brotli compresses on top — combined TOON+Brotli typically lands at ~25% of the original JSON-uncompressed wire footprint.

Continuation: QA workstream (queued — `qaherald` Go binary driving pherald ⍰ Telegram round-trip automation; QA evidence under docs/qa/<run-id>/; final close-out tag `v1.0.0-dev-0.0.1`). Wave 3c carry-over remains queued (HRD-024 iherald paging, HRD-033 destructive-guard body, HRD-018..027 flavor bindings, the rest of the §43 command catalogue).

---

§30. V2 self-review log

> Added 2026-05-19 by a focused self-review pass on V2 r1. Each finding is tagged `V2-R-NN` to distinguish from V1's `R-NN` series. **All findings in this section have been applied in V2 r2** unless explicitly marked Deferred.

30.1 Gaps closed (applied this revision)

- **V2-R-01. Missing Go value types referenced by the `Channel` interface.** §11 previously declared `Channel` with `Capabilities`, `OutboundMessage`, `Receipt`, and `InboundHandler` as forward references without definitions. Applied: new **§11.0** ("Channel adapter contract") defines all referenced types — `Capabilities`, `DeliveryEvidence` (enum), `OutboundMessage`, `Body`, `Attachment`, `Recipient`, `Action` / `ActionType`, `Priority`, `Receipt`, `InboundHandler`, `InboundEvent`. These live in `commons/types.go`; adapters are forbidden from inventing equivalents.
- **V2-R-02. Missing `webhook_sources` table schema.** §5.5 referenced the table but never defined it. Applied: full schema (signature_kind, signature_header, secret_encrypted, secret_rotated_at, ip_allowlist, replay_window_s) with RLS policy and <flavor>herald webhook rotate CLI.
- **V2-R-03. Missing `channel_addresses` table schema.** §6 referenced the table but never defined it. Applied: full schema with `address_url` (env-interpolated at load time so secrets stay out of the row), `tags` GIN index, `priority_floor`, `enabled` + health-check columns.
- **V2-R-04. Missing `thread_refs` table schema.** §12 prose-only definition. Applied: full schema with composite PK `(tenant_id, logical_thread_id, channel)`, `last_activity_at` for time-window GC.
- **V2-R-05. Missing `quarantined_messages` table schema.** §15.2 referenced but never defined. Applied: full schema with `triage_status` enum, partial index on pending rows, 30-day auto-expiry policy, CLI verbs.
- **V2-R-06. `subscribers.handle` had no uniqueness constraint.** Original schema allowed duplicate handles within a tenant — operationally surprising. Applied: `UNIQUE (tenant_id, handle)`. Also added `roles TEXT[]` and `metadata JSONB` columns for the operator-mapped privilege model and extensible per-subscriber data, plus an explicit composite index on `tenant_id`.

- **V2-R-07. Go toolchain version not pinned.** §9.1 said "written in Go" without naming a minimum. Without `slog` and the OTel `slog` bridge, observability won't compile. Applied: **Go ≥ 1.22** mandatory; `toolchain` directive across all `go.mod` files; bump path documented.
- **V2-R-08. License not named.** Multiple references to "the LICENSE file" without spelling out the terms. Applied: §9.1 now points to the parent project's `LICENSE` and requires top-level license comments in every `go.mod`; license compliance check is a `cherald` responsibility.
- **V2-R-09. SIGTERM / graceful-shutdown semantics undefined.** Both modes (`send`, `serve`) lacked a documented exit contract. Applied: §3.1 now specifies trap-SIGTERM, stop-ingress, drain with `HERALD_SHUTDOWN_GRACE` (default 30 s), flush OTel exporter, close Postgres + Redis, exit 0/1 distinction. Second SIGTERM forces exit.
- **V2-R-10. OpenTelemetry env vars not enumerated.** §17.1 said "OTLP/gRPC to a Collector" without naming the standard SDK env vars. Operators couldn't deploy without reading OTel spec. Applied: full table of `OTEL_*` env vars Herald honours, including resource attributes Herald sets by default and the messaging semantic-convention version pin (v1.30+).
- **V2-R-11. Data retention + privacy was unspecified.** No section on how long Herald holds onto subscriber data, dead letters, quarantined messages, or diary entries; no GDPR right-to-erasure / right-to-portability flow. Applied: new **§16.1** with per-class retention defaults (table-driven), `<flavor>herald subscriber forget` and `subscriber export` flows, diary-redaction opt-out, data-sovereignty note.
- **V2-R-12. Operator quickstart missing.** New operators couldn't get from clone to first delivery without reading the whole spec. Applied: **§26.5** with a complete Docker Compose YAML, 6-step bring-up procedure, and a verifiable end-to-end test (curl webhook → Telegram delivery + diary append + OTel trace).
- **V2-R-13. Disaster recovery posture unspecified.** Backups were mentioned but RPO/RTO targets, cold-start procedure, and tested scenarios were absent. Applied: **§26.6** with per-class RPO/RTO table, cold-start procedure, and named integration-test scenarios that MUST live under `tests/dr/`.
- **V2-R-14. Cost considerations missing.** Operators had no order-of-magnitude TCO guidance to choose between self-hosted SMTP vs Resend/Postmark, between Grafana Cloud vs self-host Prometheus, etc. Applied: **§27.3** with infrastructure / ESP / messaging-platform / supply-chain-tooling / observability-backend pricing tables (with `verify-at-deploy` caveat).

30.2 Deferred (out of scope for this self-review)

These findings were noted but require their own focused pass:

- **V2-R-15. ASCII architecture diagram alignment.** Current diagram in §2.3 has minor box-drawing misalignment at the └─ corners under proportional-font renderers. Deferred — fixing requires Mermaid or PlantUML adoption + a renderer pipeline; not blocker for V2.
- **V2-R-16. Heading-depth normalisation.** Some §18.X flavor subsections go 4 levels deep (heading → field → subfield → list). Considered but rejected — depth tracks meaningful structure; flattening loses navigation cues.
- **V2-R-17. "V1 minimum" vs "V1 (MVP)" wording inconsistency.** Cosmetic. Deferred to a docs-only polish PR.

30.3 Method

The pass was a single-author read-through immediately following V2 r1 authoring, focused on internal consistency (prose ? formal definitions), operational completeness (can someone deploy this?), and compliance gaps (GDPR, license, version pinning). Findings rated High when a downstream reader would file a bug, Medium when a future implementer would have to invent a missing detail, Low for stylistic. Only High + Medium findings were applied.

30.4 Audit trail (r2)

- Pre-review commit: V2 r1 at `96b7cc6`.
- r2 commit: `9648545` (one logical commit covering V2-R-01..V2-R-14).
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.

30.5 V3 review log (r3, this revision)

A second self-review pass following r2. Targeted a *different cut* than r1→r2: where r1→r2 closed missing tables and added operational content, r2→r3 closes the second-layer Go types (referenced by §11.0 but not defined) and adds the implementer-grade operational detail that turns "we have a spec" into "we have a buildable spec".

30.5.1 Gaps closed (applied this revision)

- **V3-R-01. Remaining undefined Go types in §11.0.** `Subscriber`, `CloudEventEnvelope`, `TraceContext`, `Branding` (as Go struct, cross-ref §6.3), `ChannelID` (typed string constants), `SubscriberAlias`, `PreferenceSet`/`CategoryPref`/`WorkflowPref`/`QuietHours` (Go-decoded forms of the JSON in §7.2) — all referenced by `OutboundMessage` / `InboundEvent` but never defined. **Applied:** all defined in §11.0 with package + file home pins (`commons/types.go`, `commons/preferences.go`, etc.). Closes the "Channel interface compiles in isolation" gap.
- **V3-R-02. Database migration tooling unspecified.** §26.3 mentioned `<flavor>herald migrate` but never named the tool, file layout, or numbering scheme. **Applied:** new **§9.6** picks `golang-migrate/migrate` (embedded), specifies the `commons_storage/migrations/000001_..._up.sql` layout, the runtime contract (`migrate up/down/status/force/validate`), the role split (`herald_migrator` BYPASSRLS vs `herald_app`), and the forward-compatibility rule (two minor versions).
- **V3-R-03. Worker-pool sizing absent.** No guidance on how many concurrent workers Herald runs. **Applied:** new **§3.4** specifies three independently-sized pools (HTTP ingress, router/dispatch, River channel-delivery) with defaults keyed off `NumCPU`, env-var overrides, and sizing guidance by deployment tier (small / multi-tenant / high-burst).
- **V3-R-04. SIGHUP hot-reload semantics missing.** `<flavor>herald serve` long-running but no way to refresh config without restart. **Applied:** new **§3.4** specifies SIGHUP trap, the safe-to-change vs requires-restart partition (channels/router-workers/log-level/allowlists vs ports/DSN/RLS-policies), the diff-and-validate cycle, and the `digital.vasic.herald.system.config.reloaded` audit event.
- **V3-R-05. Ingress HTTP URL paths never enumerated.** Operators couldn't write reverse-proxy / API-gateway rules without reading the source. **Applied:** new **§5.7** tabulates every `/v1/*` and `/webhooks/*` and `/livez/readyz/startupz/metrics/admin/*` endpoint with method + auth mode, plus the rate-limit defaults and the `/v1/` versioning policy.
- **V3-R-06. Workable-item lifecycle missing.** §8 defined the prefix algorithm but never explained how an `HRD-NNN` actually moves from `Bug:` command → `Issues.md` row → `Fixed.md` resolution. **Applied:** new **§8.3** with full lifecycle ASCII flow, the `workable_items` schema (lightweight pointer table — canonical record stays in

Markdown per Universal §6 human-edit-ability), per-tenant advisory-lock sequence allocation, and the reopen flow that composes with Universal §11.4.55 Reopens.md history.

- **V3-R-07.** `null://` **sandbox channel undocumented.** No first-class way to test routing without hitting real channels. **Applied:** new **§11.14** specifies the URL grammar (`null://?fail_rate=...&latency_ms=...&ceiling=...`), the in-memory ring buffer, the `/admin/null/*` inspector API, the per-environment gate (`CM-NULL-CHANNEL-DISABLED-IN-PROD`), and a recommended test-fixtures pattern for every `commons_messaging/channels/<name>/` package.
- **V3-R-08. Time / clock abstraction missing.** `time.Now()` called directly anywhere makes tests of quiet hours, batching, backoff, TTLs unreliable. **Applied:** new **§3.5** with the `commons/clock` interface (RealClock + FakeClock), the `Default` package-global variable swap pattern, and the `golangci-lint` rule `herald-no-direct-time-now` that fails compilation if anyone calls `time.Now()` outside `commons/clock`.
- **V3-R-09. Machine-readable API specs not committed.** Spec referenced `<flavor>herald openapi` without describing where the specs live. **Applied:** new **§24.1** specifies `docs/api/openapi.v1.yaml` (OpenAPI 3.1, hand-authored from Go handler signatures, with `CM-OPENAPI-DRIFT` parity gate) and `docs/api/asyncapi.v1.yaml` (AsyncAPI 2.6, machine-generated from `commons/events.go`); also lists the CLI helpers (`openapi`, `asyncapi`, `schema dump`).
- **V3-R-10. Outbound idempotency unspecified.** §4.3 covered ingress idempotency but not the case where the channel send itself succeeds-but-loses-response and gets retried. **Applied:** new **§5.4.1** with the `(tenant_id, event_id, channel, channel_address_id)` outbound key, the `outbound_dedup` table (24h TTL — narrower than ingress 7d), and the rule that adapters supporting upstream idempotency (Slack/Stripe-style/Email Message-ID) MUST forward the key while those that don't (raw SMTP/Telegram/Discord) MUST consult `outbound_dedup` in the same River-job transaction.
- **V3-R-11. Per-channel SLO budgets missing.** §17.4 aggregate SLOs hid one bad channel behind good channels. **Applied:** new **§17.4.1** publishes per-channel success and p95-latency targets for each of the 12 channels with a multi-window / multi-burn-rate alert rule (Google SRE Workbook pattern: 2h/14× AND 6h/6× = page; either alone = warn).
- **V3-R-12. AI-agent subscribers conflated with humans.** Spec assumes "subscriber" without distinguishing the runaway-loop risk of agents. **Applied:** new **§7.5** introduces a `subscribers.kind` enum (`human`/`agent`/`service`), the `agent_tokens` table (argon2id-hashed bearer with per-token rate limit + expiry + revocation), the default throttles (60 req/min/token, 3-strike auto-cool-down at 30min), the audit-span attribute (`herald.subscriber.kind="agent"` + `herald.agent.token_id`), and the rule that quiet-hours are ignored for agents by default.

30.5.2 Deferred (out of scope for r3)

- **V3-R-13. Migration-from-existing-systems operator guide.** Mentioned briefly in §24 doc layout but no actual content for migrating from Apprise/Gotify/Mattermost-bridge/etc. Deferred — needs its own dedicated `docs/migration/` content cycle.
- **V3-R-14. Multi-region / global deployments.** What if one tenant spans regions? Out-of-scope; treated as schema-per-tenant + region-pinned operator concern; will resurface when first cross-region customer appears.

30.5.3 Method

Same as r2: single-author read-through immediately following r2 commit, focused on three categories — (a) prose?formal-definition gaps in the type contract layer that §11.0 introduced, (b) operational completeness for an implementer who hasn't read the constitution, (c) correctness boundaries (outbound dedup composition, agent throttling) the architecture document needs to nail down before code lands. Findings rated as before; only High + Medium applied this round.

30.5.4 Audit trail (r3)

- Pre-review commit: V2 r2 at `9648545`.
- r3 commit: covered V3-R-01..V3-R-12 in one logical commit on top of r2.
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.

Statistical context: r1→r2 added ~33 KB to the Markdown source closing 14 findings. r2→r3 closed 12 findings — the curve is flattening, which is the expected pattern (the easier high-leverage gaps go first). A fourth pass would likely yield only stylistic findings; the next high-value pass should come *after* first-implementation cycle surfaces real-world spec gaps that desk-review alone can't find.

30.6 V3 r1 review log (this revision)

This is **not** a desk-review of V2 — it is a *requirements-driven* expansion. V2 was architecturally complete; V3 r1 adds the operator-product story the user described in their V3 brief (project integration, inbound pipeline, LLM dispatch, reply protocol, versioned reports, multi-format attachments) and folds in the **explicit project-side requirements** the user introduced during V3 authoring: 30 s poll cadence, FIFO inbound queue, anti-spam, Investigation-before-Fixing, Claude Code project-named session, `issues/users/attachments/WORKABLE_ITEM_ID/`, multi-format outbound attachments (`.md` + `.html` + `.pdf` + `.docx`).

30.6.1 Findings applied (V3-R1-NN)

- **V3-R1-01. Project integration contract was implicit.** Spec assumed a consuming project existed; never specified the contract. Applied: **§31** with 6-point integration checklist, configuration template, and `test_project_integration.sh` audit gate composing with 11 Universal Constitution mandates.
- **V3-R1-02. Inbound pipeline had no defined cadence.** §3 mentioned "daemon mode listens" but no polling rule. Applied: **§32.2** mandates ≤ 30 s upstream check rate; per-channel table maps to underlying mechanism (long-poll vs Socket Mode vs Gateway vs webhook vs IMAP IDLE).
- **V3-R1-03. No FIFO guarantee for inbound processing.** V2 mentioned River queue but no per-tenant per-channel ordering rule. Applied: **§32.3** specifies advisory-lock-per-`(tenant_id, channel_address_id)` to guarantee strict arrival order.
- **V3-R1-04. Anti-spam unspecified.** Spec referenced "security validation" but no anti-spam layer. Applied: **§32.5** with four-layer anti-spam (per-sender rate / burst detection / reputation / channel frequency); `spam_audit` Redis stream for tuning.
- **V3-R1-05. Investigation-before-Fixing pattern missing.** Spec went straight from "Bug:" command to workable item. Applied: **§18.2.1** mandates intermediate investigation item (`HRD-INV-NNN`); LLM analyzes reproducibility before final item is allocated; rejects duplicates/out-of-scope/non-reproducible.
- **V3-R1-06. LLM/agent dispatch had no concrete first integration.** V2 left LLM dispatch as future work. Applied: **§33** wires Claude Code as the V3 r1 default: `resolve_session(project_name)` algorithm, `<<<HERALD-DISPATCH-v1>>>` envelope, JSON reply schema, pluggable `Dispatcher` interface.
- **V3-R1-07. Claude Code session was assumed to support `--session-name`.** It doesn't. Applied: **§33.2** documents the gap and specifies the resolution algorithm using the

project-name as an anchor-file key, with `--resume <UUID>` as the actual CLI invocation.

- **V3-R1-08. Reply protocol was single-shot.** V2 said "Herald replies" without staging. Applied: **§34** specifies three replies (queued / processing / result) with per-channel edit-in-place mechanics, criticality-driven SLAs, and verbose error reporting that complies with Universal §11.4 anti-bluff.
- **V3-R1-09. Reports vs one-off messages were conflated.** V2 had no separate semantic for documents that change over time. Applied: **§35** introduces report-aware event types, `report_publications` dedup table, Git commit SHA + URL + diff URL embedding, edit-in-place where supported.
- **V3-R1-10. Outbound attachments were single-format.** V2 sent the rendered file in whatever format the channel preferred. Applied: **§36** mandates four-format bundle (`.md` + `.html` + `.pdf` + `.docx`) so subscribers choose. Pandoc generation pipeline (already a dependency) + cache + per-channel delivery rules.
- **V3-R1-11. Attachment storage path for user uploads was implicit.** V2 said "validate and store" without naming the path. Applied: **§18.2.4** mandates `issues/users/attachments/<WORKABLE_ITEM_ID>/` with `attachments_index.md` as source-of-truth + 8-step validation pipeline (download → ClamAV → MIME → extension → size-per-file → size-total → move-on-pass → quarantine-on-fail).
- **V3-R1-12. Criticality classification had no SLA mapping.** V2 had a 4-level scale but no concrete SLA per level. Applied: **§18.2.2** table maps each criticality to specific Reply A/B/C SLA windows + investigation deadline.

30.6.2 Deferred to V3 r2 (the user's follow-up scope)

- **V3-R1-13. Other flavors (§18.3–§18.10) not yet refined for richer channel interaction.** sherald/bherald/dherald/aherald/scherald/iherald/rherald/cherald still carry V2 subscriber-command vocabulary. V3 r2 will add: interactive buttons (Slack Block Kit action buttons / Telegram inline-keyboard callbacks / Discord components / Teams Adaptive Card actions), reaction-based ack/silence, slash-command discovery via `/help`, per-flavor command palettes.
- **V3-R1-14. Full re-export of V1+V2+V3 + push EVERYTHING.** Deferred to V3 r3 as the final polish/release pass.

30.6.3 Audit trail (r3 → V3 r1)

- Pre-V3 commit: V2 r3 at `f4ebba1`.
- V3 r1 commit: covers V3-R1-01..V3-R1-12 in one logical commit on top of V2 r3.
- Inheritance gate before and after: 12 PASS / 0 FAIL. Meta-test: ✓.
- All four Herald mirrors targeted on push.
- V2 metadata bumped to `Status=superseded`; V2 re-exported to keep §11.4.65 invariant green.

Statistical context: V1 was 594 lines; V2 r1 was 1745 lines (+1151); V2 r3 was ~3000 lines (+1255); V3 r1 adds another ~1100 lines. The spec has grown by ~5× in two days of iteration — about half of that is the V3 first-version requirements (§31–§36 + §18.2 expansion), the rest was architectural maturity in V2's three revisions.

30.7 V3 r2 review log (this revision)

V3 r1 specified the operator-product architecture but left every non-`pherald` flavor at V2's text-prefix command vocabulary. V3 r2 closes that gap by adding **per-flavor channel-interaction surfaces** — slash commands, reaction-based quick ops, interactive buttons, modal forms, threads/

forum-topics — so subscribers can interact with each flavor through the channel's native UI affordances.

30.7.1 Findings applied (V3-R2-NN)

- **V3-R2-01. No cross-flavor interaction primitives.** Each flavor invented its own button labels and slash commands → cognitive load for multi-flavor deployments. Applied: **§18.1.1** establishes universal primitives every flavor inherits — /help / /status / /whoami / /silence / /subscribe slash commands with text-prefix fallback, 9-emoji reaction set wired to ack/silence/resolve/escalate/reopen/reclassify/spam, interactive-button conventions, modal-form patterns, thread/forum-topic affinity, capability-degradation rule.
- **V3-R2-02. pherald had no quick-action UI for investigation triage.** Applied: **§18.2.6** adds /investigate, /promote, /reject slash commands; investigation-summary modal (Slack views.open / Discord modal); status-action buttons ([Validate] [Reject] [Need more info] [Reassign]); pinned workable-item card refreshed via §35.
- **V3-R2-03. sherald quick-silence pattern undocumented.** Applied: **§18.3.1** adds [Ack] / [Silence 1h] / [Silence 24h] / [Resolve] / [Runbook] / [Escalate] buttons; one-tap 🔇 silence; /silence-similar for same-service grouping; per-service forum topic on Telegram.
- **V3-R2-04. bherald had no PR/build cross-flavor handoff.** Applied: **§18.4.1** introduces 🐛 reaction → promotes failed-build to pherald bug investigation (single-emoji cross-flavor handoff); [Retry] / [Snooze flaky] buttons; /triage slash for security-scan decisions; coverage-delta card; per-branch build digest forum topic.
- **V3-R2-05. dherald lacked confirmation flow for destructive ops.** Applied: **§18.5.1** mandates Slack modal with release_note_signoff field on [Rollback] and [Hold env]; [Promote canary] button; environment status card; [Page on-call] 🚨 reaction freezes deploys to env.
- **V3-R2-06. aherald did not specify per-channel ack semantics.** Applied: **§18.6.1** documents [Ack] carrying an implicit escalation_window timer; /aherald group and /aherald inhibit slash commands; Email reply-by-keyword (button-less channel still gets ack/silence/resolve/escalate via parsed keywords); Telegram inline-keyboard expand button avoids message-length limits.
- **V3-R2-07. scherald reminders had no snooze UX.** Applied: **§18.7.1** adds [Snooze 1h] / [Snooze 1d] / [Snooze custom] / [Cancel] buttons; /remind <subject> in <duration> with permissive duration parser; /digest <daily|weekly|monthly> on-demand digest; 🔁 reaction re-schedules with the same delta; "[I read this]" reaction feeds digest-section deprioritisation.
- **V3-R2-08. iherald lacked a "command room" pattern.** Applied: **§18.8.1** introduces incident command room (Slack thread / Discord forum-channel post / Telegram forum-topic / Teams channel); [Take IC] atomic compare-and-swap on incidents.commander_id; [Acknowledge page] button works on Email + WhatsApp via signed URLs; /postmortem slash generates §36 multi-format template seeded from incident_events; real-time timer card refreshed every 60 s.
- **V3-R2-09. rherald/cherald had no compliance-grade UX.** Applied: **§18.9.1** adds release card + [Promote to staging/prod] / [Hold] buttons with Slack modal confirmation requiring release_note_signoff field; dependency-update [Approve] / [Reject] / [Defer 7d]; /cve and /release-notes slash commands; ✅/❌ operator reactions drive cherald triage queue; breaking-changes thread. **§18.10.1** mandates restricted-by-default channels for cherald; [Acknowledge violation] / [Mark false-positive] / [Open ticket] (cross-flavor handoff to pherald); operator-only /audit <subscriber-id> (which itself emits a compliance.audit.review event — audits-of-audits are audited); 🔒 reaction marks

events sensitive; email-as-system-of-record with `keep`/`ignore` keyword triage; one forum topic per compliance domain.

30.7.2 Audit trail (r2)

- Pre-review commit: V3 r1 at `e26a8dc`.
- r2 commit: covers V3-R2-01..V3-R2-09 in one logical commit on top of V3 r1.
- Inheritance gate before and after: 12 PASS / 0 FAIL.
- All four Herald mirrors targeted on push.
- V2 untouched in r2 (already at Status=superseded from r1 commit).

30.7.3 What remains for r3

- **V3-R3-01. Full re-export of V1 + V2 + V3** to keep §11.4.65 universal-export invariant green across the full spec set.
- **V3-R3-02. Polish pass** — minor refinements surfaced during r2 authoring (e.g., the §31 mandatory-integration list and the §18.x.1 channel-interaction tables share an implicit "every channel supports degradation" assumption that should be explicit; the Roadmap §27.1 table doesn't yet mention the V3 additions).
- **V3-R3-03. Touch test on parent-project docs** — check whether `README.md`/`CLAUDE.md`/`AGENTS.md`/`HERALD_CONSTITUTION.md` reference the old `specification.md` path and need redirection to V3.
- **Final commit + push EVERYTHING** to all 4 Herald mirrors (plus constitution mirrors if any constitution edit is needed).

30.8 V3 r3 review log (this revision — final polish)

V3 r3 is the closing-out commit for the user-defined "after r1 + r2, refining and improvements and re-export and push EVERYTHING" ask. Where r1 added the operator-product layer and r2 refined every flavor's channel interactions, r3 closes the cross-doc sync gap and finalises the spec evolution chain.

30.8.1 Findings applied (V3-R3-NN)

- **V3-R3-01. Stale path string in V3 §23 spec-change anchor.** §23 still referenced `specification.V2.md` even though V3 supersedes V2. Applied: rewrote §23 to point at `specification.V3.md` and added a note clarifying that the I7-gated enforcement anchor is the *phrase* `comprehensive planning and implementation`, not the path — so the gate stayed green throughout the path-string churn.
- **V3-R3-02. Parent docs (README/CLAUDE.md/AGENTS.md/ HERALD_CONSTITUTION.md) referenced the pre-V1-rename path `specification.md`.** Three of the four still said `MVP spec (TBD)` even though V3 is ~3900 lines and architecturally complete. Applied: all four files updated:
    - **README.md** — repo-layout block now shows `specification.V3.md` + `archive/specification.V1.md` + `archive/specification.V2.md`. Read-order item 7 now points at V3. Status updated to describe the current spec.
    - **CLAUDE.md** — all four `docs/specs/mvp/specification.md` occurrences replaced with …`specification.V3.md` (ToC entry, read-order, spec-change-rule heading, body reference).
    - **AGENTS.md** — same path-replacement across all occurrences.
    - **HERALD_CONSTITUTION.md** — §106 forensic anchor + §"Notes" section both updated; Notes now reflects "V3 is active; V1+V2 in archive/".

- **V3-R3-03. Metadata revisions out of date in all four parent docs.** Applied: bumped Revision on README (1→2), CLAUDE.md (1→2), AGENTS.md (2→3), HERALD_CONSTITUTION.md (2→3); refreshed Status summary / Fixed / Fixed summary on each to capture the V3-path-sync work.

30.8.2 Audit trail (r3)

- Pre-review commit: V3 r2 at `f8b8073`.
- r3 commit: covers V3-R3-01..V3-R3-03 + parent-doc updates + .html/.pdf regen for V3 and all four parent docs.
- Inheritance gate before and after: 12 PASS / 0 FAIL. I7a/b/c specifically re-verified (the path-string change could have affected them in principle but does not because I7 grep keys on the phrase, not the path).
- All four Herald mirrors targeted on push.
- Constitution submodule untouched in r3 (no constitution-level change needed; the §11.4.61 + §11.4.65 invariants are satisfied by the in-repo re-export).
- V1 + V2 untouched in r3 (already current in `archive/`).

30.8.3 What r3 does NOT do

- Does NOT refactor V3 body content. Polish was deliberately limited to cross-doc sync to keep r3 commits surgical.
- Does NOT bump V1 or V2 revision/status (they're frozen archives).
- Does NOT touch the constitution submodule (out of scope; the parent constitution already carries the §11.4.61/§11.4.65 mandates Herald composes with).

30.8.4 Spec-evolution chain — final state after r3

Version	Path	Revision	Status	Lines (md)	Role
V1	`docs/specs/mvp/archive/specification.V1.md`	3	superseded	~590	Historical (first cut + R-NN audit baseline)
V2	`docs/specs/mvp/archive/specification.V2.md`	4	superseded	~3000	Historical (architectural maturity over three revisions r1-r3)
V3	`docs/specs/mvp/archive/specification.V3.md`	13	superseded	~5000	Historical (operator-product layer + nine refined flavors + Waves 2–7)
V4	`docs/specs/mvp/specification.V4.md`	1	active	~5200	**Current** — adds participant identity/attribution/@-tagging (§7.6/§8.4/§34.7) + natural-language intent recognition (§32.6.B/§33.8/§34.8)

Anyone reading the spec set starts at V4. V1 + V2 + V3 exist for traceability when a later change ID needs cross-reference to a prior audit log (§30.5) or ID-system (§30.1).

Sources verified

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). Every design decision in this specification that references an external service contract was cross-referenced against the LATEST official documentation of that service before publication.

Last verified: 2026-05-28

Source	URL / path	Authored / verified (which §s)
Telegram official Bot API documentation	https://core.telegram.org/bots/api	§11.1 (Telegram channel `Capabilities` declaration); §11.4 (outbound idempotency — Telegram NOT supporting upstream `Idempotency-Key`, requiring `outbound_dedup` Postgres-side); §32.2 (`getUpdates` long-poll 25 s timeout + 30 s safety-net timer); §32.9 (anti-echo-loop self-filter — `bot.Me.Username` via `getMe`); §43 (per-chat-type routing matrix); §"Costs" table (Bot API free tier).
Slack Web API method index	https://api.slack.com/methods	§11.2 (Slack channel `Capabilities`); §11.4 (Slack `idempotency_key` extension — adapter MUST forward); §11.9 (Slack channel-vs-DM addressing).
Slack Events API + Socket Mode	https://api.slack.com/apis/connections/socket + https://api.slack.com/apis/events-api	§32.2 (Slack inbound — Socket Mode WebSocket + Events API webhook variants); §43 (Slack routing matrix).
Email RFCs — RFC 5322 (Message Format), RFC 6376 (DKIM), RFC 7208 (SPF), RFC 7489 (DMARC)	https://www.rfc-editor.org/rfc/rfc5322 / rfc6376 / rfc7208 / rfc7489	§11.4 (Email `Message-ID` header forward for outbound idempotency); §11.4 (SMTP-vs-Resend choice); §"Costs" table (SMTP free; Resend transactional rates).
Anthropic — Claude Code documentation	https://docs.anthropic.com/claude-code	§33 (LLM/agent dispatch architecture); §33.2 (session-resolution algorithm — `<workdir>/.herald/claude-code/sessions/<project>.session` anchor file); §33.3 (`<<<HERALD-REPLY>>>` envelope schema); §33.5 (`claude --resume <UUID> --print <prompt>` non-interactive batch invocation); §33.7 (Opus pin via `--model claude-opus-4-7` argv form).
CloudEvents v1.0 specification	https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md	§10 / §32 (inbound CloudEvents v1.0 envelope contract — `id` / `source` / `type` / `specversion` / `datacontenttype` / `time` / `data` fields); §"Events processed" table semantics.
PostgreSQL official documentation	https://www.postgresql.org/docs/15/	§6/§9 schema (`events_processed`, `subscribers`, `subscriber_aliases`, `outbound_delivery_evidence`,

Source	URL / path	Authored / verified (which §s)
		`constitution_state`, `constitution_bindings`, `claude_code_sessions`); §"RLS" (Row-Level Security on every multi-tenant table, gated by `app.current_tenant_id` GUC).
Redis official documentation	https://redis.io/docs/	§"Hot idempotency cache" (SETNX with 24h TTL default); §"JWKS cache" (5 min TTL default).
OAuth 2.0 / OIDC + RFC 7517 (JWK) + RFC 7518 (JWA) + RFC 7519 (JWT)	https://www.rfc-editor.org/rfc/rfc7517 / rfc7518 / rfc7519 + https://openid.net/specs/openid-connect-core-1_0.html	§"Auth" (JWT verification via JWKS; RS256 / ES256 algorithm selection; `iss` / `aud` / `exp` / `nbf` claim validation).
HelixConstitution §11.4.99 (this document's authority)	`<parent>/constitution/Constitution.md` §11.4.99 (HelixConstitution commit `c640947`)	This footer pattern + cadence semantics.

Re-verification cadence (per §11.4.99 (C)): This document aggregates contracts of many external services. Per §11.4.99 (D): Telegram + Slack + Email sections (`§11.1`, `§11.2`, `§11.4`, `§32.2`, `§32.9`, `§43` rows) → **90-day staleness** (next due **2026-08-26**); Anthropic / Claude Code sections (`§33.x`) → **180-day staleness** (next due **2026-11-24**); CloudEvents / Postgres / Redis / OAuth / OIDC sections → **365-day staleness** (next due **2027-05-28**) — these protocols are stable + standards-track, breaking changes are years apart. Re-verify earlier on: any service changelog announces a breaking change; an operator-facing report surfaces a contradiction; Herald v1.0.0 release boundary.

Note on §11.4.99 application to specifications. §11.4.99 was authored for OPERATOR-actionable instruction documents. This specification is design-of-record, not operator instructions — operator setup is delegated to the `docs/guides/*.md` family. The cross-reference table above documents the external-contract sources the spec's design choices depend on; if a spec section is REVISED to add new operator-actionable content (the spec body documenting "how operators set up X"), the new content MUST be covered by the table on the same revision.